

FROM ZERO TO PRACTICE

LLM 基础知识概念大全

从概率、Token 与 Transformer，到训练目标推导、对齐算法、RAG、Agent、推理部署与安全评测

面向转入大语言模型领域的系统教科书与概念手册

版本：2026.07 · 教科书增强版

前言：这本手册解决什么问题

大语言模型领域的困难，并不只是知识多，而是同一个词经常同时出现在数学、算法、训练系统和产品工程四种语境中。例如，“上下文长度”既影响注意力复杂度和显存，也影响提示设计、检索策略与产品成本；“微调”可能指全参数训练，也可能指 LoRA、持续预训练、监督微调或偏好优化。只记住名词，很容易形成互相矛盾的理解。

本手册以“概念之间的关系”为主线，目标是建立一张完整的知识地图，而不是罗列模型新闻。全文分为六层：

1. **表示层**：文本如何变成 Token、向量和概率分布；
2. **模型层**：Transformer 如何计算，参数究竟存储了什么；
3. **训练层**：预训练、SFT、RLHF、DPO、蒸馏和微调分别解决什么问题；
4. **推理层**：采样、KV Cache、量化、批处理和服务系统如何工作；
5. **应用层**：RAG、工具调用、Agent、记忆与工作流；
6. **可信层**：评测、幻觉、安全、隐私和上线监控。

阅读建议

第一次阅读可先看每章的“核心概念”“常见误区”和章末总结；第二遍再看公式。数学公式的作用是消除歧义，不要求一次记住全部推导。对转行学习者而言，最重要的是能回答三个问题：**它是什么、为什么需要它、它与相邻概念有什么区别。**

教科书增强版新增内容

本版在保留全局知识地图和术语覆盖的基础上，重点重写了数学基础、Transformer 前向链、Attention、RoPE 与 KV Cache、预训练、SFT、偏好优化、LoRA/QLoRA、解码、RAG、Agent、推理性能、量化、分布式训练与评测章节。核心章节均采用“定义-直觉-逐步推导-数值例题-工程含义-练习与解答”的结构。第一次阅读可跳过紫色推导框，第二遍再完整推演。

全局知识地图



图 1: LLM 学习的主干：先理解模型，再理解训练、系统与应用。

目录

前言：这本手册解决什么问题	i
全局知识地图	ii
第一部分 第一部分：建立共同语言	1
第一章 什么是大语言模型	2
1.1 从语言模型到大语言模型	2
1.2 基础模型、语言模型、聊天模型与推理模型	2
1.3 生成能力从哪里来	3
1.4 能力、知识与行为的区别	3
1.5 LLM 系统的四层边界	3
第二章 必备数学：从概率到反向传播	4
2.1 标量、向量、矩阵与张量	4
2.1.1 用下标理解矩阵乘法	4
2.1.2 Shape 是理解 LLM 代码的第一语言	5
2.2 概率、条件概率与自回归分解	5
2.2.1 联合概率与链式法则	5
2.2.2 条件独立与建模假设	6
2.3 最大似然、负对数似然与语言模型损失	6
2.3.1 为什么取对数不会改变最优解	7
2.4 Logit、Softmax 与温度	7
2.4.1 Softmax 只关心相对差值	7
2.4.2 温度如何改变分布	7
2.5 熵、交叉熵与 KL 散度	8
2.5.1 熵：分布自身的不确定性	8
2.5.2 交叉熵：用另一个分布编码真实分布的代价	8
2.5.3 KL 散度：两个分布的额外编码代价	8
2.5.4 困惑度	8
2.5.5 Label Smoothing	9
2.6 微积分与链式法则	9
2.6.1 导数与梯度	9
2.6.2 链式法则	9
2.7 Softmax-交叉熵梯度的完整推导	10

2.7.1 第一步：求 Softmax 的偏导	10
2.7.2 第二步：链式求损失对 logit 的梯度	10
2.7.3 输出层梯度	11
2.8 梯度下降、Mini-batch 与梯度累积	11
2.8.1 梯度累积为什么等价于大 batch	11
2.9 Adam 与 AdamW 的逐项解释	12
2.9.1 学习率调度	12
2.10 梯度裁剪、消失与爆炸	12
2.11 浮点数、混合精度与数值稳定性	13
2.11.1 混合精度训练	13
2.11.2 Loss Scaling 的原理	13
2.12 期望、方差与无偏估计	14
2.13 本章综合例题	14
第三章 文本如何进入模型：Token、Tokenizer 与 Embedding	16
3.1 为什么不能直接按“字”处理	16
3.2 Token、词表与 Token ID	16
3.3 BPE、WordPiece、Unigram 与 SentencePiece	16
3.3.1 BPE	16
3.3.2 WordPiece	16
3.3.3 Unigram	17
3.3.4 SentencePiece	17
3.4 字节级 Tokenization 与特殊 Token	17
3.5 Chat Template 不是普通字符串拼接	17
3.6 Embedding：从离散 ID 到连续向量	17
3.7 Padding、Attention Mask 与 Label Mask	18
3.8 Tokenizer 对模型效果的影响	18
第四章 神经网络基础：参数、层、激活与归一化	19
4.1 参数与超参数	19
4.2 线性层与多层感知机	19
4.3 激活函数	19
4.4 残差连接	20
4.5 LayerNorm 与 RMSNorm	20
4.6 Pre-Norm 与 Post-Norm	20
4.7 Dropout、正则化与训练/推理模式	20
第二部分 第二部分：Transformer 的内部机制	21
第五章 Transformer 总览：从 Token 到下一个 Token	22

5.1 三种经典架构	22
5.2 一条完整的 Decoder-only 前向链	22
5.2.1 第一步: Token Embedding	22
5.2.2 第二步: 位置处理	23
5.2.3 第三步: 堆叠 Transformer Block	23
5.2.4 第四步: 输出头	23
5.3 隐藏状态究竟表示什么	24
5.4 Attention 子层的 shape	24
5.5 MLP/FFN 子层	24
5.5.1 为什么 MLP 参数通常很多	24
5.6 残差连接为什么重要	25
5.7 LayerNorm、RMSNorm 与 Pre-Norm	25
5.8 Teacher Forcing: 一次训练全部位置	25
5.9 为什么训练可并行, 生成仍串行	26
5.10 参数量的系统估算	26
5.10.1 Attention 参数	26
5.10.2 MLP 参数	26
5.10.3 总参数	27
5.11 FLOPs 的粗略估算	27
5.12 模型参数“存储知识”是什么意思	27
5.13 训练模式与推理模式	28
5.14 本章综合练习	28
第六章 注意力机制: Q、K、V 的完整推导	29
6.1 从固定权重到动态信息路由	29
6.2 从输入到 Q、K、V	29
6.3 为什么使用点积相似度	30
6.3.1 缩放因子 $1/\sqrt{d_k}$ 的推导	30
6.4 缩放点积注意力	31
6.4.1 逐行展开	31
6.5 一个可手算的数值例子	31
6.6 因果 Mask 与 Padding Mask	32
6.6.1 因果 Mask	32
6.6.2 Padding Mask	32
6.6.3 低精度实现细节	32
6.7 多头注意力	32
6.7.1 批量 shape	33
6.7.2 多头不等于“每头对应一种固定语言关系”	33
6.8 MHA、MQA 与 GQA	33

6.8.1 Multi-Head Attention (MHA)	33
6.8.2 Multi-Query Attention (MQA)	33
6.8.3 Grouped-Query Attention (GQA)	33
6.8.4 参数量公式	34
6.8.5 KV Cache 缩减比例	34
6.9 Attention 的复杂度分解	34
6.9.1 QKV 与输出投影	34
6.9.2 分数矩阵与 Value 聚合	35
6.9.3 显存与 IO 不等于 FLOPs	35
6.10 FlashAttention: 在线 Softmax 的关键思想	35
6.10.1 为什么 Softmax 分块不容易	35
6.10.2 在线 Softmax 合并公式	35
6.11 训练与推理中的注意力差异	36
6.11.1 训练	36
6.11.2 Prefill	36
6.11.3 Decode	36
6.12 稀疏、局部与线性注意力	36
6.13 注意力权重能否解释模型	36
6.14 本章综合练习	37
第七章 位置编码、上下文窗口与 KV Cache	38
7.1 为什么自注意力本身不知道顺序	38
7.2 绝对位置编码	38
7.2.1 正弦位置编码	39
7.2.2 为什么正弦编码包含相对位置信息	39
7.3 RoPE: 把位置写入 Q 和 K	39
7.3.1 RoPE 的相对位置性质	39
7.3.2 多频率 RoPE	40
7.4 相对位置偏置与 ALiBi	40
7.5 上下文窗口的三个不同含义	40
7.6 长度外推与位置缩放	41
7.7 KV Cache 为什么存在	41
7.7.1 没有 Cache 的重复计算	41
7.8 KV Cache 显存公式	41
7.8.1 每新增一个 Token 的 Cache	42
7.9 Prefill 与 Decode	42
7.9.1 Prefill	42
7.9.2 Decode	42
7.10 Prefix Cache 与 Prompt Cache	43

7.11 PagedAttention 与缓存碎片	43
7.12 滑动窗口、Cache 驱逐与摘要	43
7.13 长上下文为何容易失效	43
7.13.1 Lost in the Middle	43
7.13.2 信号稀释	44
7.13.3 组合而非检索	44
7.13.4 位置外推	44
7.14 如何评测长上下文	44
7.15 本章综合练习	44
第八章 现代架构组件: MoE、长序列与模型家族	46
8.1 Dense 模型与 MoE 模型	46
8.2 Router、Top-k 与负载均衡	46
8.3 模型家族的命名维度	46
8.4 Encoder、Decoder 与嵌入模型不能混用	47
8.5 替代 Transformer 的方向	47
第三部分 第三部分: 模型是如何训练出来的	48
第九章 预训练: 数据、目标、计算与 Scaling Law	49
9.1 预训练学到的是什么	49
9.2 从原始文档到训练 Token	49
9.3 文档边界、EOS 与 Packing	50
9.4 Token、样本、Step 与 Epoch	50
9.5 数据混合与采样权重	50
9.6 为什么去重重要	51
9.7 质量过滤的偏差	51
9.8 训练计算量: 为什么常见 $6ND$	51
9.8.1 $6ND$ 的边界	52
9.9 Model FLOPs Utilization	52
9.10 Scaling Law 的基本思想	52
9.10.1 固定计算预算的权衡	52
9.11 学习率、Batch 与训练稳定性	53
9.11.1 Loss Spike 的处理	53
9.12 持续预训练与领域预训练	53
9.13 数据污染与可记忆性	54
9.14 预训练评测	54
9.15 本章综合练习	54
第十章 监督微调: 从会续写到会按指令回答	56

10.1 SFT 改变的是行为分布	56
10.2 SFT 数据的数学形式	56
10.2.1 多轮对话	56
10.3 Loss Mask: 哪些 Token 应贡献梯度	57
10.3.1 为什么不一定训练 User Token	57
10.4 按 Token 平均还是按样本平均	57
10.5 Packing 与样本隔离	58
10.5.1 跨样本注意力问题	58
10.6 数据质量: SFT 的核心瓶颈	58
10.7 数据混合与能力保持	58
10.8 全参数微调与参数高效微调	59
10.9 灾难性遗忘	59
10.10 拒绝采样与合成数据	59
10.11 SFT 与知识注入	60
10.12 训练超参数的作用	60
10.12.1 学习率	60
10.12.2 Epoch	60
10.12.3 序列长度	60
10.12.4 Warmup 和 Weight Decay	60
10.13 SFT 的评测闭环	60
10.14 一个最小 SFT 实验设计	61
10.15 本章综合练习	61
第十一章 偏好对齐: 从策略梯度到 PPO、DPO 与 GRPO	62
11.1 为什么最大似然不能表达全部偏好	62
11.2 序列概率与 Token 概率	62
11.3 奖励模型与 Bradley-Terry 偏好概率	63
11.3.1 为什么只看分数差	63
11.3.2 奖励模型的常见偏差	63
11.4 把文本生成写成强化学习	63
11.5 策略梯度的完整推导	64
11.6 Baseline 为什么降低方差且不引入偏差	64
11.7 Return、Value、Q 与 Advantage	65
11.7.1 Temporal-Difference 误差与 GAE	65
11.8 重要性采样与策略概率比	65
11.9 PPO 裁剪目标逐情况解释	65
11.9.1 当 $\hat{A}_t > 0$	65
11.9.2 当 $\hat{A}_t < 0$	66
11.10 完整 PPO 损失的组件	66

11.11 KL 正则化与参考模型	66
11.12 KL 正则化最优策略的推导	67
11.13 DPO 的逐步推导	67
11.13.1 DPO 的直觉	68
11.13.2 β 的含义	68
11.14 DPO 与 SFT 梯度的差异	68
11.15 RLOO: 留一法基线	68
11.16 GRPO: 组内相对优势	68
11.16.1 组内标准化的作用	69
11.16.2 Token 级优势分配	69
11.17 PPO、DPO、RLOO 与 GRPO 对比	69
11.18 可验证奖励 (RLVR)	69
11.19 Reward Hacking 与 Goodhart 定律	70
11.20 训练监控指标	70
11.21 一个玩具 DPO 例子	70
11.22 本章综合练习	71
第十二章 参数高效微调、QLoRA 与模型合并	72
12.1 为什么需要参数高效微调	72
12.2 LoRA 的低秩假设	72
12.3 为什么低秩可能有效	73
12.4 LoRA 参数量	73
12.5 缩放 α/r	74
12.6 初始化为什么通常令初始更新为零	74
12.7 LoRA 前向与合并	74
12.8 Target Modules 如何选择	74
12.8.1 不同模块的直觉	75
12.9 Rank、Alpha、Dropout 与学习率	75
12.9.1 Rank	75
12.9.2 Alpha	75
12.9.3 LoRA Dropout	75
12.9.4 学习率	75
12.10 LoRA 节省了什么, 没有节省什么	75
12.11 QLoRA 的计算链	76
12.11.1 QLoRA 不是 “用 INT4 训练全部权重”	76
12.12 NF4、分组量化与双重量化	76
12.13 量化误差如何影响微调	76
12.14 Adapter、Prefix Tuning 与 Prompt Tuning	76
12.14.1 Adapter	76

12.14.2 Prefix Tuning	77
12.14.3 Prompt Tuning	77
12.15 多个 LoRA 的组合	77
12.16 模型合并	77
12.17 微调、CPT、RAG 与 Prompt 的选择	78
12.18 LoRA 实验最小闭环	78
12.19 本章综合练习	78
第十三章 蒸馏、剪枝与训练数据合成	80
13.1 知识蒸馏	80
13.2 序列级蒸馏与合成数据	80
13.3 数据飞轮	80
13.4 剪枝与稀疏化	80
13.5 彩票假设与低秩/稀疏假设	81
第四部分 第四部分：让模型完成任务	82
第十四章 提示工程与解码：从概率分布到最终文本	83
14.1 Prompt 的本质：给条件概率增加条件	83
14.2 Chat Template 与指令层级	83
14.3 零样本、少样本与 In-Context Learning	84
14.4 自回归生成算法	84
14.5 贪心解码	84
14.6 Beam Search	85
14.6.1 长度偏差	85
14.7 随机采样与温度	85
14.8 Top-k Sampling	85
14.9 Top-p / Nucleus Sampling	85
14.10 Min-p、Typical Sampling 与其他截断	86
14.11 采样顺序与实现差异	86
14.12 重复惩罚	86
14.12.1 Repetition Penalty	86
14.12.2 Frequency/Presence Penalty	86
14.13 停止条件与 EOS	86
14.14 结构化与约束解码	87
14.15 为什么相同参数仍可能不完全确定	87
14.16 按任务选择解码策略	87
14.17 Prompt 失败时先判断是哪一层	88
14.18 本章综合练习	88

第十五章 推理能力: CoT、搜索、验证与 Test-Time Compute	89
15.1 Chain-of-Thought	89
15.2 Scratchpad 与最终答案分离	89
15.3 Self-Consistency	89
15.4 问题分解与规划	89
15.5 Tree of Thoughts 与显式搜索	90
15.6 程序辅助与工具验证	90
15.7 Process Reward 与 Outcome Reward	90
15.8 Test-Time Compute	90
15.9 推理失败的常见类型	90
第十六章 RAG: 从检索公式到可追溯生成	92
16.1 RAG 解决什么问题	92
16.2 离线索引流程	92
16.3 Chunking: 检索与上下文的基本单元	93
16.3.1 Overlap 的权衡	93
16.4 Sparse Retrieval 与 BM25	93
16.5 Dense Retrieval	94
16.6 Embedding 模型如何训练	94
16.6.1 In-batch Negatives	94
16.6.2 Hard Negatives	94
16.7 ANN 与向量索引	95
16.8 Hybrid Retrieval	95
16.9 Query Rewrite、Multi-Query 与 HyDE	95
16.10 Metadata Filter 与权限	95
16.11 Reranker	96
16.12 Context Construction	96
16.13 生成与引用	96
16.14 检索评测指标	96
16.14.1 Recall@k	96
16.14.2 Precision@k	97
16.14.3 MRR	97
16.14.4 nDCG	97
16.15 生成层评测	97
16.16 端到端错误树	97
16.17 RAG 安全	98
16.17.1 间接提示注入	98
16.17.2 索引污染	98
16.17.3 权限泄漏	98

16.18 本章综合练习	98
第十七章 Agent: 状态、工具、规划、记忆与环境	100
17.1 什么是 Agent	100
17.2 Prompt Chain、Workflow 与 Agent 的区别	100
17.3 部分可观察环境	101
17.4 工具调用与 Function Calling	101
17.5 工具设计原则	101
17.5.1 语义单一	101
17.5.2 参数可验证	101
17.5.3 最小权限	102
17.5.4 幂等与可重试	102
17.5.5 结果结构化	102
17.6 ReAct 循环	102
17.7 规划方式	102
17.7.1 Plan-then-Execute	102
17.7.2 Interleaved Planning	102
17.7.3 Hierarchical Planning	102
17.7.4 Search-based Planning	102
17.8 状态管理	103
17.9 记忆三类	103
17.9.1 Working Memory	103
17.9.2 Semantic Memory	103
17.9.3 Episodic Memory	103
17.10 Reflection 与 Verifier	103
17.11 Harness: 把模型变成可运行 Agent	104
17.12 Sandbox 与副作用控制	104
17.13 终止条件与预算	104
17.14 错误恢复	105
17.15 Multi-Agent 的边界	105
17.16 Agent 评测	105
17.16.1 任务结果	105
17.16.2 效率	105
17.16.3 鲁棒性	105
17.16.4 安全	106
17.16.5 可复现性	106
17.17 本章综合练习	106
第十八章 推理性能: 延迟、吞吐、批处理与显存	107

18.1 端到端延迟的分解	107
18.2 核心指标	107
18.2.1 TTFT	107
18.2.2 TPOT 与 ITL	107
18.2.3 吞吐	108
18.2.4 Goodput	108
18.3 权重显存	108
18.4 推理显存的组成	108
18.5 FLOPs、带宽与算术强度	108
18.6 为什么 Prefill 更偏 Compute-bound	109
18.7 为什么 Decode 更偏 Memory-bound	109
18.8 KV Cache 对 Decode 的影响	109
18.9 Static、Dynamic 与 Continuous Batching	110
18.9.1 Static Batching	110
18.9.2 Dynamic Batching	110
18.9.3 Continuous Batching	110
18.10 Batch 增大为何吞吐升高、延迟也可能升高	110
18.11 排队论直觉与 Little 定律	110
18.12 Prefill/Decode 分离	110
18.13 Prefix Cache	111
18.14 Speculative Decoding	111
18.14.1 为什么能保持目标分布	111
18.14.2 加速取决于什么	111
18.15 量化、并行与内核优化的作用层级	112
18.16 容量规划的简化模型	112
18.17 Benchmark 应如何报告	112
18.18 本章综合练习	113
第十九章 量化：从实数权重到 INT8/INT4	114
19.1 量化解决什么问题	114
19.2 均匀仿射量化	114
19.3 对称与非对称量化	115
19.3.1 对称量化	115
19.3.2 非对称量化	115
19.4 量化粒度	115
19.5 量化误差：Rounding 与 Clipping	115
19.5.1 均匀误差近似	116
19.6 权重量化为何相对容易	116
19.7 Weight-only MatMul 的计算	116

19.8 PTQ 与 QAT	116
19.8.1 Post-Training Quantization (PTQ)	116
19.8.2 Quantization-Aware Training (QAT)	116
19.9 GPTQ 的二阶误差直觉	117
19.10 AWQ 的激活感知直觉	117
19.11 SmoothQuant: 把激活难度迁移给权重	117
19.12 Outlier Channels	117
19.13 KV Cache 量化	118
19.14 量化并不等于实际加速	118
19.15 精度评测	118
19.16 本章综合练习	119
第二十章 分布式训练: DP、ZeRO/FSDP、TP、PP 与并行组合	120
20.1 为什么单卡放不下训练	120
20.2 数据并行 (DP/DDP)	120
20.2.1 全局 Batch	121
20.2.2 为什么参数保持一致	121
20.3 Ring All-Reduce 通信量	121
20.4 梯度 Bucket 与通信重叠	121
20.5 ZeRO 的基本思想	121
20.6 FSDP 的执行过程	122
20.7 Tensor Parallel: 从矩阵乘法推导	122
20.7.1 列并行 (按输出维切分)	122
20.7.2 行并行 (按输入维切分)	123
20.7.3 MLP 中的配对	123
20.8 Attention 的 Tensor Parallel	123
20.9 Pipeline Parallelism	123
20.9.1 GPipe Bubble	123
20.9.2 1F1B 调度	124
20.9.3 Stage 划分	124
20.10 Sequence Parallelism	124
20.11 Context Parallelism	124
20.12 Expert Parallelism	124
20.13 3D/4D 并行组合	125
20.14 Activation Checkpointing	125
20.15 CPU/NVMe Offload	125
20.16 通信与计算重叠	125
20.17 训练吞吐指标	126
20.18 分布式故障模式	126

20.18.1 Collective 顺序不一致	126
20.18.2 Shape/Dtype 不一致	126
20.18.3 OOM 不同步	126
20.18.4 梯度不同步	126
20.18.5 Checkpoint 分片不兼容	126
20.18.6 通信/计算竞争	126
20.19 并行策略选择	126
20.20 本章综合练习	127
第二十一章 从实验到上线：模型服务与 LLMOps	128
21.1 模型生命周期	128
21.2 离线评测、Shadow、Canary 与 A/B Test	128
21.3 可观测性	128
21.4 缓存	128
21.5 回退与降级	129
21.6 成本模型	129
21.7 版本兼容	129
第五部分 第六部分：评测、可靠性与安全	130
第二十二章 如何正确评测 LLM：指标、统计与污染	131
22.1 评测首先是目标定义	131
22.2 能力评测与系统评测	131
22.3 分类指标	132
22.4 Exact Match、Token F1 与执行评测	132
22.5 Pass@k 的推导	132
22.6 采样预算必须公平	133
22.7 均值、方差与标准误	133
22.8 准确率的置信区间	133
22.9 配对评测与 Bootstrap	134
22.10 人工评测	134
22.11 LLM-as-a-Judge	134
22.12 成对比较与 Bradley-Terry 模型	134
22.13 多维指标与 Pareto 前沿	135
22.14 评测集设计	135
22.15 切片分析	135
22.16 数据污染	135
22.17 Prompt 与解码敏感性	136
22.18 线上 A/B Test	136

22.19 本章综合练习	136
第二十三章 幻觉、事实性与不确定性	138
23.1 什么是幻觉	138
23.2 为什么会幻觉	138
23.3 校准	138
23.4 拒答与选择性预测	138
23.5 减少幻觉的手段	138
23.6 引用不是事实性的充分条件	139
第二十四章 LLM 与 Agent 安全基础	140
24.1 威胁建模	140
24.2 Prompt Injection	140
24.3 Jailbreak 与安全对齐绕过	140
24.4 越权工具调用	140
24.5 数据与隐私风险	141
24.6 模型供应链	141
24.7 训练阶段攻击	141
24.8 模型攻击与知识产权	141
24.9 安全架构的分层防御	141
第二十五章 红队、鲁棒性与上线安全评测	143
25.1 红队目标	143
25.2 测试维度	143
25.3 攻击成功率与正常能力	143
25.4 自动化红队	143
25.5 安全回归集	143
25.6 上线护栏	144
第六部分 第七部分：扩展能力与实践路线	145
第二十六章 多模态大模型基础	146
26.1 多模态的核心问题	146
26.2 视觉编码器与 Patch Token	146
26.3 连接视觉与 LLM	146
26.4 图像分辨率与视觉 Token	146
26.5 音频与语音	146
26.6 视频	147
26.7 多模态幻觉与安全	147

第二十七章 模型选型、开源生态与许可证	148
27.1 选型的第一原则：从任务和约束出发	148
27.2 模型卡应读什么	148
27.3 开源、开放权重与开放源码	148
27.4 常见模型文件与格式	148
27.5 推理框架与训练框架的角色	149
27.6 选型实验矩阵	149
第二十八章 从零完成一个 LLM 项目的标准流程	150
28.1 第一步：定义问题	150
28.2 第二步：建立无模型基线	150
28.3 第三步：建立最小模型基线	150
28.4 第四步：错误分析	150
28.5 第五步：选择增强手段	150
28.6 第六步：离线与线上闭环	151
28.7 最小工程清单	151
28.8 错误优先级	151
第二十九章 学习路线与面试自检	152
29.1 四阶段学习路线	152
29.1.1 阶段一：会使用 and 解释	152
29.1.2 阶段二：会训练和评测	152
29.1.3 阶段三：会部署和优化	152
29.1.4 阶段四：会设计可靠系统	152
29.2 建议动手项目	152
29.3 高频自检问题	153
附录 A 核心术语速查表	154
附录 B 关键公式速查	158
附录 C 精选参考文献	160
结语	163

第一部分

第一部分：建立共同语言

什么是大语言模型

1.1 从语言模型到大语言模型

语言模型 (Language Model, LM) 的核心任务，是为一段 **Token** 序列分配概率，或者根据已有 **Token** 预测下一个 **Token**。对于序列 $x_{1:T}$ ，自回归语言模型使用概率链式法则：

$$P(x_{1:T}) = \prod_{t=1}^T P(x_t | x_{<t}). \quad (1.1)$$

模型生成文本时，只是在不断重复“计算下一个 **Token** 的概率分布并选择一个 **Token**”。翻译、问答、写代码和总结之所以都能统一为这一过程，是因为任务描述和输入输出都可以序列化成 **Token**。

LLM 的三个“大”

- **参数规模大**：模型拥有大量可学习权重，容量更强；
- **训练数据大**：覆盖多领域、多语言、多任务的数据分布；
- **训练计算大**：需要大量矩阵运算和分布式系统支撑。

“大”本身不是定义能力的充分条件。数据质量、训练目标、架构、后训练和推理策略同样重要。

1.2 基础模型、语言模型、聊天模型与推理模型

概念	主要训练形态	典型行为
基础模型	大规模自监督预训练	能续写，但不一定服从指令，输出格式和安全性不可控
语言模型	对序列概率建模的统称	可包含小模型、大模型、掩码模型、自回归模型
指令/聊天模型	基础模型加 SFT、偏好对齐与安全训练	能识别角色、遵循自然语言指令并进行多轮对话
推理模型	在复杂推理数据、可验证奖励或强化学习上进一步训练	更愿意分配计算进行搜索、校验、反思或调用工具
多模态模型	联合处理文本、图像、音频、视频等表示	能跨模态理解和生成；核心仍是表示对齐与条件生成

1.3 生成能力从哪里来

LLM 的能力并不是数据库式地逐条存储答案，而是在训练中压缩数据中的统计规律、概念关系、语言结构和任务模式。参数构成一种**分布式表示**：一个知识点通常不是存放在单个神经元中，而是由大量参数共同编码；同一组参数也会参与多个能力。

直觉理解

可以把预训练理解为“读完海量材料后，训练出一个能够预测文本如何继续的函数”。为了准确预测，模型不得不学习语法、事实相关性、常见推理套路、代码结构和人类表达习惯。但“预测得像”不等于“事实一定正确”，这正是幻觉的根源之一。

1.4 能力、知识与行为的区别

- **知识**：参数或外部上下文中包含的信息，例如某个概念的定义；
- **能力**：执行某类计算或转换的泛化能力，例如抽取、翻译、规划；
- **行为**：在特定提示、采样参数和安全策略下实际输出什么；
- **性能**：在指定数据集、指标、预算和工具条件下测得的结果。

同一模型可能“知道”某事实，却因提示不清、检索噪声、上下文冲突或解码随机性而没有正确输出。评估时必须把模型能力、系统能力与单次输出区分开。

常见误区

误区一：LLM 就是搜索引擎。搜索引擎检索已有文档；LLM 根据条件概率生成 Token。RAG 系统才把检索与生成组合起来。

误区二：参数越多一定越好。参数只代表容量的一部分，训练 Token 数、数据配比、架构与后训练会改变实际能力。

误区三：模型输出流畅就表示理解正确。流畅度来自语言建模能力，不提供事实保证。

1.5 LLM 系统的四层边界

讨论一个“模型效果”时，至少要明确四层：

1. **权重层**：模型架构、参数与训练方式；
2. **推理层**：量化、采样、上下文长度、系统提示；
3. **编排层**：RAG、工具、记忆、Agent 工作流；
4. **产品层**：权限、界面、缓存、审核、监控和人工兜底。

很多线上问题并非模型权重导致。例如引用错误可能来自检索召回不准；工具误调用可能来自参数校验不足；回答变慢可能来自排队和长上下文，而非模型单步计算变慢。

本节要点

LLM 的数学核心是条件概率建模；聊天、RAG 和 Agent 是围绕模型建立的系统形态。理解任何术语时，都要先判断它属于权重、训练、推理还是应用编排层。

必备数学：从概率到反向传播

本章学习目标

读完本章后，应能够：

1. 根据张量 **shape** 判断矩阵乘法、广播和参数量是否正确；
2. 从概率链式法则推导自回归语言模型目标；
3. 解释为什么最大似然等价于最小化交叉熵；
4. 推导 Softmax-交叉熵对 **logit** 的梯度；
5. 理解反向传播、AdamW、梯度累积和混合精度分别解决什么问题。

2.1 标量、向量、矩阵与张量

标量是单个数，**向量**是一维有序数组，**矩阵**是二维数组，**张量**是对更高维数组的统称。深度学习框架中的文本、参数、激活、梯度和优化器状态最终都表示为张量。

LLM 中最常见的隐藏状态形状为

$$X \in \mathbb{R}^{B \times L \times d}, \quad (2.1)$$

其中 B 是 **batch size**， L 是序列长度， d 是隐藏维度。若线性层权重

$$W \in \mathbb{R}^{d \times d_{out}}, \quad b \in \mathbb{R}^{d_{out}}, \quad (2.2)$$

则输出为

$$Y = XW + b \in \mathbb{R}^{B \times L \times d_{out}}. \quad (2.3)$$

矩阵乘法只作用于 X 的最后一维；偏置 b 会沿 **batch** 维和序列维**广播 (broadcast)**。

2.1.1 用下标理解矩阵乘法

对二维矩阵 $A \in \mathbb{R}^{m \times n}$ 、 $W \in \mathbb{R}^{n \times p}$ ，有

$$Y_{ij} = \sum_{k=1}^n A_{ik} W_{kj}. \quad (2.4)$$

被求和的维度必须一致。这个下标式有三个用途：

- 检查代码中的转置是否正确；
- 判断 **Tensor Parallel** 应沿哪一维切分；
- 估算乘加次数约为 mnp 。

例：线性层参数量与计算量

设隐藏维度 $d = 4096$ ，一个 4096×4096 的线性层含

$$4096^2 = 16,777,216$$

个权重，若使用 BF16，仅权重约占 $16,777,216 \times 2 \approx 32$ MiB。对 $B \times L = 2048$ 个 Token 执行该层，乘加次数约为 2048×4096^2 ；若把一次乘法和一次加法分别计为一个 FLOP，则约为 $2BLd^2$ FLOPs。

2.1.2 Shape 是理解 LLM 代码的第一语言

常见张量 shape 如下：

张量	典型形状	含义
Token ID	$B \times L$	每个位置的整数词表索引
Embedding/隐藏状态	$B \times L \times d$	每个 Token 的连续表示
Attention score	$B \times H \times L_q \times L_k$	Query 与 Key 的两两分数
Logit	$B \times L \times V$	每个位置对词表 V 的未归一化分数
Label	$B \times L$	目标 Token ID，常含忽略标记 -100
KV Cache	$N \times 2 \times B \times H_{kv} \times L \times d_h$	每层 Key/Value 缓存，具体轴顺序依实现而异

工程视角

阅读模型实现时，建议在纸上写出每一步的 **shape**。变量名可能抽象或复用，但 **shape**、切分轴和广播规则不会说谎。分布式训练中的大量错误，本质是“数学上应相同的张量，在设备、dtype、shape 或布局上不一致”。

2.2 概率、条件概率与自回归分解**2.2.1 联合概率与链式法则**

句子由 Token 序列 $x_{1:T} = (x_1, \dots, x_T)$ 构成。概率链式法则给出

$$P(x_{1:T}) = P(x_1)P(x_2 | x_1)P(x_3 | x_1, x_2) \cdots P(x_T | x_{<T}) \quad (2.5)$$

$$= \prod_{t=1}^T P(x_t | x_{<t}). \quad (2.6)$$

这不是近似，而是任意联合分布都成立的恒等式。Decoder-only 语言模型选择按从左到右的顺序建模每个条件概率。

推导：链式法则为什么成立

由条件概率定义

$$P(A | B) = \frac{P(A, B)}{P(B)},$$

可得 $P(A, B) = P(A | B)P(B)$ 。对三个变量：

$$P(x_1, x_2, x_3) = P(x_3 | x_1, x_2)P(x_1, x_2) = P(x_3 | x_1, x_2)P(x_2 | x_1)P(x_1).$$

递归展开即可得到一般形式。

2.2.2 条件独立与建模假设

语言模型并没有假设“下一个 Token 只依赖最近几个 Token”。在完整因果注意力下，理论上 x_t 可以依赖上下文窗口内全部 $x_{<t}$ 。真正的限制来自：

- 上下文窗口有限；
- 模型容量和训练数据有限；
- 优化只保证降低平均损失，不保证每个依赖都被学到；
- 长距离信息可能因位置偏差、注意力稀释或检索失败而未被有效利用。

2.3 最大似然、负对数似然与语言模型损失

设训练集 $\mathcal{D} = \{x^{(n)}\}_{n=1}^N$ ，并近似认为样本独立同分布。模型参数为 θ ，整个数据集的似然为

$$\mathcal{L}_{\text{like}}(\theta) = \prod_{n=1}^N P_{\theta}(x^{(n)}). \quad (2.7)$$

最大似然估计选择

$$\theta^* = \arg \max_{\theta} \prod_{n=1}^N P_{\theta}(x^{(n)}). \quad (2.8)$$

直接连乘许多小概率会数值下溢，而且乘积不便求导，因此取对数：

$$\theta^* = \arg \max_{\theta} \log \prod_{n=1}^N P_{\theta}(x^{(n)}) \quad (2.9)$$

$$= \arg \max_{\theta} \sum_{n=1}^N \log P_{\theta}(x^{(n)}). \quad (2.10)$$

再利用自回归分解：

$$\theta^* = \arg \max_{\theta} \sum_{n=1}^N \sum_{t=1}^{T_n} \log P_{\theta}(x_t^{(n)} | x_{<t}^{(n)}). \quad (2.11)$$

把最大化改写为最小化负值，得到**负对数似然 (NLL)**：

$$\mathcal{J}_{\text{NLL}}(\theta) = -\frac{1}{\sum_n T_n} \sum_{n,t} \log P_{\theta}(x_t^{(n)} | x_{<t}^{(n)}). \quad (2.12)$$

直觉理解

模型每次都会为整个词表分配概率。训练数据告诉它“这一步实际出现的是哪个 Token”，损失只读取该 Token 的负对数概率。正确 Token 概率越接近 1，损失越接近 0；概率越小，惩罚增长越快。

2.3.1 为什么取对数不会改变最优解

对数函数严格单调递增，因此若 $a > b > 0$ ，则 $\log a > \log b$ 。最大化正数乘积与最大化其对数拥有相同的最优点。对数同时把乘法变成加法，便于对不同 Token 和样本求平均。

2.4 Logit、Softmax 与温度

神经网络最后输出 **logit** $z_i \in \mathbb{R}$ ，它不是概率。Softmax 将 V 个 logit 转成概率分布：

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^V e^{z_j}}, \quad \sum_i p_i = 1. \quad (2.13)$$

2.4.1 Softmax 只关心相对差值

对任意常数 c ,

$$\text{softmax}(z + c\mathbf{1}) = \text{softmax}(z). \quad (2.14)$$

因为分子分母都多乘 e^c 。因此实现通常使用

$$p_i = \frac{e^{z_i - z_{\max}}}{\sum_j e^{z_j - z_{\max}}} \quad (2.15)$$

避免 e^{z_i} 溢出，这称为 **log-sum-exp trick** 的基本形式。

概率比满足

$$\frac{p_i}{p_j} = e^{z_i - z_j}. \quad (2.16)$$

所以 logit 差 1 意味着概率比约为 $e \approx 2.718$ ，logit 差 5 意味着概率比约为 148。

2.4.2 温度如何改变分布

带温度的 Softmax 为

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}, \quad T > 0. \quad (2.17)$$

当 $T < 1$ ，logit 差被放大，分布更尖锐；当 $T > 1$ ，差值被压缩，分布更平坦。 $T \rightarrow 0^+$ 时趋近 **argmax**； $T \rightarrow \infty$ 时趋近均匀分布。

例：温度不是“随机开关”

若三个 logit 为 $(2, 1, 0)$ ，则 $T = 1$ 时概率约为 $(0.665, 0.245, 0.090)$ ； $T = 0.5$ 等价于使用 $(4, 2, 0)$ ，概率约为 $(0.867, 0.117, 0.016)$ ； $T = 2$ 等价于使用 $(1, 0.5, 0)$ ，概率约为 $(0.506, 0.307, 0.186)$ 。温度改变的是整个候选分布，而不是直接规定某个 Token 是否可选。

2.5 熵、交叉熵与 KL 散度

2.5.1 熵：分布自身的不确定性

离散分布 q 的熵为

$$H(q) = - \sum_i q_i \log q_i. \quad (2.18)$$

确定分布的熵为 0；均匀分布的熵最大。熵描述“真实分布本身有多不确定”。

2.5.2 交叉熵：用另一个分布编码真实分布的代价

真实分布为 q 、模型分布为 p 时：

$$H(q, p) = - \sum_i q_i \log p_i. \quad (2.19)$$

若标签是 one-hot，正确类别为 y ，则 $q_y = 1$ 、其他为 0，因此

$$H(q, p) = - \log p_y. \quad (2.20)$$

这就是分类交叉熵，也是每个 Token 的 NLL。

2.5.3 KL 散度：两个分布的额外编码代价

$$D_{\text{KL}}(q||p) = \sum_i q_i \log \frac{q_i}{p_i}. \quad (2.21)$$

展开可得

$$D_{\text{KL}}(q||p) = \sum_i q_i \log q_i - \sum_i q_i \log p_i \quad (2.22)$$

$$= -H(q) + H(q, p), \quad (2.23)$$

即

$$\boxed{H(q, p) = H(q) + D_{\text{KL}}(q||p)}. \quad (2.24)$$

对固定训练数据， $H(q)$ 与模型参数无关，因此最小化交叉熵等价于最小化 $D_{\text{KL}}(q||p)$ 。

为什么 KL 非负，但不对称

KL 非负可由 Jensen 不等式证明；直觉上，使用错误分布 p 表示来自 q 的数据，不会比知道真实分布 q 更省平均编码长度。它不对称，因为“用 p 近似 q ”和“用 q 近似 p ”对低概率区域的惩罚不同。对齐中 $D_{\text{KL}}(\pi_\theta||\pi_{ref})$ 与 $D_{\text{KL}}(\pi_{ref}||\pi_\theta)$ 会产生不同的模式覆盖行为，不能随意互换。

2.5.4 困惑度

平均 Token NLL 为 $\bar{\ell}$ 时，困惑度定义为

$$\text{PPL} = e^{\bar{\ell}}. \quad (2.25)$$

若模型在每一步都相当于从 K 个等概率候选中选择，则 $\bar{\ell} = \log K$ ，PPL 恰为 K ，因此可把 PPL 粗略理解为“平均有效候选数”。

常见误区

PPL 不能跨 tokenizer 盲目比较。同一文本被切成的 Token 数不同，会改变每 Token 损失。
低 PPL 不等于高事实性。PPL 衡量数据分布上的下一个 Token 预测，不能直接度量引用真实性、安全性或复杂推理。

2.5.5 Label Smoothing

若类别数为 V ，标签平滑把 one-hot 目标改成

$$q_i^{(\epsilon)} = (1 - \epsilon)\mathbf{1}[i = y] + \frac{\epsilon}{V}. \quad (2.26)$$

它抑制过度自信，但在语言建模中并非总是有益：词表巨大且目标本身明确，平滑可能降低最大似然拟合能力。是否使用应由具体任务和校准需求决定。

2.6 微积分与链式法则**2.6.1 导数与梯度**

标量函数 $f(x)$ 的导数描述局部变化率。多变量标量函数 $f(\mathbf{x})$ 的梯度为

$$\nabla_{\mathbf{x}} f = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)^\top. \quad (2.27)$$

梯度指向函数增长最快的方向，因此梯度下降沿负梯度更新。

2.6.2 链式法则

若 $y = f(u)$ 、 $u = g(x)$ ，则

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}. \quad (2.28)$$

神经网络是许多函数的复合，反向传播就是从损失出发，沿计算图反向重复应用链式法则。

例：一元线性回归的反向传播

设预测 $\hat{y} = wx + b$ ，平方损失

$$\ell = \frac{1}{2}(\hat{y} - y)^2.$$

先求损失对预测的导数：

$$\frac{\partial \ell}{\partial \hat{y}} = \hat{y} - y.$$

再由 $\partial \hat{y} / \partial w = x$ 、 $\partial \hat{y} / \partial b = 1$ ，得到

$$\frac{\partial \ell}{\partial w} = (\hat{y} - y)x, \quad \frac{\partial \ell}{\partial b} = \hat{y} - y.$$

这里“误差”通过链式法则传回参数。大型 Transformer 的原理相同，只是计算图更大，矩阵求导由自动微分完成。

2.7 Softmax-交叉熵梯度的完整推导

这是理解分类和语言模型训练最重要的推导之一。设

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad \ell = - \sum_i y_i \log p_i, \quad (2.29)$$

其中 $\mathbf{y}$ 是 one-hot 或一般概率标签。

2.7.1 第一步：求 Softmax 的偏导

当 $i = k$:

$$\frac{\partial p_i}{\partial z_i} = p_i(1 - p_i). \quad (2.30)$$

当 $i \neq k$:

$$\frac{\partial p_i}{\partial z_k} = -p_i p_k. \quad (2.31)$$

可统一写成

$$\frac{\partial p_i}{\partial z_k} = p_i(\delta_{ik} - p_k), \quad (2.32)$$

其中 δ_{ik} 是 Kronecker delta。

2.7.2 第二步：链式求损失对 logit 的梯度

$$\frac{\partial \ell}{\partial z_k} = - \sum_i y_i \frac{1}{p_i} \frac{\partial p_i}{\partial z_k} \quad (2.33)$$

$$= - \sum_i y_i (\delta_{ik} - p_k) \quad (2.34)$$

$$= -y_k + p_k \sum_i y_i. \quad (2.35)$$

因为标签分布满足 $\sum_i y_i = 1$ ，所以

$$\boxed{\frac{\partial \ell}{\partial z_k} = p_k - y_k.} \quad (2.36)$$

向量形式为

$$\boxed{\nabla_{\mathbf{z}} \ell = \mathbf{p} - \mathbf{y}.} \quad (2.37)$$

直觉理解

对正确 Token, $y_k = 1$, 梯度为 $p_k - 1 < 0$, 梯度下降会提高其 logit; 对错误 Token, $y_k = 0$, 梯度为 $p_k > 0$, 梯度下降会降低其 logit。模型越自信地预测错误 Token, 受到的惩罚越大。

2.7.3 输出层梯度

若 $\mathbf{z} = W\mathbf{h} + b$, 则

$$\frac{\partial \ell}{\partial W} = (\mathbf{p} - \mathbf{y})\mathbf{h}^\top, \quad (2.38)$$

$$\frac{\partial \ell}{\partial \mathbf{h}} = W^\top(\mathbf{p} - \mathbf{y}), \quad (2.39)$$

$$\frac{\partial \ell}{\partial b} = \mathbf{p} - \mathbf{y}. \quad (2.40)$$

误差信号因此从词表输出层传回隐藏状态, 再继续经过所有 Transformer 层。

2.8 梯度下降、Mini-batch 与梯度累积

全数据目标为

$$J(\theta) = \frac{1}{N} \sum_{n=1}^N \ell_n(\theta). \quad (2.41)$$

全量梯度代价过高, 因此每步从 mini-batch $\mathcal{B}$ 估计:

$$g_t = \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} \nabla_{\theta} \ell_n(\theta_t). \quad (2.42)$$

若采样无偏, 则 $\mathbb{E}[g_t] = \nabla J(\theta_t)$, 但有随机方差。较大 batch 通常降低梯度噪声, 却增加显存和通信需求。

最基本的 SGD 更新为

$$\theta_{t+1} = \theta_t - \eta g_t. \quad (2.43)$$

带动量的 SGD 维护速度:

$$v_t = \mu v_{t-1} + g_t, \quad (2.44)$$

$$\theta_{t+1} = \theta_t - \eta v_t. \quad (2.45)$$

2.8.1 梯度累积为什么等价于大 batch

将一个逻辑 batch 拆为 K 个 micro-batch, 分别计算梯度并求平均:

$$g = \frac{1}{K} \sum_{k=1}^K g^{(k)}. \quad (2.46)$$

在模型状态与随机操作保持一致时, 梯度累积与一次处理合并后的大批次在数学上近似等价。全局 batch 常写为

$$B_{\text{global}} = B_{\text{micro}} \times N_{\text{DP}} \times N_{\text{accum}}. \quad (2.47)$$

注意 loss 是按 Token 还是按样本平均, 会影响变长序列下的精确等价性。

2.9 Adam 与 AdamW 的逐项解释

LLM 训练最常见的是 AdamW。Adam 对梯度维护一阶矩和二阶矩：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (2.48)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2. \quad (2.49)$$

由于初始 $m_0 = v_0 = 0$ ，早期估计偏小，进行偏差修正：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad (2.50)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (2.51)$$

Adam 更新为

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (2.52)$$

分母使每个参数按历史梯度尺度自适应调整步长。

AdamW 将权重衰减与梯度更新解耦：

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (2.53)$$

λ 是 weight decay 系数。实践中通常不对 bias 和归一化缩放参数施加同样的衰减。

常见误区

Weight decay 不完全等于“把 L_2 正则项加到 Adam 的 loss”。对普通 SGD 二者可对应，但在自适应优化器中，梯度耦合的 L_2 正则与解耦衰减产生不同更新；AdamW 的“W”正是强调解耦。

2.9.1 学习率调度

常见流程为 warmup 后衰减。线性 warmup 可写为

$$\eta_t = \eta_{max} \frac{t}{T_w}, \quad t \leq T_w. \quad (2.54)$$

余弦衰减可写为

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left[1 + \cos \left(\pi \frac{t - T_w}{T - T_w} \right) \right]. \quad (2.55)$$

Warmup 减少训练初期激活、梯度和优化器统计尚不稳定时的大步更新。

2.10 梯度裁剪、消失与爆炸

全局梯度范数为

$$\|g\|_2 = \sqrt{\sum_i g_i^2}. \quad (2.56)$$

若超过阈值 c ，范数裁剪使用

$$g \leftarrow g \cdot \min \left(1, \frac{c}{\|g\|_2 + \epsilon} \right). \quad (2.57)$$

这保留方向，只缩小整体幅度。深层网络中的残差连接、归一化、初始化和学习率共同影响梯度稳定性。

2.11 浮点数、混合精度与数值稳定性

浮点数由符号位、指数和尾数组成。指数位决定范围，尾数位决定相对精度。常见格式：

格式	指数位	尾数位	典型特点
FP32	8	23	范围与精度较高，显存和带宽代价大
FP16	5	10	尾数较细，但指数范围小，训练易溢出/下溢
BF16	8	7	范围接近 FP32，精度较粗，LLM 训练常用
FP8	视格式而定	视格式而定	需要缩放与硬件支持，适合特定矩阵计算
INT8/INT4	无浮点指数	整数	需量化尺度，常用于推理权重/激活

2.11.1 混合精度训练

混合精度并不是“全部换成低精度”，而是让矩阵乘法等大算子使用低精度，同时在必要处保留高精度状态。典型做法包括：

- BF16/FP16 参数和激活参与前向、反向；
- 优化器一阶、二阶矩使用 FP32；
- 某些归一化、归约或累加使用更高精度；
- FP16 训练可能使用 loss scaling，把微小梯度放大后再还原。

2.11.2 Loss Scaling 的原理

若原损失为 ℓ ，乘比例 S ：

$$\ell' = S\ell, \quad \nabla_{\theta}\ell' = S\nabla_{\theta}\ell. \quad (2.58)$$

反向后再除以 S ，理论梯度不变，但中间梯度更不易在 FP16 中下溢。动态 loss scaling 会在检测到 inf/NaN 时降低 S 。

工程视角

遇到 NaN 时不要只降低学习率。应定位首次出现异常的层和算子，并检查输入范围、mask 值、除零、指数溢出、低精度归约、梯度范数、数据异常和分布式同步。NaN 可能在某一步产生，却在若干层之后才显现。

2.12 期望、方差与无偏估计

随机变量 X 的期望和方差为

$$\mathbb{E}[X] = \sum_x xP(X=x), \quad (2.59)$$

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2. \quad (2.60)$$

“无偏”表示估计量的期望等于真实量。Mini-batch 梯度、策略梯度和离线评测都需要区分：

- **偏差 (bias)**：平均估计是否系统性偏离；
- **方差 (variance)**：重复采样时波动有多大；
- **均方误差**： $\text{MSE} = \text{Bias}^2 + \text{Variance}$ （加不可约噪声时还需额外项）。

后续 RLHF 中引入 baseline 的目的，正是降低策略梯度方差而不引入偏差。

2.13 本章综合例题

练习 1：从概率到损失

一个长度为 3 的序列，模型分别给正确 Token 的概率为 0.5, 0.25, 0.1。求该序列似然、总 NLL、平均 NLL 和 PPL。

解答

似然为 $0.5 \times 0.25 \times 0.1 = 0.0125$ 。总 NLL 为

$$-\log 0.5 - \log 0.25 - \log 0.1 \approx 4.382.$$

平均 NLL 约为 1.461，PPL 为 $e^{1.461} \approx 4.31$ 。

练习 2：交叉熵梯度

三分类模型概率为 (0.7, 0.2, 0.1)，真实类别是第二类。写出损失对三个 logit 的梯度。

解答

$\mathbf{y} = (0, 1, 0)$ ，因此

$$\nabla_{\mathbf{z}} \ell = \mathbf{p} - \mathbf{y} = (0.7, -0.8, 0.1).$$

梯度下降会降低第一、第三类 logit，并提高第二类 logit。

练习 3：全局 Batch

每卡 micro-batch 为 2，数据并行 16 卡，梯度累积 8 次。全局 batch 是多少？若每个样本平均含 2048 个有效 Token，每次参数更新看到多少 Token？

解答

$B_{\text{global}} = 2 \times 16 \times 8 = 256$ 个样本；每次更新约处理 $256 \times 2048 = 524,288$ 个有效 Token。

本节要点

LLM 的基础数学主线是：联合概率用链式法则分解为下一 Token 条件概率；最大似然经取对数变成 NLL；Softmax 把 logit 变成概率；交叉熵梯度简化为 $p - y$ ；反向传播用链式法则把误差传到全部参数；AdamW、学习率调度、梯度裁剪与混合精度使大规模优化在工程上可行。

文本如何进入模型：Token、Tokenizer 与 Embedding

3.1 为什么不能直接按“字”处理

神经网络只能处理数值。最直接的方法是把每个字符或单词映射为整数，但纯字符序列过长，纯单词词表又无法处理生词和多语言。现代 LLM 通常使用子词 **Tokenization**，在字符和单词之间折中。

3.2 Token、词表与 Token ID

Token 是 tokenizer 切分出的基本单元，可能是一个字、词的一部分、空格、标点、字节或特殊标记。**词表 (vocabulary)** 给每个 Token 分配唯一整数 ID。模型接收的是 ID 序列，而不是字符串。

必须分清四个长度

- 字符数：字符串有多少字符；
- Token 数：经 tokenizer 后有多少单元；
- 上下文长度：单次前向可容纳的最大 Token 数；
- 生成长度：允许新生成多少 Token。

它们不能互换。同一句中文、英文或代码，在不同 tokenizer 下 Token 数可能差异显著。

3.3 BPE、WordPiece、Unigram 与 SentencePiece

3.3.1 BPE

Byte Pair Encoding 从字符或字节开始，反复合并语料中最常见的相邻单元。高频片段会形成较长 Token，低频词仍可拆解。

3.3.2 WordPiece

WordPiece 与 BPE 类似，但合并规则更强调语言模型似然或词表收益。BERT 系模型常见。

3.3.3 Unigram

Unigram 从较大候选词表开始，逐步删除对语料似然贡献较小的 Token，最终保留一个概率词表。

3.3.4 SentencePiece

SentencePiece 是直接在原始 Unicode 字符串上训练 tokenizer 的工具体系，不要求预先按空格分词，可实现 BPE 或 Unigram。它常把空格编码为显式符号。

3.4 字节级 Tokenization 与特殊 Token

字节级方案原则上能表示任意文本，避免未知字符，但可能增加序列长度。常见特殊 Token 包括：

- BOS/EOS：序列开始与结束；
- PAD：批处理时填充长度；
- UNK：未知 Token，字节级方案通常可避免；
- MASK：掩码语言模型使用；
- 系统、用户、助手角色标记；
- 工具调用、图像占位、思考边界等协议标记。

3.5 Chat Template 不是普通字符串拼接

聊天模型在训练时看到的通常不是自然语言对话原文，而是带角色和边界标记的序列。**Chat Template**规定 system、user、assistant 和 tool 消息如何序列化。

Listing 3.1: 示意性的聊天模板序列化

```
<bos><system>You are helpful.</system>
<user>解释注意力机制</user>
<assistant>注意力机制的核心是...</assistant><eos>
```

模板不匹配会导致角色混乱、重复生成、无法停止或性能下降。部署开源聊天模型时，必须使用其模型卡规定的模板和停止 Token。

3.6 Embedding: 从离散 ID 到连续向量

Embedding 表本质上是矩阵

$$E \in \mathbb{R}^{|V| \times d}, \quad (3.1)$$

其中 $|V|$ 是词表大小， d 是隐藏维度。Token ID i 通过查表得到向量 E_i 。训练使语义、语法和使用模式相近的 Token 在某些方向上表现相似。

输入 Embedding 与输出分类权重有时共享参数，称为 **weight tying**。输出层把隐藏状态映射回词表 logit：

$$z_t = W_{\text{vocab}} h_t. \quad (3.2)$$

3.7 Padding、Attention Mask 与 Label Mask

批处理通常将不同长度序列 padding 到相同长度。三类 mask 容易混淆：

- **Padding Mask**：防止注意力读取填充位置；
- **Causal Mask**：防止当前位置看见未来 Token；
- **Loss/Label Mask**：指定哪些位置计算训练损失，例如 SFT 中通常不训练用户提示部分。

常见误区

Tokenizer 是模型的一部分。换 tokenizer 不只是换预处理器，还会改变词表索引、Embedding 和输出头，通常不能直接兼容原权重。新增特殊 Token 后，必须扩展并训练相应 Embedding。

3.8 Tokenizer 对模型效果的影响

Tokenizer 会影响：序列长度、训练计算、数字拆分、代码缩进、多语言公平性、低资源语言效率、复制能力和特殊格式稳定性。词表越大不一定越好：大词表减少序列长度，却增加 Embedding/输出层参数和 Softmax 开销，也可能使低频 Token 学习不足。

本节要点

Tokenizer 决定模型看到的离散世界；Embedding 将其映射到连续空间。实际工程中，Token 数直接关联上下文、延迟、显存和成本，Chat Template 与各种 mask 则决定训练和推理语义是否正确。

神经网络基础：参数、层、激活与归一化

4.1 参数与超参数

参数是训练中通过梯度更新的数值，例如权重矩阵；**超参数**由训练者设定，例如层数、隐藏维度、学习率和 batch size。模型“7B”通常表示约 70 亿参数，不代表显存只需 7GB。

参数存储的粗略计算：

$$\text{权重显存} \approx N_{\text{param}} \times \text{每参数字节数}. \quad (4.1)$$

例如 7B 参数使用 FP16/BF16 权重约需 14GB，仅是权重，不含 KV Cache、激活、梯度和优化器状态。

4.2 线性层与多层感知机

线性层执行 $Y = XW + b$ 。单纯堆叠线性层仍等价于一个线性变换，因此需要非线性激活。Transformer 的前馈网络 (FFN/MLP) 通常为：

$$\text{FFN}(x) = W_2 \sigma(W_1 x), \quad (4.2)$$

现代模型常使用门控变体，例如 SwiGLU：

$$\text{SwiGLU}(x) = (\text{SiLU}(xW_g) \odot xW_u)W_d. \quad (4.3)$$

其中 $\odot$ 表示逐元素乘法。门控允许模型动态控制信息通过。

4.3 激活函数

激活	形式/特征	理解
ReLU	$\max(0, x)$	简单高效，但负区间梯度为零
GELU	平滑近似 $x\Phi(x)$	BERT、GPT 等早期 Transformer 常用
SiLU/Swish	$x\sigma(x)$	平滑，现代 LLM 常与门控结构组合
SwiGLU	SiLU 门控的 GLU	提高 FFN 表达能力，常调整中间维度以控制参数量

4.4 残差连接

残差连接写作

$$y = x + F(x). \quad (4.4)$$

它让信息和梯度可沿恒等路径传播，使深层网络更易训练。Transformer 的每个 Attention 和 MLP 子层通常都有残差。

4.5 LayerNorm 与 RMSNorm

LayerNorm 对一个 Token 的隐藏维度进行归一化：

$$\text{LN}(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta. \quad (4.5)$$

RMSNorm 不减均值，只按均方根缩放：

$$\text{RMSNorm}(x) = \gamma \frac{x}{\sqrt{\frac{1}{d} \sum_i x_i^2 + \epsilon}}. \quad (4.6)$$

RMSNorm 计算更简洁，现代 decoder-only LLM 中很常见。

4.6 Pre-Norm 与 Post-Norm

- **Post-Norm**: $x \leftarrow \text{Norm}(x + F(x))$;
- **Pre-Norm**: $x \leftarrow x + F(\text{Norm}(x))$ 。

Pre-Norm 通常更易训练深层网络；Post-Norm 在某些设置下可能获得不同的表示性质，但需要更谨慎的初始化和优化。

4.7 Dropout、正则化与训练/推理模式

Dropout 在训练时随机置零部分激活，推理时关闭。模型必须正确切换 `train()` 与 `eval()` 模式，否则 dropout 和某些归一化层行为会错误。现代超大规模预训练中 dropout 比早期模型更少使用，但在微调和小数据场景仍可能有用。

常见误区

“层数更多” “隐藏维度更大” “FFN 更宽” 都会增加参数和计算，但影响不相同。粗略地说，Attention 参数主要随 d^2 增长，FFN 也随 $d \cdot d_{ff}$ 增长；序列长度主要影响 Attention 的计算和 KV Cache，而不直接增加模型参数。

本节要点

Transformer 不是一个神秘整体，而是线性层、非线性、残差和归一化的重复组合。参数量描述权重规模；显存还取决于数值精度、激活、优化器、KV Cache 与并发。

第二部分

第二部分：Transformer 的内部机制

Transformer 总览：从 Token 到下一个 Token

本章学习目标

本章建立完整前向计算链。读完后，应能够从 Token ID 开始，写出 Embedding、位置处理、Norm、Attention、MLP、残差和输出头的 shape 与公式，并能粗略估算参数量和 FLOPs。

5.1 三种经典架构

Transformer 最初采用 Encoder-Decoder 结构 [1]，随后形成三类主流架构：

架构	可见上下文	典型用途
Encoder-only	双向，自注意力可读取左右两侧	表示学习、分类、检索编码、序列标注；典型如 BERT[2]
Decoder-only	因果，只读取当前位置及之前 Token	自回归生成、聊天、代码和通用 LLM；现代基础模型主流
Encoder-Decoder	Encoder 双向编码，Decoder 因果生成并交叉注意输入	翻译、摘要、结构化条件生成

Decoder-only 并非“只能生成，不能理解”。它通过预测下一个 Token 学习语义、句法和任务模式，只是在计算结构上使用因果方向。

5.2 一条完整的 Decoder-only 前向链

设输入 Token ID 为

$$I \in \{0, \dots, V-1\}^{B \times L}, \quad (5.1)$$

词表大小为 V ，隐藏维度为 d ，层数为 N 。

5.2.1 第一步：Token Embedding

嵌入矩阵

$$E \in \mathbb{R}^{V \times d} \quad (5.2)$$

相当于一个查表。第 i 个 Token ID 对应 E 的第 i 行：

$$X^{(0)} = E[I] \in \mathbb{R}^{B \times L \times d}. \quad (5.3)$$

Embedding 不是把 ID 当连续数值输入，而是为每个离散符号选择一个可训练向量。

5.2.2 第二步：位置处理

模型必须区分“猫追狗”和“狗追猫”。位置可以通过把位置向量加到 embedding、对注意力分数加偏置，或旋转 Q/K 等方式编码。现代 Decoder-only 模型常使用 RoPE，详见下一章。

5.2.3 第三步：堆叠 Transformer Block

以常见 Pre-Norm 结构为例，第 ℓ 层可写为

$$U^{(\ell)} = X^{(\ell-1)} + \text{Attention}^{(\ell)} \left(\text{Norm}_1^{(\ell)}(X^{(\ell-1)}) \right), \quad (5.4)$$

$$X^{(\ell)} = U^{(\ell)} + \text{MLP}^{(\ell)} \left(\text{Norm}_2^{(\ell)}(U^{(\ell)}) \right). \quad (5.5)$$

每个子层都以“归一化-变换-残差相加”的形式出现。

5.2.4 第四步：输出头

最后一层隐藏状态经最终归一化后得到

$$H = \text{Norm}_{final}(X^{(N)}) \in \mathbb{R}^{B \times L \times d}. \quad (5.6)$$

输出矩阵 $W_{out} \in \mathbb{R}^{d \times V}$ 将每个位置映射到词表 logit：

$$Z = HW_{out} \in \mathbb{R}^{B \times L \times V}. \quad (5.7)$$

许多模型令 $W_{out} = E^\top$ ，称为**权重共享 (weight tying)**，可减少参数并让输入、输出词空间耦合。

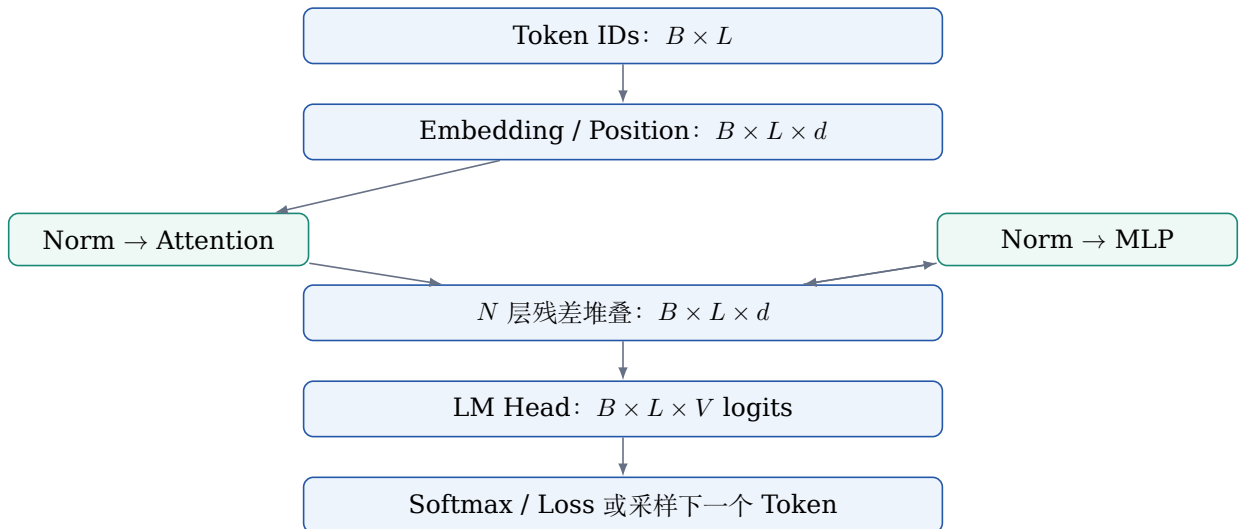


图 5.1: Decoder-only LLM 的主计算链。

5.3 隐藏状态究竟表示什么

隐藏状态 $X_{b,t,:}^{(\ell)} \in \mathbb{R}^d$ 是第 b 个样本、第 t 个位置在第 ℓ 层的连续表示。它不是某个可直接阅读的“词义坐标”，而是供后续层使用的分布式特征。

同一 Token 在不同上下文中的隐藏状态不同。例如“苹果发布新产品”和“吃了一个苹果”使用同一输入 embedding，但经过上下文注意力后会形成不同表示。层越深，表示通常越融合上下文、任务和输出需求。

5.4 Attention 子层的 shape

对归一化后的输入

$$X \in \mathbb{R}^{B \times L \times d}, \quad (5.8)$$

设 Query 头数为 H_q ，每头维度 $d_h = d/H_q$ ，则

$$Q = XW_Q \in \mathbb{R}^{B \times L \times H_q d_h}, \quad (5.9)$$

$$K = XW_K \in \mathbb{R}^{B \times L \times H_{kv} d_h}, \quad (5.10)$$

$$V = XW_V \in \mathbb{R}^{B \times L \times H_{kv} d_h}. \quad (5.11)$$

reshape 后常表示为

$$Q \in \mathbb{R}^{B \times H_q \times L \times d_h}, \quad (5.12)$$

$$K, V \in \mathbb{R}^{B \times H_{kv} \times L \times d_h}. \quad (5.13)$$

Attention 输出再拼接 Query heads 并通过 W_O 投影回 d 维，因此残差相加前后 shape 不变。

5.5 MLP/FFN 子层

最简单的前馈网络为

$$\text{FFN}(x) = W_2 \phi(W_1 x + b_1) + b_2, \quad (5.14)$$

其中 $W_1 \in \mathbb{R}^{d \times d_{ff}}$ ， $W_2 \in \mathbb{R}^{d_{ff} \times d}$ ，且 $d_{ff} > d$ 。它对每个 Token 独立应用相同参数，不直接在位置之间交换信息；位置交互由 Attention 完成。

现代 LLM 常用门控结构，例如 SwiGLU：

$$\text{SwiGLU}(x) = W_2 (\text{SiLU}(xW_g) \odot (xW_u)), \quad (5.15)$$

其中

$$\text{SiLU}(a) = a\sigma(a), \quad (5.16)$$

$\odot$ 表示逐元素乘法。门控分支让模型能够按输入选择通过哪些特征。

5.5.1 为什么 MLP 参数通常很多

标准 MLP 有两块大矩阵，参数约为

$$2dd_{ff}. \quad (5.17)$$

门控 MLP 有三块大矩阵，约为

$$3dd_{ff}. \quad (5.18)$$

因此即使 Attention 更受关注，许多 Dense LLM 的大部分参数和 FLOPs 实际位于 MLP。

5.6 残差连接为什么重要

若一层直接写成 $x_{\ell+1} = F_{\ell}(x_{\ell})$ ，深层网络的梯度必须连续乘许多 Jacobian，容易消失或爆炸。残差形式为

$$x_{\ell+1} = x_{\ell} + F_{\ell}(x_{\ell}). \quad (5.19)$$

对 x_{ℓ} 求导：

$$\frac{\partial x_{\ell+1}}{\partial x_{\ell}} = I + \frac{\partial F_{\ell}}{\partial x_{\ell}}. \quad (5.20)$$

恒等路径 I 为信息和梯度提供直接通道。即使 F_{ℓ} 初期学得较差，网络仍可接近恒等映射。

直觉理解

可以把残差流想象成贯穿所有层的“主干道路”，每个 Attention 或 MLP 子层只向主干写入修正量，而不是每层都完全重写表示。

5.7 LayerNorm、RMSNorm 与 Pre-Norm

LayerNorm 对单个 Token 的隐藏维度归一化：

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad (5.21)$$

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2, \quad (5.22)$$

$$\text{LN}(x)_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i. \quad (5.23)$$

RMSNorm 不减均值：

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_i x_i^2 + \epsilon}, \quad (5.24)$$

$$\text{RMSNorm}(x)_i = \gamma_i \frac{x_i}{\text{RMS}(x)}. \quad (5.25)$$

RMSNorm 参数和计算更简洁，现代 LLM 常用 [7]。

Pre-Norm 在子层之前归一化：

$$y = x + F(\text{Norm}(x)), \quad (5.26)$$

Post-Norm 在残差后归一化：

$$y = \text{Norm}(x + F(x)). \quad (5.27)$$

Pre-Norm 通常更利于深层稳定训练，但最终架构还会配合初始化、残差缩放和归一化位置共同设计。

5.8 Teacher Forcing: 一次训练全部位置

给定序列

$$\langle \text{BOS} \rangle, x_1, x_2, \dots, x_T,$$

模型输入前 T 个位置，标签右移一位：

$$\text{input} = (\langle BOS \rangle, x_1, \dots, x_{T-1}), \quad (5.28)$$

$$\text{label} = (x_1, x_2, \dots, x_T). \quad (5.29)$$

因果 mask 保证第 t 个位置只能读取 $x_{<t}$ 。由于训练时所有真实前缀已经给出，所有位置可在一次矩阵计算中并行预测，这称为 **Teacher Forcing**。

例：训练输入与标签对齐

文本 Token 为 [101, 37, 52, 9]，加入 BOS=1 后：

$$\text{input} = [1, 101, 37, 52], \quad \text{label} = [101, 37, 52, 9].$$

模型输出四个位置的词表分布，分别计算正确标签的交叉熵。

5.9 为什么训练可并行，生成仍串行

训练时第 t 个位置的真实前缀已知，可同时计算全部位置。生成时 x_t 尚未产生，必须先采样 x_t ，再把它加入上下文以产生 x_{t+1} ：

$$x_t \sim P_\theta(\cdot \mid x_{<t}). \quad (5.30)$$

因此单条序列的 Token 维存在真实依赖。KV Cache 只能避免重复计算历史 K/V，不能消除“下一个 Token 依赖上一个 Token”的顺序性。

5.10 参数量的系统估算

忽略 bias 和归一化参数，设 Dense Decoder 有 N 层、隐藏维度 d 、词表 V 、中间维度 d_{ff} 。

5.10.1 Attention 参数

MHA 的 W_Q, W_K, W_V, W_O 均近似 $d \times d$ ：

$$P_{attn} \approx 4d^2. \quad (5.31)$$

GQA 时 K/V 输出维度缩小为 $H_{kv}d_h$ ：

$$P_{attn} \approx 2d^2 + 2d(H_{kv}d_h), \quad (5.32)$$

其中两块 d^2 对应 Q 和 O。

5.10.2 MLP 参数

普通 FFN：

$$P_{ffn} \approx 2dd_{ff}. \quad (5.33)$$

门控 FFN：

$$P_{ffn} \approx 3dd_{ff}. \quad (5.34)$$

5.10.3 总参数

若输入输出权重共享：

$$P \approx Vd + N(P_{\text{attn}} + P_{\text{ffn}}). \quad (5.35)$$

若不共享，还需再加 Vd 。

例：粗估一个 7B 级模型

设 $N = 32, d = 4096, d_{\text{ff}} = 11008, V = 32000$ ，使用门控 FFN 和近似 MHA：

$$\begin{aligned} P_{\text{embed}} &\approx 32000 \times 4096 \approx 0.131\text{B}, \\ P_{\text{attn/layer}} &\approx 4 \times 4096^2 \approx 0.067\text{B}, \\ P_{\text{ffn/layer}} &\approx 3 \times 4096 \times 11008 \approx 0.135\text{B}, \\ P_{\text{blocks}} &\approx 32 \times (0.067 + 0.135) \approx 6.46\text{B}. \end{aligned}$$

加上 embedding、归一化等，得到约 6.6-7B。该估算能帮助检查配置，却不能替代实际模型清单。

5.11 FLOPs 的粗略估算

对每个 Token，一次 $d \times d'$ 线性层前向约需 $2dd'$ FLOPs。标准 Dense Block 的线性层计算量近似：

$$C_{\text{linear/token}} \approx 2(4d^2 + 3dd_{\text{ff}}) \quad (5.36)$$

(门控 MLP)。Attention score 和加权聚合还增加约

$$C_{\text{attn/token}} \sim 4Ld \quad (5.37)$$

量级，完整序列则呈 L^2d 。短序列时线性层常占主导；长序列时二次注意力项上升。

训练还包含反向传播。经验上，Dense Transformer 对每个训练 Token 的总计算常粗估为参数量的约 6 倍：

$$C_{\text{train}} \approx 6ND_{\text{token}}, \quad (5.38)$$

其中 N 是参数量， D_{token} 是训练 Token 数。该式是数量级估计，不含稀疏激活、重计算、通信和低利用率等细节。

5.12 模型参数“存储知识”是什么意思

参数并不是键值数据库。训练通过大量梯度更新，使神经网络形成对模式和条件分布的压缩表示：

- 部分事实和语言规律分布在许多权重中；
- 同一参数同时参与大量上下文和任务；
- 精确回忆取决于提示、上下文、解码和模型状态；
- 修改一条知识可能影响其他行为，这使“精确编辑”困难。

所以“模型知道某事实”通常表示在合适提示下以较高概率生成正确内容，不表示存在可直接定位的一条记录。

5.13 训练模式与推理模式

- 训练需要保存激活用于反向传播，并可能启用 **dropout**；
- 推理无需梯度，可释放中间激活，但自回归生成需要 **KV Cache**；
- 某些层在 **train/eval** 模式行为不同，错误模式会导致评测不一致；
- 推理通常只使用最后位置 **logit**，但 **Prefill** 仍计算所有输入位置。

5.14 本章综合练习

练习 1：输出 shape

输入 shape 为 $B = 8, L = 1024, d = 4096$ ，词表 $V = 50000$ 。LM Head 的 **logit shape** 是什么？若全部以 **BF16** 显式保存，约占多少内存？

解答

shape 为 $8 \times 1024 \times 50000$ ，元素数约 4.096×10^8 ，**BF16** 约占 819.2 MB（十进制）或约 781 MiB。训练框架常使用融合交叉熵、张量并行或分块避免长期保存完整 **logit**。

练习 2：残差约束

为什么 **Attention** 和 **MLP** 子层最终输出必须回到隐藏维度 d ？

解答

因为要与残差主干 $x \in \mathbb{R}^d$ 相加。**Attention** 可在内部拆成多头，**MLP** 可扩展到 d_{ff} ，但输出投影必须回到 d ，否则无法逐元素相加。

本节要点

Decoder-only LLM 的计算链是：Token ID 查表得到 **embedding**，经位置处理和多层“**Norm-Attention-残差-Norm-MLP-残差**”变换，最后由 LM Head 输出词表 **logit**。**Teacher Forcing** 让训练并行处理全部位置，自回归依赖使生成逐 Token 进行。参数量主要来自 **Attention** 投影、**MLP** 和词表矩阵。

注意力机制：Q、K、V 的完整推导

本章学习目标

读完本章后，应能够：

1. 从“按内容读取信息”的目标写出注意力公式；
2. 解释 $1/\sqrt{d_k}$ 缩放因子的统计原因；
3. 写出单头、多头、GQA 与 MQA 的精确 shape；
4. 区分计算复杂度、显存复杂度和 IO 复杂度；
5. 解释 FlashAttention 为什么保持精确结果却更快。

6.1 从固定权重到动态信息路由

普通线性层使用固定权重矩阵：同一参数对所有输入执行相同映射。序列任务还需要一种能力：当前位置应根据当前内容，动态决定读取上下文中的哪些位置。

假设上下文有 L 个信息项，每项有键 k_j 和值 v_j 。当前位置产生查询 q_i ，先计算匹配分数 $s(q_i, k_j)$ ，再归一化为权重 a_{ij} ：

$$e_{ij} = s(q_i, k_j), \quad (6.1)$$

$$a_{ij} = \frac{\exp(e_{ij})}{\sum_{m=1}^L \exp(e_{im})}, \quad (6.2)$$

$$o_i = \sum_{j=1}^L a_{ij} v_j. \quad (6.3)$$

这就是**软注意力**：不是只选择一个位置，而是对所有可见位置做可微分加权和。

直觉理解

Query 表示“我现在要找什么”；Key 表示“我能用什么特征被找到”；Value 表示“找到我后实际读取什么”。Key 和 Value 分开，使“寻址依据”和“传递内容”可以不同。

6.2 从输入到 Q、K、V

给定单个样本的隐藏状态

$$X \in \mathbb{R}^{L \times d}, \quad (6.4)$$

通过三个可学习投影：

$$Q = XW_Q, \quad W_Q \in \mathbb{R}^{d \times d_k}, \quad (6.5)$$

$$K = XW_K, \quad W_K \in \mathbb{R}^{d \times d_k}, \quad (6.6)$$

$$V = XW_V, \quad W_V \in \mathbb{R}^{d \times d_v}. \quad (6.7)$$

因此

$$Q, K \in \mathbb{R}^{L \times d_k}, \quad V \in \mathbb{R}^{L \times d_v}. \quad (6.8)$$

第 i 行 q_i 是第 i 个位置的 Query，第 j 行 k_j, v_j 是第 j 个位置的 Key 和 Value。

6.3 为什么使用点积相似度

最常用分数是点积：

$$e_{ij} = q_i^\top k_j. \quad (6.9)$$

点积可批量写成矩阵乘法

$$E = QK^\top \in \mathbb{R}^{L \times L}, \quad (6.10)$$

适合 GPU 高吞吐执行。与余弦相似度不同，点积同时受方向和向量范数影响；模型可通过学习投影自行利用范数表示置信度或强度。

6.3.1 缩放因子 $1/\sqrt{d_k}$ 的推导

假设 q 、 k 各分量独立、均值为 0、方差为 1：

$$q^\top k = \sum_{r=1}^{d_k} q_r k_r. \quad (6.11)$$

每项 $q_r k_r$ 期望为 0。若近似独立，方差为

$$\text{Var}(q^\top k) = \sum_{r=1}^{d_k} \text{Var}(q_r k_r) \quad (6.12)$$

$$\approx d_k. \quad (6.13)$$

因此点积标准差约为 $\sqrt{d_k}$ 。维度越大，logit 幅度越大，Softmax 更容易饱和到接近 one-hot，梯度变小。除以 $\sqrt{d_k}$ 后：

$$\text{Var}\left(\frac{q^\top k}{\sqrt{d_k}}\right) \approx 1. \quad (6.14)$$

这使不同 head dimension 下的分数尺度更稳定。

常见误区

缩放因子不是为了让注意力权重“总和等于 1”；总和为 1 是 Softmax 的作用。缩放是为了控制输入 Softmax 的 logit 方差。

6.4 缩放点积注意力

加入 mask M 后, 完整公式为

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V. \quad (6.15)$$

令

$$A = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) \in \mathbb{R}^{L_q \times L_k}, \quad (6.16)$$

则

$$O = AV \in \mathbb{R}^{L_q \times d_v}. \quad (6.17)$$

Softmax 沿 Key 维执行, 即每个 Query 的一行权重和为 1。

6.4.1 逐行展开

对第 i 个 Query:

$$e_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} + M_{ij}, \quad (6.18)$$

$$a_{ij} = \frac{e^{e_{ij}}}{\sum_m e^{e_{im}}}, \quad (6.19)$$

$$o_i = \sum_j a_{ij} v_j. \quad (6.20)$$

注意 o_i 是 Value 的线性组合, 而不是 Key 的线性组合。

6.5 一个可手算的数值例子

设 $d_k = d_v = 2$, 三个位置:

$$\begin{aligned} q_1 &= (1, 0), \\ k_1 &= (1, 0), \quad k_2 = (0, 1), \quad k_3 = (1, 1), \\ v_1 &= (1, 0), \quad v_2 = (0, 2), \quad v_3 = (3, 3). \end{aligned}$$

未缩放点积为

$$(q_1^\top k_1, q_1^\top k_2, q_1^\top k_3) = (1, 0, 1). \quad (6.21)$$

除以 $\sqrt{2}$ 后约为 $(0.707, 0, 0.707)$ 。Softmax 权重约为

$$(a_{11}, a_{12}, a_{13}) \approx (0.401, 0.198, 0.401). \quad (6.22)$$

输出为

$$o_1 = 0.401(1, 0) + 0.198(0, 2) + 0.401(3, 3) \quad (6.23)$$

$$\approx (1.604, 1.599). \quad (6.24)$$

输出不是复制某个位置, 而是根据内容匹配产生的加权表示。

6.6 因果 Mask 与 Padding Mask

6.6.1 因果 Mask

自回归模型中，第 i 个位置不能读取未来 $j > i$ ：

$$M_{ij}^{causal} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (6.25)$$

Softmax 后未来权重为 0。对长度 4：

$$M^{causal} = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}. \quad (6.26)$$

6.6.2 Padding Mask

批处理中序列长度不同，需要 PAD 补齐。Padding mask 禁止真实 Token 读取 PAD Key。通常把 causal mask 与 padding mask 相加或逻辑合并。

6.6.3 低精度实现细节

实际代码常使用 dtype 可表示的最小有限值，而非真正的 $-\infty$ 。若某一行全部被 mask，Softmax 可能得到 NaN。因此需要保证至少一个可见位置，或使用安全的融合内核。

6.7 多头注意力

单头在一个投影空间中匹配。多头注意力令不同头拥有不同投影：

$$Q_h = XW_Q^{(h)}, \quad (6.27)$$

$$K_h = XW_K^{(h)}, \quad (6.28)$$

$$V_h = XW_V^{(h)}, \quad (6.29)$$

$$O_h = \text{Attention}(Q_h, K_h, V_h). \quad (6.30)$$

拼接后：

$$\text{MHA}(X) = \text{Concat}(O_1, \dots, O_H)W_O. \quad (6.31)$$

若 $d = Hd_h$ ，则每个头输出 $L \times d_h$ ，拼接回 $L \times d$ 。

6.7.1 批量 shape

常见实现：

$$X : [B, L, d], \quad (6.32)$$

$$Q, K, V : [B, L, H, d_h], \quad (6.33)$$

$$\text{transpose} \rightarrow [B, H, L, d_h], \quad (6.34)$$

$$QK^\top : [B, H, L, L], \quad (6.35)$$

$$AV : [B, H, L, d_h], \quad (6.36)$$

$$\text{concat} \rightarrow [B, L, d]. \quad (6.37)$$

轴顺序可能不同，但数学含义一致。

6.7.2 多头不等于“每头对应一种固定语言关系”

不同头可能表现出局部、复制、语法或位置模式，但功能不是人工指定，也会跨层和 MLP 共同编码。某个头可被多个功能复用，许多头也可能冗余。把可视化权重直接解释为模型因果理由通常不可靠。

6.8 MHA、MQA 与 GQA

设 Query 头数为 H_q ，KV 头数为 H_{kv} ，每头维度 d_h 。

6.8.1 Multi-Head Attention (MHA)

$$H_{kv} = H_q. \quad (6.38)$$

每个 Query 头有独立 K/V，表达灵活，但 KV Cache 最大。

6.8.2 Multi-Query Attention (MQA)

$$H_{kv} = 1. \quad (6.39)$$

所有 Query 头共享一组 K/V，大幅降低 K/V 投影和 Cache，但共享过强可能影响质量。

6.8.3 Grouped-Query Attention (GQA)

$$1 < H_{kv} < H_q, \quad (6.40)$$

每组 H_q/H_{kv} 个 Query 头共享一组 K/V，是常见折中。

形式	H_{kv}	KV Cache	主要权衡
MHA	H_q	最大	头间 K/V 独立, 质量稳健
GQA	$1 < H_{kv} < H_q$	按 H_{kv}/H_q 缩小	质量与解码效率折中
MQA	1	最小	解码带宽低, 共享最强

6.8.4 参数量公式

Q 投影参数为

$$P_Q = d(H_q d_h) = d^2. \quad (6.41)$$

K/V 各为

$$P_K = P_V = d(H_{kv} d_h). \quad (6.42)$$

输出投影仍约为 d^2 。因此

$$P_{attn} = 2d^2 + 2dH_{kv}d_h. \quad (6.43)$$

MHA 时 $H_{kv}d_h = d$, 恢复 $4d^2$ 。

6.8.5 KV Cache 缩减比例

每个 Token、每层需存 Key 和 Value:

$$M_{token/layer} = 2H_{kv}d_h s, \quad (6.44)$$

s 为每元素字节数。因此从 MHA 改为 GQA 的 Cache 比例近似

$$\frac{M_{GQA}}{M_{MHA}} = \frac{H_{kv}}{H_q}. \quad (6.45)$$

例: GQA 节省多少 KV Cache

若 $H_q = 32, H_{kv} = 8$, 其他相同, 则 KV Cache 约降为 MHA 的 $8/32 = 1/4$ 。注意 Q 的计算、输出投影和 Attention 对历史长度的读取仍存在, 因此总体推理不会获得 4 倍加速。

6.9 Attention 的复杂度分解

设单个样本、隐藏维度 $d = Hd_h$ 、序列长度 L 。

6.9.1 QKV 与输出投影

四个大投影约为

$$\mathcal{O}(Ld^2). \quad (6.46)$$

6.9.2 分数矩阵与 Value 聚合

$QK^\top$ 和 AV 各约为

$$\mathcal{O}(L^2d). \quad (6.47)$$

因此总量级为

$$\mathcal{O}(Ld^2 + L^2d). \quad (6.48)$$

当 $L \ll d$, 投影和 MLP 可能主导; 当 L 很长, L^2d 变得突出。

6.9.3 显存与 IO 不等于 FLOPs

朴素实现会显式存储 $L \times L$ 分数和概率矩阵, 激活显存为 $\mathcal{O}(L^2)$ 。即使 FLOPs 可接受, 把这些大矩阵反复写入和读取 HBM 也可能成为瓶颈。

6.10 FlashAttention: 在线 Softmax 的关键思想

FlashAttention 不改变注意力定义, 而是分块计算并避免把完整 $L \times L$ 矩阵写入 HBM[8]。

6.10.1 为什么 Softmax 分块不容易

对一行分数 $x_1, \dots, x_L$:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}. \quad (6.49)$$

若只看到一块, 尚不知道全局最大值和分母, 不能直接独立归一化。

6.10.2 在线 Softmax 合并公式

对已处理块维护最大值 m 和指数和 ℓ :

$$m = \max_j x_j, \quad \ell = \sum_j e^{x_j - m}. \quad (6.50)$$

处理新块, 其最大值和指数和为 m_b, ℓ_b 。合并最大值

$$m' = \max(m, m_b). \quad (6.51)$$

旧块的指数和需重标定:

$$\ell' = e^{m - m'} \ell + e^{m_b - m'} \ell_b. \quad (6.52)$$

同样可重标定已累积的加权 Value 和。这样每个块只需在片上内存中停留, 最终结果与全量 Softmax 相同。

直觉理解

FlashAttention 的关键不是“少算很多乘法”, 而是“让数据尽量留在更快的片上存储中, 并用在线归一化避免保存完整分数矩阵”。这是一种 IO-aware 算法设计。

6.11 训练与推理中的注意力差异

6.11.1 训练

通常一次处理完整序列，需要全部位置输出和反向传播。计算可高度并行，但需保存或重算激活。

6.11.2 Prefill

输入提示一次性处理，形态类似训练前向但无反向。需要为每层建立全部提示 Token 的 K/V。

6.11.3 Decode

每步 Query 长度通常为 1，而 K/V 长度随历史增长：

$$Q : [B, H_q, 1, d_h], \quad K, V : [B, H_{kv}, L_{past}, d_h]. \quad (6.53)$$

每步不再形成完整 $L \times L$ ，但必须读取历史 KV Cache，因此常受显存带宽限制。

6.12 稀疏、局部与线性注意力

为减轻长序列成本，常见方法包括：

- **滑动窗口**：位置只读取附近 w 个 Token，复杂度约 $\mathcal{O}(Lwd)$ ；
- **块稀疏**：按预设图连接局部块和少量全局块；
- **检索式注意力**：动态选择少量相关位置；
- **线性注意力**：利用核分解重排 $Q(K^\top V)$ ，避免显式 L^2 ；
- **状态空间/递归模型**：维护压缩状态而非全部两两关系。

降低理论复杂度并不自动保证更快：内核效率、并行性、硬件利用率和有效上下文质量同样重要。

6.13 注意力权重能否解释模型

Attention 权重是某层某头某个 Query 对 Value 的局部加权系数，但模型最终输出还受到：

- Value 投影内容；
- 多头拼接和输出投影；
- 残差主干；
- MLP 非线性；
- 后续几十层变换。

因此权重可用于诊断和可视化，但不能直接等同于“模型做出结论的原因”。更严谨的分析通常需要干预、消融、因果追踪或梯度方法。

6.14 本章综合练习

练习 1：注意力 shape

$B = 4, L = 2048, d = 4096, H_q = 32, H_{kv} = 8, d_h = 128$ 。写出 Q、K、V reshape 后的 shape，以及单层 BF16 K/V Cache 的元素数。

解答

$$Q : [4, 32, 2048, 128], \quad K, V : [4, 8, 2048, 128].$$

K/V 总元素数为

$$2 \times 4 \times 8 \times 2048 \times 128 = 16,777,216,$$

BF16 约 32 MiB/层。

练习 2：复杂度主导项

比较 $L = 1024, d = 8192$ 与 $L = 32768, d = 4096$ 时， Ld^2 和 L^2d 的数量级，说明哪个场景更受二次注意力影响。

解答

第一种： $Ld^2 \approx 6.9 \times 10^{10}$ ， $L^2d \approx 8.6 \times 10^9$ ，线性投影更大。第二种：两者都约 5.5×10^{11} ，二次项已与投影同量级；继续增大 L 时注意力项更快增长。

本节要点

注意力是一种按内容动态寻址的可微分读操作：Q 与 K 产生匹配分数，Softmax 得到权重，再对 V 聚合。 $1/\sqrt{d_k}$ 稳定分数方差；多头提供多个投影子空间；GQA/MQA 通过共享 K/V 降低 Cache；FlashAttention 用在线 Softmax 和分块减少 HBM IO，而不改变精确注意力结果。

位置编码、上下文窗口与 KV Cache

本章学习目标

读完本章后，应能够：

1. 解释没有位置编码的自注意力为何无法区分顺序；
2. 推导正弦位置编码和 RoPE 的相对位置性质；
3. 区分训练长度、可输入长度和有效利用长度；
4. 从模型配置计算 KV Cache 显存；
5. 解释 Prefill、Decode、Prefix Cache 和滑动窗口的关系。

7.1 为什么自注意力本身不知道顺序

若不加入位置信息，自注意力对输入位置的排列具有**置换等变性**。设置换矩阵为 P ，把输入 X 重排为 PX 。由于

$$Q' = PXW_Q = PQ, \quad (7.1)$$

$$K' = PK, \quad (7.2)$$

$$V' = PV, \quad (7.3)$$

且

$$Q'K'^\top = PQK^\top P^\top, \quad (7.4)$$

Softmax 按行应用后，输出也只是相应重排：

$$\text{Attn}(PX) = P \text{Attn}(X). \quad (7.5)$$

这意味着模型知道“有哪些 Token”，但不能仅凭注意力区分它们的绝对或相对顺序。必须显式引入位置结构。

7.2 绝对位置编码

最直接方法是为第 t 个位置提供向量 $p_t \in \mathbb{R}^d$ ，与 Token embedding 相加：

$$h_t^{(0)} = e(x_t) + p_t. \quad (7.6)$$

p_t 可以是可学习参数，也可以是固定函数。

7.2.1 正弦位置编码

经典 Transformer 使用不同频率的正弦、余弦：

$$PE(t, 2i) = \sin\left(t/10000^{2i/d}\right), \quad (7.7)$$

$$PE(t, 2i+1) = \cos\left(t/10000^{2i/d}\right). \quad (7.8)$$

低维通道变化快，编码局部位置；高维通道变化慢，编码更长尺度。

7.2.2 为什么正弦编码包含相对位置信息

对固定频率 ω ：

$$\sin((t+\Delta)\omega) = \sin(t\omega)\cos(\Delta\omega) + \cos(t\omega)\sin(\Delta\omega), \quad (7.9)$$

$$\cos((t+\Delta)\omega) = \cos(t\omega)\cos(\Delta\omega) - \sin(t\omega)\sin(\Delta\omega). \quad (7.10)$$

因此位置 $t+\Delta$ 的二维编码可由位置 t 的编码乘一个只依赖 Δ 的旋转矩阵得到。这使网络有机会通过线性变换学习相对位移关系。

常见误区

固定正弦编码“可以计算到任意位置”不等于模型“能有效外推到任意长度”。模型只在有限长度和位置分布上训练，注意力模式、归一化和内容分布仍可能在更长序列上失效。

7.3 RoPE: 把位置写入 Q 和 K

RoPE 不把位置向量直接加到隐藏状态，而是对每个位置的 Q/K 子向量执行旋转 [6]。先看二维情形。位置 t 的旋转矩阵为

$$R_t(\omega) = \begin{bmatrix} \cos(t\omega) & -\sin(t\omega) \\ \sin(t\omega) & \cos(t\omega) \end{bmatrix}. \quad (7.11)$$

对 Query 和 Key：

$$\tilde{q}_t = R_t q_t, \quad \tilde{k}_s = R_s k_s. \quad (7.12)$$

7.3.1 RoPE 的相对位置性质

旋转矩阵满足 $R_t^\top = R_{-t}$ 和 $R_a R_b = R_{a+b}$ ，因此

$$\tilde{q}_t^\top \tilde{k}_s = (R_t q_t)^\top (R_s k_s) \quad (7.13)$$

$$= q_t^\top R_t^\top R_s k_s \quad (7.14)$$

$$= q_t^\top R_{s-t} k_s. \quad (7.15)$$

最终点积只通过 $s-t$ 依赖相对位置。这是 RoPE 的核心：绝对位置旋转后，注意力分数自然包含相对位移。

复数视角

把二维向量 (x_1, x_2) 看成复数 $z = x_1 + ix_2$ 。旋转等价于

$$\tilde{z}_t = z e^{it\omega}.$$

Query 与 Key 的相互关系包含

$$e^{it\omega} \overline{e^{is\omega}} = e^{i(t-s)\omega},$$

只依赖位置差。实际模型把 **head dimension** 两两配对，并为不同维对使用不同频率。

7.3.2 多频率 RoPE

对 d_h 维向量，将相邻维两两配对，共 $d_h/2$ 个频率：

$$\omega_i = \theta^{-2i/d_h}, \quad (7.16)$$

其中 θ 常称为 **RoPE base**。不同频率使模型同时表示局部和长距离位置信息。

7.4 相对位置偏置与 ALiBi

另一类方法直接给注意力分数加入与距离有关的偏置：

$$e_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} + b(i - j). \quad (7.17)$$

ALiBi 使用不同 **head** 的线性距离惩罚：

$$e_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} - m_h(i - j), \quad j \leq i, \quad (7.18)$$

其中 $m_h > 0$ 是第 h 个头的斜率。它不需要位置 **embedding** 表，但远距离位置会受到更强先验惩罚。

7.5 上下文窗口的三个不同含义

“模型支持 128K 上下文”可能混合了三个概念：

1. **训练长度**：预训练或长上下文阶段实际见过的序列长度分布；
2. **接口/实现上限**：位置编码、内核和显存允许输入的最大 **Token** 数；
3. **有效上下文长度**：在给定任务上仍能可靠检索、组合和推理的长度。

第三项通常小于第二项，而且依任务、信息位置、干扰内容和提示结构而变。

常见误区

上下文窗口不是长期记忆。请求结束后，模型权重和 **KV Cache** 通常不会自动保存本次对话事实。产品中的“记忆”往往由数据库、摘要、用户画像或检索系统实现。

7.6 长度外推与位置缩放

若模型只在长度 L_{train} 内训练，却直接输入远大于该长度的位置，RoPE 角度可能进入未见分布。常见扩展思路包括：

- **位置插值**：把长序列位置压缩回训练范围，例如 $t' = t \cdot L_{train}/L_{target}$ ；
- **频率缩放**：调整 RoPE base 或不同频率的增长速度；
- **继续训练**：使用长序列数据适配新的位置分布；
- **局部/分块注意力**：控制远距离计算和缓存；
- **检索压缩**：不把所有历史原样放入上下文，只保留相关内容。

任何扩展都可能在局部分辨率、远距离外推、训练成本和短上下文能力之间做权衡。

7.7 KV Cache 为什么存在

自回归生成第 t 个 Token 时，第 ℓ 层需要当前 Query 与全部历史 Key/Value 做注意力。若每一步都重新计算历史 Token 的 K/V，会重复执行大量投影和前层计算。

对第 t 步：

$$K_{1:t} = [K_{1:t-1}; K_t], \quad (7.19)$$

$$V_{1:t} = [V_{1:t-1}; V_t]. \quad (7.20)$$

历史 $K_{1:t-1}, V_{1:t-1}$ 一旦由固定模型计算完成便不会改变，因此可缓存，只为新 Token 计算 K_t, V_t 。这就是 KV Cache。

7.7.1 没有 Cache 的重复计算

生成 T 个新 Token，若每一步重新前向整个前缀，处理 Token 数近似

$$1 + 2 + \dots + T = \frac{T(T+1)}{2} = \mathcal{O}(T^2). \quad (7.21)$$

使用 Cache 后，历史层表示无需重复通过模型，每步只处理一个新 Token 的主干计算；但新 Query 仍要读取长度不断增长的历史 K/V。

7.8 KV Cache 显存公式

设 batch 为 B ，总缓存长度 L ，层数 N ，KV 头数 H_{kv} ，每头维度 d_h ，每元素 s 字节。Key 和 Value 各一份：

$$M_{KV} = B \times L \times N \times 2 \times H_{kv} \times d_h \times s. \quad (7.22)$$

若考虑 allocator、对齐、分页元数据和临时 buffer，实际会更高。

例：16K 上下文的 KV Cache

设 $B = 1, L = 16384, N = 32, H_{kv} = 8, d_h = 128$, BF16 的 $s = 2$:

$$\begin{aligned} M_{KV} &= 1 \times 16384 \times 32 \times 2 \times 8 \times 128 \times 2 \\ &= 2,147,483,648 \text{ bytes} \approx 2 \text{ GiB}. \end{aligned}$$

若是 MHA 且 $H_{kv} = 32$, 则约 8 GiB。GQA 对长上下文并发非常关键。

7.8.1 每新增一个 Token 的 Cache

单请求每 Token Cache 为

$$m_{token} = N \times 2 \times H_{kv} \times d_h \times s. \quad (7.23)$$

以上例每 Token 为

$$32 \times 2 \times 8 \times 128 \times 2 = 131,072 \text{ bytes} = 128 \text{ KiB}.$$

生成 1000 个新 Token 还会增加约 125 MiB。

7.9 Prefill 与 Decode**7.9.1 Prefill**

Prefill 一次处理全部输入 Token:

- 计算量随提示长度增长;
- 大矩阵乘法并行度高, 通常更偏 compute-bound;
- 建立所有层的输入 KV Cache;
- 主要决定 TTFT 中的模型计算部分。

7.9.2 Decode

Decode 每步通常只输入一个 Token:

- GEMM 的 batch/token 维较小, 硬件利用率更难拉满;
- 每步读取模型权重和历史 KV Cache;
- 常更偏 memory-bandwidth-bound;
- 主要决定 TPOT 和长输出延迟。

工程视角

同一个模型的 Prefill 和 Decode 是两种不同工作负载。优化长提示 TTFT 与优化大量输出 Token 的 TPOT, 往往需要不同的批处理、并行和内核策略。

7.10 Prefix Cache 与 Prompt Cache

若多个请求共享完全相同的 Token 前缀，可复用该前缀对应的 KV Cache，避免重复 Prefill。例如固定 system prompt、长工具说明或公共文档前缀。

复用条件通常包括：

- 模型权重、adapter 和推理配置一致；
- Token 序列逐项一致，而非文本“看起来相同”；
- 位置编号和 attention mask 兼容；
- Cache 未因内存压力被回收。

Prefix Cache 节省计算，但占用额外显存，需要命中率和淘汰策略。

7.11 PagedAttention 与缓存碎片

不同请求长度和生命周期不同，连续大块分配 KV Cache 容易产生外部碎片。PagedAttention 把 Cache 分成固定大小逻辑块，通过映射表将逻辑序列映射到物理块 [9]。这类似虚拟内存分页：

- 请求可按需增长，无需预留最大连续空间；
- 空闲块可快速复用；
- 多个序列前缀可共享物理块；
- 代价是块表管理和内核间接寻址。

7.12 滑动窗口、Cache 驱逐与摘要

无限保存历史 Cache 会使显存和每步注意力读取持续增长。常见控制方式：

- **滑动窗口**：只保留最近 w 个 Token 的 K/V；
- **Sink Token**：保留开头少量特殊位置加最近窗口；
- **分层压缩**：把旧历史摘要为更少 Token 或状态；
- **检索记忆**：把旧内容移出上下文，需要时检索回来；
- **KV 量化**：降低每元素字节数，但不降低 Token 数。

丢弃 Cache 会失去对应细粒度信息，不能仅靠“模型记住了”来弥补。

7.13 长上下文为何容易失效

7.13.1 Lost in the Middle

关键信息位于长上下文中部时，模型可能不如位于开头或结尾时稳定。可能原因包括训练位置分布、注意力偏置、干扰项增加和任务要求跨段组合。

7.13.2 信号稀释

检索到的相关片段若夹在大量无关内容中，注意力和后续层需要从更多竞争信号中恢复目标。上下文越长不一定越好。

7.13.3 组合而非检索

“在长文中找到一句话”与“跨几十处证据做多步推理”难度不同。针测试验只能度量检索，不足以代表长上下文推理。

7.13.4 位置外推

即使接口接受更长输入，未充分适配的位置频率可能导致局部和全局关系退化。

7.14 如何评测长上下文

应至少分层评测：

1. 不同位置的精确检索；
2. 多证据聚合；
3. 顺序、计数和状态跟踪；
4. 长代码/长文档的一致性；
5. 不同长度下的短任务回归；
6. TTFT、KV 显存和并发下降。

只报告“最大可输入 Token 数”不能说明有效能力和系统成本。

7.15 本章综合练习

练习 1：RoPE 相对性

证明二维旋转矩阵满足 $R_t^\top R_s = R_{s-t}$ ，并解释这为何使注意力点积依赖相对距离。

解答

$R_t^\top = R_{-t}$ ，旋转矩阵相乘角度相加，所以 $R_t^\top R_s = R_{-t} R_s = R_{s-t}$ 。于是 $(R_t q)^\top (R_s k) = q^\top R_{s-t} k$ ，位置只通过 $s - t$ 进入分数。

练习 2：并发 Cache

单请求 32K 上下文需要 4 GiB KV Cache。若 GPU 可用于 KV 的空间为 48 GiB，忽略碎片和临时空间，理论最多同时容纳多少个满长度请求？为什么实际更少？

解答

理论上 $48/4 = 12$ 个。实际还需模型权重、运行时 **buffer**、采样状态、分页碎片和输出增长余量；请求长度也动态变化，因此调度器必须保留安全空间。

本节要点

位置编码打破自注意力的置换对称性。**RoPE** 通过旋转 **Q/K**，使点积自然依赖相对位置。上下文能力应区分训练长度、实现上限和有效利用长度。**KV Cache** 用显存换取避免历史重算，其成本与 **batch**、长度、层数和 **KV** 头数线性增长；**Prefill** 偏计算，**Decode** 偏带宽，长上下文最终同时受到质量和系统资源约束。

现代架构组件：MoE、长序列与模型家族

8.1 Dense 模型与 MoE 模型

Dense 模型每个 Token 都经过所有参数相关的层。**Mixture of Experts (MoE)** 通常把 FFN 替换为多个专家，并由 Router 为每个 Token 选择少数专家：

$$y = \sum_{i \in \text{TopK}(g(x))} p_i(x) E_i(x). \quad (8.1)$$

总参数可以很大，但每 Token 激活参数较少，因此在相近计算量下提高容量。

8.2 Router、Top-k 与负载均衡

Router 输出专家分数，Top-1 或 Top-2 路由选择专家。训练需要负载均衡损失，避免所有 Token 集中到少数专家。还需处理：

- **专家容量 (capacity)**：每批次每个专家能接收多少 Token；
- **Token dropping**：超容量 Token 是否丢弃或转发；
- **Expert parallelism**：专家分布在不同设备，产生 all-to-all 通信；
- **路由抖动**：小输入变化可能改变专家选择。

常见误区

MoE 的“总参数”与“激活参数”必须分开。总参数影响权重存储和加载；激活参数更接近每 Token 计算量，但通信、路由和内存带宽仍会影响延迟。

8.3 模型家族的命名维度

看模型名称时，应辨认以下维度：

- Base / Instruct / Chat / Reasoning；
- Dense / MoE；
- 参数规模与激活参数；
- 上下文长度；
- 文本模型或多模态模型；
- 预训练语言、代码或领域侧重；

- 权重许可证与商用限制；
- 原生精度和推荐量化格式。

8.4 Encoder、Decoder 与嵌入模型不能混用

用于向量检索的 **Embedding Model** 通常输出句子或文档向量，训练目标强调语义相似度；生成模型输出 **Token** 分布。虽然生成模型的隐藏状态也可池化成向量，但未经专门对比学习时，检索效果通常不稳定。

8.5 替代 Transformer 的方向

状态空间模型、线性递归、卷积混合架构和扩散式文本模型都在尝试改善长序列复杂度、并行生成或硬件效率。评价新架构时要分别看：训练扩展性、上下文质量、生成延迟、工具生态、可微调性与实际任务能力，而不是只看理论复杂度。

本节要点

现代 LLM 架构优化主要围绕容量、长上下文和硬件效率。**MoE** 用稀疏激活扩大参数容量，但引入路由与通信复杂度；不同模型家族还需区分训练阶段、模态和许可证。

第三部分

第三部分：模型是如何训练出来的

预训练：数据、目标、计算与 Scaling Law

本章学习目标

读完本章后，应能够：

1. 从语料构造自回归预训练样本和 loss；
2. 区分 Token 数、样本数、step、epoch 与有效 batch；
3. 解释数据去重、混合、质量过滤和污染控制的作用；
4. 使用 $6ND$ 粗估训练计算，并理解其适用边界；
5. 理解 Scaling Law 是经验关系，而不是“参数越大一定越好”的口号。

9.1 预训练学到的是什么

预训练用大规模无标注文本进行自监督学习。对 Decoder-only 模型，训练目标是根据前缀预测下一个 Token：

$$\mathcal{J}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \left[\sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}) \right]. \quad (9.1)$$

标签来自文本自身，不需要人工逐条标注，因此可以利用网页、书籍、论文、代码和多语种语料。

模型为了降低广泛语料上的预测损失，会学习词法、语法、语义、事实关联、代码模式和部分推理结构。但目标仍是条件概率拟合，不直接保证真实性、服从性或安全性。

9.2 从原始文档到训练 Token

典型数据链路包括：

1. 数据获取与许可证/来源记录；
2. 文档解析、编码修复、正文抽取；
3. 语言识别、质量过滤和安全过滤；
4. 文档级、段落级和近重复去重；
5. 隐私与敏感信息处理；
6. 评测集污染检查；
7. Tokenization；
8. 按长度切片、拼接和打包；

9. 按数据混合权重采样。

模型训练看到的是最终 Token 流，而不是原始网页本身。任何解析错误、重复、模板噪声和乱码都会进入梯度。

9.3 文档边界、EOS 与 Packing

多篇文档可在固定长度序列中打包：

$$[d_1, \langle EOS \rangle, d_2, \langle EOS \rangle, \dots]. \quad (9.2)$$

EOS 告诉模型文档结束。是否允许后一篇文档注意前一篇，取决于 attention mask 设计：

- 连续 Token 流可提高利用率，但引入跨文档无意义上下文；
- block-diagonal mask 可隔离样本，却需要内核支持；
- 直接 padding 最简单，但浪费计算。

9.4 Token、样本、Step 与 Epoch

设每卡 micro-batch 为 B_m ，数据并行设备数 N_{DP} ，累积步数 K ，每个样本有效长度近似 L ：

$$B_{global} = B_m N_{DP} K, \quad (9.3)$$

$$D_{step} \approx B_{global} L. \quad (9.4)$$

训练 S 个 optimizer steps，看到的 Token 数约为

$$D_{train} \approx S B_m N_{DP} K L. \quad (9.5)$$

若数据集总 Token 数为 D_{data} ，有效 epoch 为

$$E_{epoch} = \frac{D_{train}}{D_{data}}. \quad (9.6)$$

大规模预训练常以 Token 数而不是 epoch 描述，因为语料可能持续变化、按权重重采样，并非整齐遍历一次。

例：根据配置计算训练 Token

每卡 micro-batch=1，数据并行 512 卡，累积 8，序列长 4096，训练 50,000 步：

$$D_{step} = 1 \times 512 \times 8 \times 4096 \approx 16.78\text{M Token},$$

总计约 8.39×10^{11} ，即 839B Token。

9.5 数据混合与采样权重

设有 K 个数据源，每个源分布为 $\mathcal{D}_k$ ，采样权重 w_k 满足 $\sum_k w_k = 1$ ：

$$x \sim \sum_{k=1}^K w_k \mathcal{D}_k. \quad (9.7)$$

权重不必等于原始体量占比。小而高质量的数据源常被上采样，大量低质量网页被下采样。

若第 k 个数据源有 D_k Token，训练总 Token 为 D ，其期望被采样次数相当于

$$E_k = \frac{w_k D}{D_k} \quad (9.8)$$

个 epoch。小数据源上采样过多会过拟合和记忆，过少则能力不足。

9.6 为什么去重重要

重复样本会：

- 让少数文档获得不成比例的梯度权重；
- 增加记忆和隐私泄漏风险；
- 浪费计算；
- 使评测污染更难识别；
- 降低数据多样性。

精确哈希只能发现完全相同文本；MinHash、局部敏感哈希、n-gram 指纹和 embedding 可检测近重复，但阈值过严也会误删模板相似而内容不同的文档。

9.7 质量过滤的偏差

质量分类器可能依据语言流畅度、格式、来源和内容特征打分。过滤能减少垃圾文本，但也会引入偏差：

- 偏向主流语言和正式文体；
- 删除方言、口语或小众领域；
- 把模型自身偏好写入数据选择；
- 过度过滤负面安全样本，反而削弱识别能力。

因此数据质量不是单一分数，而是准确性、多样性、合法性、代表性和任务价值的组合。

9.8 训练计算量：为什么常见 $6ND$

设参数数量为 N ，训练 Token 数为 D 。对 Dense Transformer，单 Token 前向中，每个参数大致参与一次乘法和一次加法，约 $2N$ FLOPs。反向传播包含：

- 对激活求梯度，量级约与前向相当；
- 对权重求梯度，量级约与前向相当。

因此前向加反向约为

$$2N + 2N + 2N = 6N \quad (9.9)$$

FLOPs/Token，得到

$$C_{train} \approx 6ND. \quad (9.10)$$

例：粗估 7B 模型训练计算

$N = 7 \times 10^9$, $D = 10^{12}$ Token:

$$C \approx 6 \times 7 \times 10^9 \times 10^{12} = 4.2 \times 10^{22} \text{ FLOPs.}$$

若集群持续有效吞吐为 10^{18} FLOP/s, 则理想计算时间约 4.2×10^4 秒, 即约 11.7 小时; 现实还需考虑利用率、通信、检查点、故障和数据加载, 实际更长。

9.8.1 $6ND$ 的边界

该估算忽略:

- Attention 的 L^2 项;
- MoE 只激活部分参数;
- Activation Checkpointing 的重计算;
- 输出词表和稀疏操作差异;
- 通信与低硬件利用率;
- 优化器、数据和检查点开销。

它适合数量级和预算核验, 不是精确性能模型。

9.9 Model FLOPs Utilization

若硬件理论峰值为 F_{peak} , 设备数 G , 训练墙钟时间 T , 估算实际模型 FLOPs 为 C , 则

$$\text{MFU} = \frac{C}{F_{peak}GT}. \quad (9.11)$$

MFU 反映模型数学计算占理论峰值的比例。它会受到矩阵尺寸、通信、padding、重计算、内核、故障和数据管道影响。不同论文的 FLOP 计数口径可能不同, 比较时要核对定义。

9.10 Scaling Law 的基本思想

Scaling Law 描述模型规模 N 、数据规模 D 、计算 C 与验证损失之间的经验幂律关系 [4]。简化形式可写为

$$L(N, D) \approx L_{\infty} + aN^{-\alpha} + bD^{-\beta}. \quad (9.12)$$

其中 L_{∞} 是不可约损失, $\alpha, \beta > 0$ 由实验拟合。含义是: 增大模型或数据通常持续降低损失, 但边际收益递减。

9.10.1 固定计算预算的权衡

训练计算近似 $C \propto ND$ 。固定 C 时, 参数和数据不能同时任意增大。若模型过大、数据过少, 参数未充分训练; 若模型过小、数据过多, 容量成为瓶颈。

Chinchilla 工作强调在固定计算下使用更多训练 Token、相对更小模型可获得更优损失 [5]。但“最优”依目标而异：推理成本、延迟、数据可得性、训练复用和下游能力都可能改变选择。

常见误区

Scaling Law 是特定数据、架构、优化和指标下的经验规律，不意味着所有能力平滑增长，也不能直接预测安全、工具使用或分布外任务。某些能力看似“涌现”，也可能来自评测阈值和度量非线性。

9.11 学习率、Batch 与训练稳定性

大规模预训练常见稳定性问题包括 loss spike、梯度爆炸、NaN、数据异常和并行不同步。关键因素：

- 初始化和残差尺度；
- warmup 与峰值学习率；
- global batch 和 Token 数归一化；
- 梯度裁剪；
- BF16/FP16 数值范围；
- optimizer state 一致性；
- 数据样本长度、空样本和极端字符；
- checkpoint 恢复后的随机数、调度器和数据游标。

9.11.1 Loss Spike 的处理

偶发 spike 可能来自难样本或数据分布变化，不一定需要回滚；持续发散则需定位。常见策略：跳过异常 batch、降低学习率、恢复较早 checkpoint、记录样本 ID、检查梯度范数和逐层激活。不能只看全局 loss，因为异常可能被平均掩盖。

9.12 持续预训练与领域预训练

在已有 Base Model 上继续下一 Token 训练称为 Continued Pretraining (CPT)。它适合：

- 适配法律、医学、金融、代码等领域语言；
- 引入新时间段或新语种知识；
- 扩展上下文长度；
- 改善特定格式和术语建模。

CPT 仍是语言建模目标，与 SFT 的“提示-回答行为”不同。领域数据过于单一会导致灾难性遗忘，常混入一定比例通用语料并做能力回归。

9.13 数据污染与可记忆性

评测题进入预训练、SFT、合成数据或提示模板，会让分数虚高。污染检测可比较 **n-gram**、近重复、时间戳和来源，但无法完全证明“未见过”。应：

- 在建库阶段维护评测黑名单；
- 使用保密或动态生成测试；
- 报告训练数据时间边界和去重方法；
- 分析答案模板匹配与异常高置信度；
- 用新题、对抗变体和执行验证补充静态 **benchmark**。

9.14 预训练评测

除验证 **loss/PPL** 外，还应观察：

- 不同数据源和语种的分片 **loss**；
- 训练稳定性、梯度范数和异常率；
- 知识、代码、数学、长上下文和安全基线；
- 记忆、隐私和污染风险；
- 每 **Token** 训练成本与系统利用率。

验证 **loss** 是最直接的训练指标，但最终模型能力需要多维下游评测。

9.15 本章综合练习

练习 1：数据源有效 Epoch

某代码数据源有 100B Token，总训练计划为 2T Token，代码采样权重为 20%。该数据源期望被采样多少个 epoch？

解答

代码期望采样量为 $2T \times 0.2 = 400B$ Token，相当于 $400B/100B = 4$ 个 epoch。

练习 2：训练预算

估算 70B Dense 模型训练 2T Token 的 FLOPs。若集群有效算力为 5 EFLOP/s，理想需要多少天？

解答

$$C \approx 6 \times 70 \times 10^9 \times 2 \times 10^{12} = 8.4 \times 10^{23} \text{ FLOPs.}$$

5 EFLOP/s 为 5×10^{18} FLOP/s，时间约 1.68×10^5 秒，即 1.94 天。现实墙钟时间会因非理想利用率和非模型开销更长。

本节要点

预训练把原始语料经过清洗、去重、混合和 Tokenization 转成下一 Token 预测任务。Token 数决定训练数据规模， $6ND$ 可做 Dense 模型计算量粗估。Scaling Law 描述参数、数据与损失的经验权衡，但数据质量、系统利用率、稳定性和评测污染同样决定最终能力。

监督微调：从会续写到会按指令回答

本章学习目标

读完本章后，应能够：

1. 区分预训练、持续预训练、SFT 和偏好优化；
2. 写出聊天数据经过模板后的 Token 序列和 loss mask；
3. 解释只对 assistant Token 计算 loss 的原因；
4. 理解 Packing、样本权重、数据混合和灾难性遗忘；
5. 设计一个有训练、验证、回归和错误分析的 SFT 闭环。

10.1 SFT 改变的是行为分布

Base Model 主要学习“给定前缀，什么文本最可能继续出现”。聊天模型还需学习：

- 识别 system/user/assistant 角色；
- 回答而非继续伪造用户文本；
- 遵循格式、长度和风格要求；
- 适当拒答、调用工具或输出结构化参数；
- 在多轮上下文中维持任务状态。

SFT 使用人工或合成的高质量示范，让模型最大化目标回答的条件概率。

10.2 SFT 数据的数学形式

一个样本由上下文 x 和目标回答 $y = (y_1, \dots, y_T)$ 组成。目标为

$$\mathcal{J}_{SFT}(\theta) = - \sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t}). \quad (10.1)$$

这仍是交叉熵，没有改变基本训练算法；改变的是输入分布、标签范围和数据质量。

10.2.1 多轮对话

对话可表示为

$$(system, u_1, a_1, u_2, a_2, \dots, u_k, a_k). \quad (10.2)$$

Chat Template 把角色和内容序列化为模型约定的特殊 Token:

$$\langle system \rangle s \langle eos \rangle \langle user \rangle u_1 \langle eos \rangle \langle assistant \rangle a_1 \langle eos \rangle \dots \quad (10.3)$$

模板是模型协议的一部分。训练和推理模板不一致, 会导致角色边界、停止条件和工具调用明显退化。

10.3 Loss Mask: 哪些 Token 应贡献梯度

设序列 Token 为 $z_1, \dots, z_L$, mask $m_t \in \{0, 1\}$:

$$\mathcal{J} = -\frac{\sum_{t=1}^L m_t \log P_{\theta}(z_t | z_{<t})}{\sum_{t=1}^L m_t}. \quad (10.4)$$

常见做法只令 assistant 回答 Token 的 $m_t = 1$, system/user 和 padding 为 0。

例: 单轮对话的 Mask

序列为

$$[\langle user \rangle, \text{问题}, \langle eos \rangle, \langle assistant \rangle, \text{答案}, \langle eos \rangle].$$

可设置 label:

$$[-100, -100, -100, -100, \text{答案 Token}, \langle eos \rangle],$$

框架的交叉熵忽略 -100。是否训练 assistant 起始标记和 EOS, 取决于模板设计。

10.3.1 为什么不一定要训练 User Token

若对整段对话都计算 loss, 模型会学习复现用户问题和 system 指令, 这可能:

- 浪费容量在输入建模;
- 使回答边界更模糊;
- 让长用户输入在 loss 中占过大权重;
- 降低不同样本之间的目标可比性。

但持续预训练式指令数据或某些多角色建模也可能训练全部 Token。关键是明确目标, 而非机械套用单一规则。

10.4 按 Token 平均还是按样本平均

设第 n 个回答有 T_n 个目标 Token。按 Token 平均:

$$J_{token} = \frac{\sum_n \sum_t \ell_{nt}}{\sum_n T_n}. \quad (10.5)$$

长回答贡献更多权重。按样本平均:

$$J_{sample} = \frac{1}{N} \sum_n \left(\frac{1}{T_n} \sum_t \ell_{nt} \right). \quad (10.6)$$

每条样本权重相等。两者会改变长度偏好和梯度分布, 尤其在短问答与长推理混合时。

10.5 Packing 与样本隔离

SFT 样本长度差异大。若每条 padding 到固定长度，利用率低。Packing 把多条短样本拼入同一序列。

10.5.1 跨样本注意力问题

若直接拼接

$$[x^{(1)}, y^{(1)}, EOS, x^{(2)}, y^{(2)}],$$

第二条样本可读取第一条内容。解决方法：

- 使用 block-diagonal causal mask 隔离样本；
- 接受轻微跨样本上下文，并用 EOS 学习边界；
- 按长度分桶，减少 padding 而不拼接过多样本。

不同训练框架的 packing 语义不同，必须检查 position ID、attention mask、label mask 和 FlashAttention 接口。

10.6 数据质量：SFT 的核心瓶颈

高质量 SFT 样本应满足：

- 指令明确，回答真正完成任务；
- 事实、代码和计算可验证；
- 风格与目标产品一致；
- 难度、领域、语言和长度有覆盖；
- 不泄漏 system prompt 或隐私；
- 拒答边界不过宽也不过窄；
- 工具调用参数可执行，结果与最终回答一致。

大量低质量合成数据会把教师模型的错误、套话和评审偏差蒸馏进学生模型。

10.7 数据混合与能力保持

设通用指令、领域任务、安全数据、工具数据的采样权重分别为 w_k 。SFT 实际优化的是混合分布：

$$J = \sum_k w_k \mathbb{E}_{(x,y) \sim \mathcal{D}_k} [-\log \pi_\theta(y | x)]. \quad (10.7)$$

权重决定模型“每天练什么”。领域数据占比过高可能导致：

- 通用问答退化；
- 回答风格过度专业化；

- 拒答和安全边界变化；
- 输出长度、格式或语言偏移。

常用通用 replay 数据和独立回归集保持原有能力。

10.8 全参数微调与参数高效微调

全参数 SFT 更新全部 N 个参数，表达能力强，但训练状态显存、通信和 checkpoint 成本高。PEFT 只训练少量新增参数，成本低且易管理多个领域版本。选择取决于：

- 数据量和分布变化幅度；
- 基础模型与目标任务差距；
- 训练资源和部署形态；
- 是否需要长期持续迭代；
- 能否接受 adapter 加载和合并复杂度。

LoRA 的数学细节见后续章节。

10.9 灾难性遗忘

微调只在窄数据上持续更新，会让参数向新分布移动，导致旧任务性能下降。可用以下方法缓解：

- 混入通用 replay 数据；
- 降低学习率、训练 epoch 和可训练参数比例；
- 使用 LoRA/Adapter 隔离部分更新；
- 约束参数或输出分布不要偏离参考模型；
- 持续运行多任务回归评测。

“训练 loss 继续下降”不能说明整体模型继续变好。

10.10 拒绝采样与合成数据

对同一指令从模型采样多个候选 $y_1, \dots, y_K$ ，通过规则、执行器或 Judge 选择最佳：

$$y^* = \arg \max_{y_k} R(x, y_k). \quad (10.8)$$

把 (x, y^*) 加入 SFT 数据，称为拒绝采样式自训练。其效果取决于：

- 候选是否足够多样；
- 评分器是否与真实目标一致；
- 是否过滤不可验证的错误；
- 是否避免反复自蒸馏导致分布收缩。

10.11 SFT 与知识注入

SFT 可以让模型更常输出某些领域事实，但不一定是最可靠的知识更新方式：

- 少量事实可能被过拟合或在不同问法下无法调用；
- 事实随时间变化时，重新训练成本高；
- 新事实可能与旧权重冲突；
- 无法自然给出可追溯来源。

稳定行为和格式适合 SFT；大量动态知识通常更适合 RAG；大规模领域语言分布适合 CPT。

10.12 训练超参数的作用

10.12.1 学习率

过高会破坏基础能力或发散；过低可能只记住表面格式。SFT 数据量通常远小于预训练，学习率往往更保守。

10.12.2 Epoch

小数据重复过多会过拟合。应看验证集、行为回归和样本记忆，而非固定使用 3 epoch。

10.12.3 序列长度

截断会丢失回答尾部或多轮历史。长序列提高显存和计算，应分析真实长度分布并使用 packing。

10.12.4 Warmup 和 Weight Decay

短 SFT 中 warmup 步数过多会让大部分训练处于低学习率；weight decay 需结合 PEFT 和可训练参数类型设置。

10.13 SFT 的评测闭环

至少分四层：

1. **训练正确性**：mask、模板、数据解码、loss、梯度和 checkpoint；
2. **目标任务**：准确率、执行通过率、格式合法率、人工偏好；
3. **通用回归**：知识、推理、代码、多语种、长上下文；
4. **安全回归**：过拒答、漏拒答、提示注入和隐私。

错误分析应按类型统计，例如：事实错误、未遵循约束、格式错误、过长、拒答错误、工具参数错误、推理正确但最终答案错误。只有知道错误结构，才能决定继续加数据、改模板、做偏好优化还是引入 RAG/工具。

10.14 一个最小 SFT 实验设计

1. 固定基础模型、Tokenizer 和 Chat Template;
2. 建立 200-1000 条人工核验验证集;
3. 先训练小规模数据和少量 step, 验证 loss mask 与生成行为;
4. 保存 base、早期、中期、末期多个 checkpoint;
5. 对目标任务和通用回归同时评测;
6. 按错误类型补数据, 而不是只扩充总量;
7. 记录数据版本、随机种子、超参数和代码 commit。

10.15 本章综合练习

练习 1: Loss 权重

数据集中一条答案长 1000 Token, 另一条长 10 Token。按 Token 平均时二者的总 loss 权重比是多少? 按样本平均时是多少?

解答

按 Token 平均约为 $1000 : 10 = 100 : 1$; 按样本平均为 $1 : 1$ 。这会显著改变长回答样本对训练方向的影响。

练习 2: 方法选择

需求是“让模型始终按固定 JSON Schema 输出, 并掌握每周更新的产品价格”。分别应优先使用什么方法?

解答

固定输出行为可通过 SFT、结构化解码和 schema 校验; 动态价格应放入可更新的数据源并通过 RAG/工具查询, 而不是每周微调权重。

本节要点

SFT 本质仍是带 mask 的条件语言建模, 但它用高质量示范塑造角色、格式、工具和任务行为。效果取决于模板、目标 Token 范围、数据混合、长度权重和回归评测。SFT 适合教行为, 不是解决动态知识、事实验证和所有偏好的万能方法。

偏好对齐：从策略梯度到 PPO、DPO 与 GRPO

本章学习目标

读完本章后，应能够：

1. 区分 SFT 似然目标与偏好/奖励目标；
2. 推导 REINFORCE 的策略梯度和 baseline 无偏性；
3. 理解 Value、Advantage、GAE 和 PPO 裁剪；
4. 从 KL 正则化最优策略推导 DPO 损失；
5. 比较 PPO、DPO、RLOO、GRPO 与可验证奖励的适用条件。

11.1 为什么最大似然不能表达全部偏好

开放式任务常有多个正确回答。SFT 对一条参考回答 y^* 最大化

$$\log \pi_{\theta}(y^* | x), \quad (11.1)$$

但无法直接表达：回答 A 和 B 都正确，A 更简洁、更安全或更符合风格。偏好数据通常为

$$(x, y_w, y_l), \quad (11.2)$$

其中 y_w 是优选回答， y_l 是劣选回答。目标不是复现唯一标签，而是调整整个输出分布，使更好的回答更可能出现。

11.2 序列概率与 Token 概率

策略模型生成完整回答的概率为

$$\pi_{\theta}(y | x) = \prod_{t=1}^T \pi_{\theta}(y_t | x, y_{<t}). \quad (11.3)$$

取对数后：

$$\log \pi_{\theta}(y | x) = \sum_{t=1}^T \log \pi_{\theta}(y_t | x, y_{<t}). \quad (11.4)$$

因此一个序列级奖励最终要通过所有 Token 的 log probability 更新模型。回答越长，序列 log probability 通常越负，这也是偏好训练中需要关注长度偏差的原因。

11.3 奖励模型与 Bradley-Terry 偏好概率

奖励模型 $r_\phi(x, y)$ 输出标量。假设人类选择 y_w 而不是 y_l 的概率由分数差决定：

$$P(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)), \quad (11.5)$$

其中 $\sigma(z) = 1/(1 + e^{-z})$ 。最大化观测偏好的对数似然得到

$$\mathcal{L}_{RM}(\phi) = -\mathbb{E} \log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)). \quad (11.6)$$

11.3.1 为什么只看分数差

对任意只依赖提示的常数 $c(x)$ ，令

$$r'(x, y) = r(x, y) + c(x), \quad (11.7)$$

则优劣分数差不变。因此偏好数据只能确定同一提示下的相对奖励，不能唯一确定绝对零点。

11.3.2 奖励模型的常见偏差

奖励模型可能把长度、礼貌、引用格式、特定短语或冗长解释当作质量代理。训练准确率高也不保证对当前策略新生成的分布有效。RM 评测应包括：

- 独立偏好对准确率与分片结果；
- 长度控制后的准确率；
- 对抗性套话和格式攻击；
- 分布外任务；
- 与人工评分的一致性和校准。

11.4 把文本生成写成强化学习

对提示 x ：

- 状态 $s_t = (x, y_{<t})$ ；
- 动作 $a_t = y_t$ ，即下一个 Token；
- 策略 $\pi_\theta(a_t \mid s_t)$ ；
- 终止时得到回答 y ；
- 奖励 $R(x, y)$ 可来自 RM、规则、执行器或环境。

目标是最大化期望回报：

$$J(\theta) = \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x)} [R(x, y)]. \quad (11.8)$$

11.5 策略梯度的完整推导

直接对期望求导：

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \sum_y \pi_{\theta}(y | x) R(x, y) \quad (11.9)$$

$$= \sum_y \nabla_{\theta} \pi_{\theta}(y | x) R(x, y). \quad (11.10)$$

利用 **log-derivative trick**：

$$\nabla_{\theta} \pi_{\theta}(y) = \pi_{\theta}(y) \nabla_{\theta} \log \pi_{\theta}(y), \quad (11.11)$$

得到

$$\nabla_{\theta} J(\theta) = \sum_y \pi_{\theta}(y | x) R(x, y) \nabla_{\theta} \log \pi_{\theta}(y | x) \quad (11.12)$$

$$= \mathbb{E}_{y \sim \pi_{\theta}} [R(x, y) \nabla_{\theta} \log \pi_{\theta}(y | x)]. \quad (11.13)$$

再展开序列 log probability：

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[R(x, y) \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(y_t | x, y_{<t}) \right]. \quad (11.14)$$

这就是 REINFORCE。采样到高奖励回答时，提高其中动作概率；低奖励回答时降低。

直觉理解

策略梯度不需要对奖励函数求导。奖励可以来自编译器、单元测试、游戏环境或人工评分，只要能对采样结果给出数值。可微分部分是模型自身的 log probability。

11.6 Baseline 为什么降低方差且不引入偏差

将奖励替换为 $R - b(x)$ ：

$$\mathbb{E}[(R - b) \nabla \log \pi]. \quad (11.15)$$

额外项为

$$\mathbb{E}[b(x) \nabla_{\theta} \log \pi_{\theta}(y | x)] = b(x) \sum_y \pi_{\theta}(y | x) \nabla_{\theta} \log \pi_{\theta}(y | x) \quad (11.16)$$

$$= b(x) \sum_y \nabla_{\theta} \pi_{\theta}(y | x) \quad (11.17)$$

$$= b(x) \nabla_{\theta} \sum_y \pi_{\theta}(y | x) \quad (11.18)$$

$$= b(x) \nabla_{\theta} 1 = 0. \quad (11.19)$$

所以只要 **baseline** 不依赖当前采样动作，它不改变梯度期望，却能把回报中心化，降低方差。

11.7 Return、Value、Q 与 Advantage

在一般 MDP 中：

$$G_t = \sum_{l=0}^{T-t} \gamma^l r_{t+l}, \quad (11.20)$$

$$V^\pi(s_t) = \mathbb{E}[G_t \mid s_t], \quad (11.21)$$

$$Q^\pi(s_t, a_t) = \mathbb{E}[G_t \mid s_t, a_t], \quad (11.22)$$

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t). \quad (11.23)$$

Advantage 表示某动作相对该状态平均行为好多少。文本 RLHF 常在序列末尾获得主奖励，再把 KL 惩罚分配到 Token 级。

11.7.1 Temporal-Difference 误差与 GAE

价值模型 V_ψ 估计未来回报，单步 TD 误差：

$$\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t). \quad (11.24)$$

Generalized Advantage Estimation (GAE) 使用

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{T-t} (\gamma \lambda)^l \delta_{t+l}. \quad (11.25)$$

较小 λ 更依赖 Value、方差低但偏差可能高； $\lambda \rightarrow 1$ 更接近 Monte Carlo return、偏差低但方差高。

11.8 重要性采样与策略概率比

PPO 使用旧策略 π_{old} 采样，但更新新策略 π_θ 。概率比为

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{old}(a_t \mid s_t)} = \exp(\log \pi_\theta - \log \pi_{old}). \quad (11.26)$$

若 $r_t > 1$ ，新策略提高了该动作概率； $r_t < 1$ 则降低。未裁剪的 surrogate objective 为

$$L^{PG} = \mathbb{E}[r_t(\theta) \hat{A}_t]. \quad (11.27)$$

11.9 PPO 裁剪目标逐情况解释

PPO 使用 [12]

$$L^{clip} = \mathbb{E} \left[\min \left(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]. \quad (11.28)$$

训练时最大化该目标，或最小化其负值。

11.9.1 当 $\hat{A}_t > 0$

该动作优于平均，应提高概率。但若 $r_t > 1 + \epsilon$ ，收益被截断，不再鼓励继续大幅提高。

11.9.2 当 $\hat{A}_t < 0$

该动作较差，应降低概率。但若 $r_t < 1 - \epsilon$ ，目标被截断，不再鼓励继续大幅降低。

Advantage	概率比变化	裁剪作用
$A > 0$	r 增大	鼓励提高好动作概率，但超过 $1 + \epsilon$ 后停止额外奖励
$A > 0$	r 减小	目标下降，不裁剪这种不利变化
$A < 0$	r 减小	鼓励降低坏动作概率，但低于 $1 - \epsilon$ 后停止额外奖励
$A < 0$	r 增大	目标下降，不裁剪这种不利变化

裁剪不是硬性保证 KL 一定小，而是形成悲观 surrogate。实践中仍会监控 KL、clip fraction 和 ratio 分布。

11.10 完整 PPO 损失的组件

常见目标还包括 Value loss 和 entropy bonus：

$$L = L_{policy} - c_v L_{value} + c_e H(\pi_\theta). \quad (11.29)$$

若写成最小化损失，符号相反。Value loss 可为

$$L_{value} = \mathbb{E}[(V_\psi(s_t) - \hat{G}_t)^2]. \quad (11.30)$$

熵奖励

$$H(\pi) = - \sum_a \pi(a | s) \log \pi(a | s) \quad (11.31)$$

防止策略过早坍缩，但语言模型词表巨大，熵项和采样温度需谨慎调节。

11.11 KL 正则化与参考模型

RLHF 常优化

$$\max_{\pi} \mathbb{E}_{y \sim \pi}[r(x, y)] - \beta D_{\text{KL}}(\pi(\cdot | x) \| \pi_{ref}(\cdot | x)). \quad (11.32)$$

参考模型通常是 SFT checkpoint。KL 惩罚提供三种作用：

- 保持语言流畅和基础能力；
- 限制奖励模型分布外利用；
- 把偏好更新解释为相对参考模型的受约束移动。

序列级 sampled KL 常估计为

$$\log \pi_\theta(y | x) - \log \pi_{ref}(y | x) = \sum_t [\log \pi_\theta(y_t | s_t) - \log \pi_{ref}(y_t | s_t)]. \quad (11.33)$$

它是 Monte Carlo 估计，单样本可为负，但期望 KL 非负。

11.12 KL 正则化最优策略的推导

对固定提示 x ，优化所有回答分布 $\pi(y)$:

$$\max_{\pi} \sum_y \pi(y) r(y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{ref}(y)}, \quad (11.34)$$

约束 $\sum_y \pi(y) = 1$ 。引入拉格朗日乘子 λ :

$$\mathcal{F} = \sum_y \pi(y) r(y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{ref}(y)} \quad (11.35)$$

$$+ \lambda \left(\sum_y \pi(y) - 1 \right). \quad (11.36)$$

对 $\pi(y)$ 求导并令零:

$$r(y) - \beta \left(\log \frac{\pi(y)}{\pi_{ref}(y)} + 1 \right) + \lambda = 0. \quad (11.37)$$

整理:

$$\log \frac{\pi^*(y)}{\pi_{ref}(y)} = \frac{r(y)}{\beta} + c(x), \quad (11.38)$$

其中 $c(x)$ 与回答无关。于是

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{ref}(y | x) e^{r(x,y)/\beta}. \quad (11.39)$$

反过来，奖励可写成

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{ref}(y | x)} + \beta \log Z(x). \quad (11.40)$$

这一步是 DPO 的核心桥梁。

11.13 DPO 的逐步推导

偏好模型假设

$$P(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l)). \quad (11.41)$$

将上节隐式奖励代入。由于 $\beta \log Z(x)$ 在同一提示的差中抵消:

$$r(x, y_w) - r(x, y_l) = \beta \left[\log \pi_{\theta}(y_w | x) - \log \pi_{ref}(y_w | x) \right] \quad (11.42)$$

$$- \log \pi_{\theta}(y_l | x) + \log \pi_{ref}(y_l | x) \Big]. \quad (11.43)$$

对偏好标签做最大似然，得到 DPO 损失 [13]:

$$\mathcal{L}_{DPO} = -\mathbb{E} \log \sigma \left(\beta [\log \pi_{\theta}(y_w | x) - \log \pi_{ref}(y_w | x)] - \beta [\log \pi_{\theta}(y_l | x) - \log \pi_{ref}(y_l | x)] \right) \quad (11.44)$$

11.13.1 DPO 的直觉

定义相对参考模型的隐式得分

$$s_{\theta}(x, y) = \beta [\log \pi_{\theta}(y | x) - \log \pi_{ref}(y | x)]. \quad (11.45)$$

DPO 就是在做二分类：让 $s(y_w) > s(y_l)$ 。参考模型消除了“高概率通用文本天然占优”的一部分影响，并限制策略漂移。

11.13.2 β 的含义

在上述约定中， β 控制相对 log-ratio 的尺度。不同实现对 β 的解释和符号约定可能不同。过强约束可能学习不足，过弱约束可能快速偏离参考模型。需要结合隐式 reward margin、KL 和下游评测调节。

11.14 DPO 与 SFT 梯度的差异

SFT 只提高优选回答概率：

$$-\log \pi_{\theta}(y_w | x). \quad (11.46)$$

DPO 同时比较优劣回答相对参考的变化。即便 y_w 的绝对概率下降，只要 y_l 下降得更多，偏好 margin 仍可能增加。因此训练时要同时监控 chosen/rejected log probability，避免整体概率异常下降或长度效应。

11.15 RLOO：留一法基线

对同一提示采样 K 个回答，奖励为 $R_1, \dots, R_K$ 。第 i 个样本的 leave-one-out baseline：

$$b_i = \frac{1}{K-1} \sum_{j \neq i} R_j. \quad (11.47)$$

优势为

$$A_i = R_i - b_i. \quad (11.48)$$

因为 b_i 不包含当前样本 R_i ，在条件独立采样下可作为与当前动作无关的 baseline，保持策略梯度无偏，同时利用同题其他回答降低方差。

若使用包含自身的组均值

$$\bar{R} = \frac{1}{K} \sum_j R_j, \quad (11.49)$$

则 $R_i - \bar{R}$ 与自身奖励耦合。实际算法是否仍满足特定无偏性质，要看完整估计器和归一化，不应仅凭“减均值”判断。

11.16 GRPO：组内相对优势

GRPO 类方法对每个提示采样一组回答，用组内奖励构造优势，常见形式：

$$\hat{A}_i = \frac{R_i - \bar{R}}{\text{std}(R_1, \dots, R_K) + \epsilon}. \quad (11.50)$$

再使用 PPO 风格概率比、裁剪和 KL 正则更新策略。它通常不训练独立 Value Model，降低显存和系统复杂度。

11.16.1 组内标准化的作用

- 消除不同题目奖励尺度差异；
- 强调同题回答的相对好坏；
- 当奖励全相同时优势接近 0，没有学习信号；
- 组大小小或奖励离散时，方差和归一化噪声可能较大。

11.16.2 Token 级优势分配

若一个回答只有序列级奖励，常把同一 $\hat{A}_i$ 分配给该回答所有有效 Token。这样简单，但信用分配粗糙：正确推理前半段和错误结尾会得到同一信号。过程奖励或 step-level verifier 可提供更细粒度反馈，但成本和被攻击面更高。

11.17 PPO、DPO、RLOO 与 GRPO 对比

方法	数据来源	主要额外组件	适用与风险
PPO-RLHF	当前策略在线采样	RM、Value、Reference、Old policy	可探索新输出和组合奖励；系统复杂、价值估计和策略漂移风险高
DPO	固定偏好对	Reference model	实现接近监督训练；受离线覆盖、长度偏差和参考模型影响
RLOO	问题多样本在线采样	奖励器、Reference，可无 Value	留一基线降低方差；采样成本和组内多样性重要
GRPO 类	问题组采样	奖励器、Reference，通常无 Value	适合可验证奖励和推理训练；组归一化、零方差组、长序列 ratio 等需谨慎

11.18 可验证奖励 (RLVR)

数学、代码、形式证明和工具任务可由执行器给出奖励：

$$R(x, y) = \mathbf{1}[\text{Verify}(x, y) = \text{pass}]. \quad (11.51)$$

也可使用部分分数、测试通过数或格式奖励。优点是客观、可扩展；风险包括：

- 稀疏奖励导致大量样本无信号；
- 测试集不完整，模型钻空子；

- 只优化最终答案，推理过程不可读或脆弱；
- 奖励与真实部署目标不一致；
- 生成长度和采样预算显著增加成本。

11.19 Reward Hacking 与 Goodhart 定律

当代理指标成为优化目标，它与真实目标的相关性可能下降。典型 reward hacking:

- 输出冗长模板博取 Judge 高分；
- 伪造引用、测试输出或工具结果；
- 利用解析器、浮点容差或未覆盖边界；
- 对奖励模型触发特定词语；
- 学会拒答以避免错误，却失去帮助性。

缓解方法包括独立验证器、隐藏测试、对抗样本、多奖励约束、人工抽检和上线监控。

11.20 训练监控指标

偏好/RL 训练不能只看 reward。建议监控：

- 原始 reward、分片 reward 和通过率；
- 与 reference 的 KL；
- response length 与 reward 的相关性；
- policy ratio、clip fraction、entropy；
- chosen/rejected log probability；
- advantage 均值、方差、零方差组比例；
- 正常能力、安全和格式回归；
- 重复、模式坍塌和异常短/长回答。

11.21 一个玩具 DPO 例子

对同一提示，设策略和参考模型的序列 log probability:

$$\begin{aligned}\log \pi_{\theta}(y_w) &= -2.0, & \log \pi_{ref}(y_w) &= -2.5, \\ \log \pi_{\theta}(y_l) &= -3.0, & \log \pi_{ref}(y_l) &= -2.8.\end{aligned}$$

相对改变量：

$$\begin{aligned}\Delta_w &= -2.0 - (-2.5) = 0.5, \\ \Delta_l &= -3.0 - (-2.8) = -0.2.\end{aligned}$$

偏好 margin 为 $\beta(0.5 - (-0.2)) = 0.7\beta$ ，偏好概率为 $\sigma(0.7\beta)$ 。策略相对参考提高了优选回答、降低了劣选回答，因此损失较小。

11.22 本章综合练习

练习 1: Baseline 无偏性

为什么 baseline 可以依赖提示 x 或状态 s_t ，却不能任意依赖当前动作 a_t ？

解答

只依赖状态时， $b(s) \sum_a \nabla \pi(a | s) = b(s) \nabla 1 = 0$ 。若 baseline 依赖动作，通常无法从求和中提出，额外项不再保证为零，会改变梯度期望，除非做专门校正。

练习 2: PPO 裁剪

设 $A = 2, \epsilon = 0.2$ 。当 $r = 1.5$ 时， rA 与裁剪项各是多少，最终使用哪个？若 $A = -2, r = 0.5$ 呢？

解答

第一种： $rA = 3$ ，裁剪比为 1.2，裁剪项 2.4，取最小为 2.4。第二种： $rA = -1$ ，裁剪比为 0.8，裁剪项 -1.6，取最小为 -1.6，阻止继续过度降低坏动作概率。

练习 3: RLOO 优势

同一提示采样奖励 (1, 0, 0.5)，计算三个 RLOO baseline 与 advantage。

解答

$$b_1 = (0 + 0.5)/2 = 0.25, \quad A_1 = 0.75;$$

$$b_2 = (1 + 0.5)/2 = 0.75, \quad A_2 = -0.75;$$

$$b_3 = (1 + 0)/2 = 0.5, \quad A_3 = 0.$$

本节要点

策略梯度通过 log-derivative trick 把不可微奖励转成可优化的 log probability 梯度；baseline 降低方差，Value/GAE 改善信用分配，PPO 用概率比裁剪限制策略更新。DPO 从 KL 正则化最优策略与偏好概率推导出离线损失。RLOO/GRPO 用同题多样本构造相对优势，减少 Value Model 依赖，但仍受奖励质量、采样覆盖和长度偏差约束。

参数高效微调、QLoRA 与模型合并

本章学习目标

读完本章后，应能够：

1. 解释全参数微调的显存成本来自哪里；
2. 从低秩分解理解 LoRA 的参数化、缩放和初始化；
3. 计算 LoRA 参数量与训练状态节省；
4. 理解 QLoRA 中“量化冻结权重、训练高精度 LoRA”的计算链；
5. 比较 Adapter、Prefix Tuning、模型合并、RAG 与持续预训练。

12.1 为什么需要参数高效微调

设基础模型有 N 个参数。全参数 AdamW 训练通常需要保存：

- 模型参数；
- 梯度；
- 一阶矩 m ；
- 二阶矩 v ；
- 某些设置下的 FP32 master weights；
- 激活和临时 buffer。

以 BF16 参数/梯度、FP32 master/m/v 粗估，仅模型状态约为

$$2N + 2N + 4N + 4N + 4N = 16N \text{ bytes.} \quad (12.1)$$

7B 模型约 112 GB，尚未计激活。PEFT 冻结绝大多数基础权重，只为少量新增参数保存梯度和优化器状态。

12.2 LoRA 的低秩假设

全参数微调把权重 $W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$ 更新为

$$W = W_0 + \Delta W. \quad (12.2)$$

LoRA 假设任务适配所需的更新 ΔW 具有较低的**内在秩**，用两个小矩阵表示 [15]：

$$\Delta W = BA, \quad (12.3)$$

其中

$$A \in \mathbb{R}^{r \times d_{in}}, \quad B \in \mathbb{R}^{d_{out} \times r}, \quad r \ll \min(d_{in}, d_{out}). \quad (12.4)$$

前向为

$$y = W_0 x + \frac{\alpha}{r} B A x. \quad (12.5)$$

W_0 冻结，只训练 A, B 。

12.3 为什么低秩可能有效

任意矩阵可做奇异值分解：

$$\Delta W = U \Sigma V^\top. \quad (12.6)$$

若只有少数奇异值较大，可截断为 **rank- r** 近似：

$$\Delta W \approx U_r \Sigma_r V_r^\top. \quad (12.7)$$

令

$$B = U_r \Sigma_r^{1/2}, \quad A = \Sigma_r^{1/2} V_r^\top, \quad (12.8)$$

即可写成 BA 。LoRA 并不在训练前知道真实 ΔW 的 SVD，而是直接学习低秩因子。其经验动机是：大模型已有丰富能力，微调常只需在少数方向上重组和偏移表示。

常见误区

低秩假设不是数学保证。任务分布变化很大、数据量巨大或需要重塑基础能力时，小 **rank** 可能成为瓶颈。LoRA 效果应由全参数基线或 **rank** 扫描验证，而不是默认“永远无损”。

12.4 LoRA 参数量

原线性层参数量为

$$P_{full} = d_{out} d_{in}. \quad (12.9)$$

LoRA 可训练参数量为

$$P_{LoRA} = r(d_{in} + d_{out}). \quad (12.10)$$

比例为

$$\rho = \frac{r(d_{in} + d_{out})}{d_{in} d_{out}}. \quad (12.11)$$

若方阵 $d_{in} = d_{out} = d$ ：

$$\rho = \frac{2r}{d}. \quad (12.12)$$

例：4096 维线性层

原权重 4096×4096 有 16.78M 参数。rank $r = 16$ 的 LoRA 有

$$16(4096 + 4096) = 131,072$$

参数，仅约 0.78%。若只给部分投影加 LoRA，整个模型可训练比例还会更低。

12.5 缩放 α/r

常写为

$$\Delta W = \frac{\alpha}{r} BA. \quad (12.13)$$

定义有效缩放 $s = \alpha/r$ 。这样改变 **rank** 时，可以通过调整 α 控制更新幅度。不同库可能使用 α/r 、 $\alpha/\sqrt{r}$ 或其他变体，比较实验时要核对实现。

12.6 初始化为什么通常令初始更新为零

常见初始化： A 随机小值， $B = 0$ 。于是训练开始时

$$BA = 0, \quad (12.14)$$

模型输出与基础模型完全一致。第一步 B 可获得非零梯度；随后 $B \neq 0$ 后， A 也开始更新。

若 $A = B = 0$ ，则

$$\frac{\partial(BAx)}{\partial A} \propto B^\top = 0, \quad \frac{\partial(BAx)}{\partial B} \propto Ax = 0, \quad (12.15)$$

两者都可能无法启动，因此不能同时全零初始化。

12.7 LoRA 前向与合并

训练/未合并推理：

$$y = W_0 x + sB(Ax). \quad (12.16)$$

先计算 $Ax \in \mathbb{R}^r$ ，再乘 B ，额外计算约为

$$2r(d_{in} + d_{out}) \quad (12.17)$$

FLOPs/Token，**rank** 小时相对原层较低。

部署前可合并：

$$W_{merged} = W_0 + sBA, \quad (12.18)$$

之后推理不再需要额外分支。但合并后的权重需重新保存，且动态切换多个 **adapter** 不再方便。

12.8 Target Modules 如何选择

Transformer 中常见目标：

- Attention 的 W_Q, W_K, W_V, W_O ；
- MLP 的 gate/up/down 投影；
- Embedding 或 LM Head（特定词表/领域）；
- 多模态 Projector。

只训练 Q/V 参数最少，但可能不足；覆盖所有线性层表达更强，显存和 **checkpoint** 增加。选择应通过消融：固定总数据、**step** 和学习率，比较目标能力与通用回归。

12.8.1 不同模块的直觉

- Q/K 改变 “如何匹配和寻址”；
- V/O 改变 “读取什么以及如何写回残差”；
- MLP 改变每个 Token 的非线性特征变换；
- Embedding/LM Head 更直接影响词表表示和输出偏好。

这些只是结构直觉，不代表某类任务只需固定模块。

12.9 Rank、Alpha、Dropout 与学习率

12.9.1 Rank

rank 越大，更新空间容量越高，但参数、通信和过拟合风险增加。常见做法从小 rank 开始，观察验证集是否饱和。

12.9.2 Alpha

决定更新缩放。只报告 rank 不报告 alpha，实验不可复现。

12.9.3 LoRA Dropout

在 LoRA 分支输入上使用 dropout，可正则化小数据训练；大数据或精确任务中也可能损伤。

12.9.4 学习率

LoRA 参数随机初始化且数量少，常使用比全参数微调更高的学习率，但不存在通用固定倍数。需结合 rank、数据量、目标模块和 batch 调节。

12.10 LoRA 节省了什么，没有节省什么

LoRA 显著减少：

- 可训练参数的梯度；
- Adam 一阶、二阶状态；
- checkpoint 大小；
- 多任务版本存储。

但基础模型前向和反向仍需通过所有层。为了计算 LoRA 梯度，仍要传播激活梯度，因此：

- 计算量不会按可训练参数比例下降；
- 激活显存仍可能是主要瓶颈；
- 冻结权重仍需驻留 GPU/CPU 或按需加载；

- 长序列成本不因 LoRA 自动消失。

12.11 QLoRA 的计算链

QLoRA 将冻结基础权重量化存储，同时训练 LoRA[16]。抽象前向：

$$y = \text{MatMul}(\text{Dequant}(W_q), x) + sBAx. \quad (12.19)$$

其中 W_q 是低 bit 权重。量化权重在算子内部按块反量化到计算 dtype（如 BF16）参与矩阵乘法，基础权重不更新。

12.11.1 QLoRA 不是“用 INT4 训练全部权重”

基础权重是冻结的量化表示；可训练 LoRA、梯度和优化器状态通常为 BF16/FP32。若直接对离散 INT4 权重做普通梯度更新，会面临不可微量化和频繁重编码问题。

12.12 NF4、分组量化与双重量化

QLoRA 常使用 NF4。其思想是为近似正态分布的权重设计非均匀量化码本，使有限 4-bit 编码更匹配权重分布。权重按 group 量化，每组有 scale：

$$W \approx s_g Q_g. \quad (12.20)$$

双重量化进一步量化这些 scale，减少元数据开销。实际内存不等于严格 4 bit/参数，还包括 scale、zero point、对齐和运行时 buffer。

12.13 量化误差如何影响微调

基础权重由 W_0 变成 $\tilde{W}_0 = W_0 + E_q$ ，模型实际优化：

$$y = (W_0 + E_q)x + sBAx. \quad (12.21)$$

LoRA 可能部分补偿量化误差，但 rank 和数据有限。对异常通道、Embedding/LM Head、数学和代码任务，应比较 BF16 LoRA 与 QLoRA 的回归差异。

12.14 Adapter、Prefix Tuning 与 Prompt Tuning

12.14.1 Adapter

在层内插入瓶颈模块：

$$\text{Adapter}(x) = x + W_{up}\phi(W_{down}x), \quad (12.22)$$

其中 $W_{down} \in \mathbb{R}^{r \times d}$ 、 $W_{up} \in \mathbb{R}^{d \times r}$ 。它与 LoRA 参数量相似，但改变网络路径，推理时通常增加串行层延迟。

12.14.2 Prefix Tuning

为每层 Attention 学习额外的虚拟 K/V 前缀，使真实 Token 可注意这些连续提示。它不修改主权重，但每层增加 Cache 长度。

12.14.3 Prompt Tuning

只在输入 embedding 前学习少量连续虚拟 Token。参数最少，部署简单，但对较小模型或复杂任务可能不如深层适配。

方法	修改位置	推理影响	特点
LoRA	线性权重的低秩更新	可合并为零额外分支	通用、生态成熟
Adapter	层内新增瓶颈网络	增加串行计算	模块隔离清晰
Prefix Tuning	每层虚拟 K/V	增加 Cache 和注意力长度	以条件前缀引导各层
Prompt Tuning	输入虚拟 embedding	增加少量输入 Token	参数极少，能力依模型规模

12.15 多个 LoRA 的组合

若有多个 adapter:

$$W = W_0 + \sum_{k=1}^K \lambda_k \Delta W_k. \quad (12.23)$$

简单线性加权可能产生干扰，因为每个 adapter 在不同数据和基座版本上训练。组合前应检查：

- 基础模型 hash 和结构一致；
- target modules、rank 和 scaling 兼容；
- 单 adapter 和组合后的回归；
- 权重范数和方向冲突；
- 是否需要路由器按输入选择 adapter。

12.16 模型合并

全模型合并可写为参数插值：

$$\theta_{merge} = \sum_k \lambda_k \theta_k, \quad \sum_k \lambda_k = 1. \quad (12.24)$$

若 checkpoints 从同一基座微调且位于兼容参数盆地，插值可能融合能力；若独立训练或参数对齐不同，直接平均可能破坏模型。

更稳妥的视角是合并 task vector:

$$\Delta_k = \theta_k - \theta_0, \quad \theta_{merge} = \theta_0 + \sum_k \lambda_k \Delta_k. \quad (12.25)$$

还可做符号冲突处理、稀疏化或层级权重，但所有方法都需要独立评测。

12.17 微调、CPT、RAG 与 Prompt 的选择

需求	优先方法	原因
改变回答格式、风格、工具协议	SFT/LoRA + 结构校验	稳定行为可通过示范学习
掌握大量领域语言与模式	CPT, 再 SFT	需要改变基础语言分布
频繁变化、需引用的知识	RAG/工具	易更新、可追溯，不必重训
少量临时规则	Prompt/System instruction	In- 成本最低、修改最快
提升偏好、简洁性和拒答边界	DPO/RLHF	需要相对偏好而非唯一标签
完全新词表或强分布变化	全参数/CPT/架构适配	小 adapter 可能容量不足

12.18 LoRA 实验最小闭环

1. 固定 base model、Tokenizer、template 和评测集；
2. 先跑 rank 8/16、Attention-only 的低成本基线；
3. 再比较全线性层或 MLP 覆盖；
4. 记录 trainable params、显存、吞吐和 checkpoint 大小；
5. 比较 BF16 LoRA 与 QLoRA，分片看退化；
6. 测试未合并、合并后和量化部署后的输出一致性；
7. 做目标任务、通用能力和安全回归。

12.19 本章综合练习

练习 1: LoRA 参数量

对 $W \in \mathbb{R}^{11008 \times 4096}$ 使用 rank 32 LoRA，可训练参数多少？占原权重比例多少？

解答

$$P = 32(11008 + 4096) = 483,328.$$

原权重 $11008 \times 4096 = 45,088,768$ ，比例约 1.07%。

练习 2: LoRA 初始化

为什么常用 “ A 随机、 $B = 0$ ”，而不是二者都随机？

解答

$B = 0$ 保证初始 $BA = 0$ ，模型从基础模型精确开始； A 随机保证 B 在第一步能得到有方向差异的梯度。二者都随机会使初始输出立刻偏离基座，尺度需额外控制。

本节要点

LoRA 把权重更新限制为低秩矩阵 BA ，大幅减少梯度、优化器状态和 adapter 存储，但不按同等比例减少主干计算和激活。QLoRA 进一步量化冻结基础权重，以低显存训练高精度 LoRA。方法选择应从“要改变知识、行为还是外部信息”出发，并用全参数或 BF16 基线验证低秩和量化假设。

蒸馏、剪枝与训练数据合成

13.1 知识蒸馏

Knowledge Distillation 让学生模型学习教师模型的输出。监督信号可包括：

- 最终答案或推理轨迹；
- soft logits / 概率分布；
- 中间隐藏状态或注意力；
- 工具调用轨迹和环境反馈；
- 多候选中的优质样本。

Soft target 含有类别间相对概率，比 one-hot 标签提供更多“暗知识”。温度 T 可软化分布：

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}. \quad (13.1)$$

13.2 序列级蒸馏与合成数据

对生成模型，最常见做法是教师生成回答，学生做 SFT。可通过多样采样、拒绝采样、执行验证、去重和难度分桶提高质量。合成数据应记录生成模型、提示模板、采样参数和筛选规则，否则难以复现。

13.3 数据飞轮

线上系统可形成“日志 - 发现失败 - 构造测试 - 生成候选 - 人工/规则验证 - 训练 - 回归”的数据飞轮。关键不是持续收集所有日志，而是把失败聚类成可行动的能力缺口，并避免把模型自己的错误重新训练进去。

13.4 剪枝与稀疏化

剪枝删除不重要的权重、通道、head 或层。**非结构化剪枝**产生稀疏矩阵，若硬件不支持可能不加速；**结构化剪枝**删除完整通道或块，更易获得实际速度，但质量损失通常更大。

13.5 彩票假设与低秩/稀疏假设

LoRA 假设任务更新近似低秩；剪枝假设模型中存在冗余连接。两者都利用过参数化，但目标不同：LoRA 关注高效适配，剪枝关注压缩和加速。

常见误区

教师模型生成的数据不是自动正确。蒸馏会同时复制教师的事实错误、偏见、格式习惯和安全漏洞。必须有独立验证信号，而不是只用另一个模型打分。

本节要点

蒸馏把强模型能力转移给小模型，剪枝减少冗余结构，合成数据扩大覆盖。三者的共同难点是质量验证：没有独立评测，压缩或扩充都可能只是复制错误。

第四部分

第四部分：让模型完成任务

提示工程与解码：从概率分布到最终文本

本章学习目标

读完本章后，应能够：

1. 区分 Prompt 改变条件、微调改变参数；
2. 写出自回归生成循环；
3. 理解贪心、Beam Search、Temperature、Top-k、Top-p 的数学作用；
4. 解释长度偏差、重复、停止条件和随机性来源；
5. 选择适合事实问答、代码、创作和结构化输出的解码策略。

14.1 Prompt 的本质：给条件概率增加条件

模型计算

$$P_{\theta}(y \mid x), \quad (14.1)$$

其中 x 包括 system instruction、用户问题、示例、检索片段、工具结果和对话历史。Prompt Engineering 不修改 θ ，而是改变条件 x ，使目标输出在条件分布中更可能。

一个清晰 Prompt 通常包含：

1. **任务**：需要做什么；
2. **上下文**：完成任务所需事实和材料；
3. **约束**：格式、长度、语言、不可做事项；
4. **示例**：输入输出模式；
5. **验证要求**：引用、计算、工具或检查步骤。

约束应可操作，例如“输出包含字段 name、risk、evidence 的 JSON”，而非抽象的“回答得专业”。

14.2 Chat Template 与指令层级

模型实际接收的是模板序列化后的 Token，而非 UI 中显示的气泡。典型层级：system > developer/application > user > tool/external content。模型本身并不能从自然语言可靠推断可信来源，系统必须通过模板、权限和数据通道区分指令与不可信内容。

训练时使用的角色 Token、换行、EOS 和 assistant prefix 必须与推理一致。模板错位可能造成：

- 模型复述用户或继续伪造下一轮；
- 停止 Token 不生效；
- 工具 JSON 前后多余文本；
- system instruction 权重下降；
- 多轮历史角色混乱。

14.3 零样本、少样本与 In-Context Learning

Zero-shot 只给任务描述；**Few-shot** 在上下文中加入少量示例。模型并未更新权重，却能根据示例推断任务映射，这称为 In-Context Learning。

示例的作用包括：

- 消除标签含义和输出格式歧义；
- 展示边界情况和风格；
- 让模型识别当前任务分布；
- 提供局部算法或推理模式。

示例也可能引入顺序偏差、标签频率偏差和错误模仿。应覆盖代表性情况，而非堆积大量近重复示例。

14.4 自回归生成算法

输入 Token 为 $x_{1:L}$ ，最大生成长度 T 。每一步：

1. 模型输出最后位置 **logit** $z^{(t)} \in \mathbb{R}^V$ ；
2. 应用约束、惩罚和温度得到修改后 **logit**；
3. 转为概率并选择 Token y_t ；
4. 若命中 EOS/stop，结束；否则把 y_t 加入上下文并继续。

数学上：

$$y_t \sim \pi_\theta(\cdot \mid x, y_{<t}). \quad (14.2)$$

生成结果是多个局部选择的乘积，早期 Token 会改变后续全部条件分布。

14.5 贪心解码

每步选择最高概率 Token：

$$y_t = \arg \max_i P(i \mid x, y_{<t}). \quad (14.3)$$

优点是快速、确定；缺点是局部最优不保证整个序列概率或任务质量最优。一次早期选择可能把后续带入较差路径。

例：局部最优不等于全局最优

第一步 A 的概率 0.6，B 的概率 0.4。若选 A，第二步最佳 Token 概率 0.4，序列概率 0.24；若选 B，第二步最佳概率 0.9，序列概率 0.36。贪心选择 A，却错过更高联合概率序列 B-后续。

14.6 Beam Search

Beam Search 保留 K 条累计分数最高的部分序列。序列 log probability:

$$S(y_{1:T}) = \sum_{t=1}^T \log P(y_t | x, y_{<t}). \quad (14.4)$$

每步扩展每个 beam，再选前 K 。它近似搜索高概率序列，常用于翻译等目标相对单一的任务。

14.6.1 长度偏差

因为每个 log probability ≤ 0 ，序列越长总分通常越低，Beam Search 偏向短输出。常用长度归一化：

$$S_{norm}(y) = \frac{S(y)}{T^\alpha} \quad (14.5)$$

或其他 length penalty。聊天任务中，高概率文本也不一定最有帮助，Beam Search 可能产生保守、重复和模式化回答。

14.7 随机采样与温度

带温度概率：

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}. \quad (14.6)$$

然后按分类分布采样。温度低时更确定，高时更多样。温度并不改变候选集合，只改变相对概率；若仍有大量极低概率 Token，可能采到不连贯内容，因此常与 Top-k/Top-p 组合。

14.8 Top-k Sampling

只保留 logit 最高的 k 个 Token，其余设为 $-\infty$ ，再归一化采样。设保留集合 S_k ：

$$p'_i = \begin{cases} p_i / \sum_{j \in S_k} p_j, & i \in S_k, \\ 0, & \text{otherwise.} \end{cases} \quad (14.7)$$

固定 k 的问题是：分布尖锐时 k 太大，平坦时 k 太小。

14.9 Top-p / Nucleus Sampling

按概率降序，选择累计概率至少达到 p 的最小集合 S_p ：

$$\sum_{i \in S_p} p_i \geq p. \quad (14.8)$$

再在集合内重归一化采样。它根据当前分布动态调整候选数。

例：Top-p 候选集合

概率降序为 (0.45, 0.25, 0.15, 0.08, 0.07)。Top-p=0.8 时，前两项累计 0.70，不足；前三项累计 0.85，因此保留前三个并重归一化为约 (0.529, 0.294, 0.176)。

14.10 Min-p、Typical Sampling 与其他截断

Min-p 可按当前最大概率的比例过滤：保留满足

$$p_i \geq p_{\min} \max_j p_j \quad (14.9)$$

的 Token。**Typical Sampling** 偏好信息量接近当前熵的 Token。它们都在尝试避免从概率尾部采样，同时适应分布形状。具体支持和定义依推理框架而异。

14.11 采样顺序与实现差异

实际 pipeline 可能按不同顺序执行：重复惩罚、温度、Top-k、Top-p、min-p、grammar mask。顺序改变结果。例如先 Top-k 再温度与先温度再 Top-k 的集合通常相同（正温度保持 logit 排序），但与概率阈值类 Top-p 的组合会改变累计概率。复现实验必须记录完整参数和框架版本。

14.12 重复惩罚

14.12.1 Repetition Penalty

常见实现对已出现 Token 的 logit 做符号相关缩放：正 logit 除以 penalty，负 logit 乘 penalty。它是启发式，不是概率理论中的标准项。

14.12.2 Frequency/Presence Penalty

可修改 logit：

$$z'_i = z_i - \lambda_p \mathbf{1}[c_i > 0] - \lambda_f c_i, \quad (14.10)$$

c_i 是已出现次数。Presence penalty 惩罚“出现过”，frequency penalty 随次数增加。过强会破坏代码变量、术语和必要重复。

14.13 停止条件与 EOS

常见结束方式：

- 生成 EOS Token；
- 命中一个或多个 stop token/string；
- 达到 max new tokens；

- 结构化语法进入接受状态；
- Agent 环境返回终止动作。

字符串 **stop** 可能跨 **Token** 边界，需要流式缓冲。过早截断会产生不完整 JSON 或代码，过晚停止会生成下一轮角色内容。

14.14 结构化与约束解码

要求 JSON、SQL 或函数参数时，只靠 **Prompt** 不能保证语法合法。约束解码维护一个语法/状态机，在每步只允许能继续形成合法结构的 **Token**：

$$z'_i = \begin{cases} z_i, & i \in \mathcal{A}(s), \\ -\infty, & i \notin \mathcal{A}(s), \end{cases} \quad (14.11)$$

其中 $\mathcal{A}(s)$ 是当前解析状态允许的 **Token** 集。

约束解码保证**语法合法**，不保证字段语义正确。例如 JSON 中的日期、权限范围和工具参数仍需业务校验。

14.15 为什么相同参数仍可能不完全确定

即使 **temperature=0** 或贪心，结果仍可能因以下因素变化：

- GPU 并行归约和浮点非结合性；
- 量化、不同内核和编译优化；
- 动态 **batching** 改变计算路径；
- 相等或极接近 **logit** 的 **tie-breaking**；
- 服务端模型或模板版本变化；
- 外部检索和工具结果变化。

“确定性”需要固定模型、权重、模板、解码实现、随机种子、硬件和外部输入。

14.16 按任务选择解码策略

任务	常见策略	说明
抽取/分类/结构化	低温或贪心 + grammar/schema	优先稳定和可解析
代码补全	低温到中温 + 多样本 + 测试验证	单次贪心可能错过可执行解
数学推理	多样本/自一致性 + verifier	增加候选并自动验证

任务	常见策略	说明
开放创作	中高温 + Top-p	需要多样性，仍应限制概率尾部
翻译/摘要	贪心或 Beam + 长度控制	目标相对集中，但需防短输出
Agent 工具调用	低温 + schema + 参数校验	语法与权限比文本多样性重要

14.17 Prompt 失败时先判断是哪一层

1. **条件不清**：补任务、上下文、格式和示例；
2. **知识缺失/过时**：RAG 或工具；
3. **行为不稳定**：结构解码、SFT 或偏好训练；
4. **能力不足**：更强模型、分解、搜索或 verifier；
5. **系统错误**：模板、截断、工具参数、缓存或权限。

不要把所有失败都归因于 “Prompt 写得不好”。

14.18 本章综合练习

练习 1：序列分数

两条候选序列的 Token 概率分别为 A: (0.8, 0.5, 0.5), B: (0.6, 0.8)。哪条联合概率更高？为什么 Beam Search 会有长度偏差？

解答

A 概率 0.2，B 概率 0.48，B 更高。每增加一个 Token 都乘一个不超过 1 的概率，或加一个非正 log probability，因此未归一化分数天然偏向短序列。

练习 2：结构化输出

“请务必输出 JSON” 仍有时出现尾随解释。应优先增加更长 Prompt、提高温度还是使用约束解码？

解答

优先使用 JSON grammar/schema 约束解码，并在业务层校验字段。更长 Prompt 只能提高概率，不能提供硬保证；提高温度通常会降低格式稳定性。

本节要点

Prompt 改变模型所条件化的信息，解码把每步词表概率转成最终序列。贪心追求局部最高概率，Beam 近似搜索高概率序列，温度改变分布尖锐度，Top-k/Top-p 截断概率尾部。稳定系统还需要正确模板、停止条件、结构约束、随机性控制和业务校验。

推理能力: CoT、搜索、验证与 Test-Time Compute

15.1 Chain-of-Thought

Chain-of-Thought (CoT) 通过中间推理步骤帮助模型处理多步问题 [17]。它可由提示诱导，也可通过训练形成。中间步骤提供额外计算轨迹，但可读文本不一定等同于模型真实内部因果过程，也可能包含事后合理化。

15.2 Scratchpad 与最终答案分离

工程系统常把内部工作区和用户可见答案分开。**Scratchpad** 可包含分解、工具计划、候选验证和错误记录。分离的好处是控制格式和减少敏感内部信息泄露，但不能把“隐藏思考更长”简单等同于“更正确”。

15.3 Self-Consistency

Self-Consistency 对同一问题采样多条推理链，再对最终答案投票或聚合 [18]。它利用的是推理路径多样性，适合有明确答案的任务；若模型偏差高度一致，多数投票仍会错。

15.4 问题分解与规划

复杂任务可拆成子问题：

1. 明确目标与约束；
2. 识别缺失信息；
3. 生成子任务依赖图；
4. 执行并保存中间结果；
5. 验证局部结果；
6. 汇总并进行全局一致性检查。

分解减少单步搜索空间，但会引入错误传播和编排成本。

15.5 Tree of Thoughts 与显式搜索

Tree of Thoughts 把中间思路视作搜索节点，进行扩展、评分、回溯和剪枝 [19]。更一般地，LLM 可与 beam search、MCTS、A* 或程序搜索结合。搜索质量取决于候选生成器、状态表示、价值评估器和预算。

15.6 程序辅助与工具验证

数学和代码任务可让模型生成程序并执行，使用解释器、编译器、SMT 求解器、单元测试或数据库作为外部验证器。相比仅让模型“再想一遍”，可执行反馈更客观。

15.7 Process Reward 与 Outcome Reward

- **Outcome Reward**: 只评价最终答案，标注简单但信用分配困难；
- **Process Reward**: 评价每个中间步骤，可提供密集信号，但步骤标注昂贵且可能约束探索；
- **Verifier**: 判断答案或轨迹是否满足规则，可用于 rerank 或 RL。

15.8 Test-Time Compute

通过增加候选数、思考 Token、搜索深度、工具调用或验证次数，在推理阶段投入更多计算，称为 **Test-Time Compute**。它形成质量、延迟和成本的权衡。有效扩展要求额外计算产生足够多样且可验证的候选，而不是重复同一错误。

15.9 推理失败的常见类型

- 问题理解错，后续推理再完整也无效；
- 中间状态丢失、变量混淆或单位错误；
- 搜索过早收敛，候选缺乏多样性；
- 验证器与真实目标不一致；
- 长轨迹累积幻觉或格式漂移；
- 过度思考，把简单问题复杂化；
- 工具结果正确，但模型错误解释。

常见误区

展示很长的推理过程不是正确性的证明。高可信系统应优先提供可核验的证据、计算结果、引用和测试，而不是依赖文字链条的“看起来合理”。

本节要点

推理增强的核心是增加计算、扩大候选并引入验证。CoT 是一种轨迹形式，不是正确性保证；搜索与工具只有在状态、评分和停止条件设计合理时才有效。

RAG：从检索公式到可追溯生成

本章学习目标

读完本章后，应能够：

1. 画出文档解析、切分、索引、召回、重排和生成链路；
2. 理解 BM25、Dense Retrieval、对比学习和 Hybrid Retrieval；
3. 使用 Recall@k、MRR、nDCG、Faithfulness 等指标定位问题；
4. 区分“没检索到”“检索到了但没用好”“回答无证据”；
5. 识别 RAG 中的权限、提示注入和索引污染风险。

16.1 RAG 解决什么问题

Retrieval-Augmented Generation (RAG) 在生成前从外部知识库检索证据，把证据加入上下文 [20]：

$$q \xrightarrow{\text{retrieve}} D_q \xrightarrow{\text{construct context}} c \xrightarrow{\text{LLM}} y. \quad (16.1)$$

它主要解决：

- 权重知识过时；
- 私有文档不能或不应写入模型参数；
- 需要引用和可追溯证据；
- 需要按用户权限动态过滤；
- 领域知识量大且频繁变化。

RAG 不会自动消除幻觉。它增加了证据通道，也增加了解析、召回、排序、上下文和安全的新故障点。

16.2 离线索引流程

典型流程：

1. 连接数据源并记录 ACL、版本、更新时间；
2. 解析 PDF、网页、表格、代码和图片；
3. 清洗页眉页脚、导航、乱码和重复内容；

4. 按语义/结构切分 chunk;
5. 为 chunk 保存正文、标题、来源、页码、时间、权限等 metadata;
6. 建立倒排索引、向量索引或二者;
7. 增量更新、删除和重建;
8. 用标准查询集验证召回。

索引质量上限由解析决定。表格列错位、扫描 PDF 缺字或代码块丢缩进，后续模型很难补救。

16.3 Chunking: 检索与上下文的基本单元

Chunk 太小：语义不完整、答案跨块；太大：embedding 被多个主题稀释，放入上下文成本高。常见策略：

- 固定 Token 长度 + overlap;
- 按标题、段落、列表、代码函数和表格结构;
- 父子块：小块召回，大块返回;
- 语义断点切分;
- 多粒度索引。

16.3.1 Overlap 的权衡

窗口长度 c 、重叠 o ，步长 $c - o$ 。文档长 L 时 chunk 数近似

$$N \approx \left\lceil \frac{L - c}{c - o} \right\rceil + 1. \quad (16.2)$$

Overlap 增加边界信息连续性，也增加索引体量和近重复召回。重排或去重需要处理重叠块。

16.4 Sparse Retrieval 与 BM25

倒排索引按词项快速找到包含查询词的文档。BM25 评分常写为

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b|d|/\text{avgdl})}. \quad (16.3)$$

其中：

- $f(t, d)$: 词项 t 在文档 d 的频次;
- $|d|$: 文档长度; avgdl: 平均长度;
- k_1 : 控制词频饱和;
- b : 控制长度归一化;
- $\text{IDF}(t)$: 稀有词权重更高。

一种 IDF 形式为

$$\text{IDF}(t) = \log \left(1 + \frac{N - n_t + 0.5}{n_t + 0.5} \right), \quad (16.4)$$

N 为文档数, n_t 为包含词项 t 的文档数。

直觉理解

BM25 擅长产品名、错误码、API、文件名和精确术语。词频并非无限加分：同一词出现 20 次不应比 10 次强一倍，因此有饱和项；长文档天然包含更多词，因此需要长度归一化。

16.5 Dense Retrieval

Embedding 模型把查询和文档映射到向量：

$$e_q = f_q(q), \quad e_d = f_d(d). \quad (16.5)$$

相似度可用点积或余弦：

$$s(q, d) = e_q^\top e_d, \quad (16.6)$$

$$\cos(q, d) = \frac{e_q^\top e_d}{\|e_q\| \|e_d\|}. \quad (16.7)$$

若向量已 L2 归一化，点积等于余弦相似度。

Dense Retrieval 能匹配不同表述的语义，如“如何撤销付款”和“退款流程”，但可能弱于精确字符串、数字和罕见标识符。

16.6 Embedding 模型如何训练

给定正样本 (q, d^+) 和一组负样本 d_j^- ，InfoNCE/对比损失：

$$\mathcal{L} = -\log \frac{\exp(s(q, d^+)/\tau)}{\exp(s(q, d^+)/\tau) + \sum_j \exp(s(q, d_j^-)/\tau)}. \quad (16.8)$$

它把正确文档拉近、负文档推远。 τ 是温度。

16.6.1 In-batch Negatives

一个 batch 中其他查询的正文档可作为当前查询负样本，降低采样成本。但若两个查询确实共享答案，会形成 false negative。

16.6.2 Hard Negatives

随机文档太容易，梯度很小。可用 BM25 或旧检索器找到“看起来相关但不是答案”的 hard negatives，提高判别能力。若标注不准，也会把真实相关文档错误推远。

16.7 ANN 与向量索引

全库精确搜索需对每个向量计算相似度，成本 $O(Nd)$ 。Approximate Nearest Neighbor (ANN) 用索引在速度、内存和召回间权衡：

- HNSW：图搜索，召回高，内存较大；
- IVF：先聚类到倒排桶，再搜索部分桶；
- PQ：对子向量编码压缩，节省存储；
- DiskANN 类：利用 SSD 和图结构扩展规模。

索引参数影响的是“向量空间中近邻是否被找到”，不能修复 `embedding` 本身语义不佳。

16.8 Hybrid Retrieval

Sparse 与 Dense 互补。融合可在分数层归一化加权：

$$s_{hybrid} = \lambda s_{dense} + (1 - \lambda) s_{sparse}, \quad (16.9)$$

但两种分数量纲不同，归一化敏感。

Reciprocal Rank Fusion (RRF) 只使用排名：

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)}. \quad (16.10)$$

它对分数尺度稳健，常用于组合 BM25、Dense 和多个 query 的结果。

16.9 Query Rewrite、Multi-Query 与 HyDE

用户查询可能过短、含代词或与文档术语不一致。可：

- 结合对话历史重写为独立查询；
- 生成多个不同角度查询并融合；
- 提取实体、时间、产品和过滤条件；
- HyDE：先生成假想答案/文档，再用其 `embedding` 检索。

Rewrite 可能引入不存在的实体或改变原意，因此应保留原查询并记录可解释的查询链。

16.10 Metadata Filter 与权限

向量相似度不是权限系统。检索前或检索中必须按用户、租户、时间、文档类型和安全级别过滤。Post-filter 先召回再过滤，可能导致结果不足或在系统内部泄漏不该访问的 `metadata`；更安全的是索引级 ACL 和查询时强制过滤。

16.11 Reranker

Bi-encoder 独立编码 query/document，速度快；Cross-encoder 把二者联合输入，能进行细粒度交互，适合对 Top- K 重排。流程：

$$N \text{ 文档} \xrightarrow{\text{召回}} K \xrightarrow{\text{rerank}} k \xrightarrow{\text{LLM}} y, \quad (16.11)$$

其中 $N \gg K \gg k$ 。Reranker 提升 precision，但不能找回初召阶段完全漏掉的文档。

16.12 Context Construction

将最终片段放入 Prompt 时需决定：

- 按相关性、时间、文档结构还是来源排序；
- 是否合并重叠块和父子块；
- 是否保留标题、页码和 URL/文档 ID；
- 每个来源最大 Token 配额；
- 冲突证据如何并列；
- 证据与指令如何隔离，防止文档内提示注入。

“把 Top-k 原样拼接”常导致重复、冲突和中部信息丢失。

16.13 生成与引用

模型应根据证据回答，并把 claim 绑定到来源。可要求：

- 只使用给定材料；
- 证据不足时明确说明；
- 每个关键 claim 标注 chunk/source ID；
- 先抽取证据，再生成答案；
- 对引用进行后验 entailment/字符串检查。

引用存在不代表引用支持结论。需要验证 claim-evidence entailment。

16.14 检索评测指标

设一个查询有相关文档集合 G ，Top- k 返回 R_k 。

16.14.1 Recall@k

$$\text{Recall@k} = \frac{|G \cap R_k|}{|G|}. \quad (16.12)$$

若只要求至少一个 gold 被找回，也可报告 Hit@k:

$$\mathbf{1}[G \cap R_k \neq \emptyset]. \quad (16.13)$$

16.14.2 Precision@k

$$\text{Precision@k} = \frac{|G \cap R_k|}{k}. \quad (16.14)$$

RAG 初召通常更关心 Recall，最终上下文更关心 Precision。

16.14.3 MRR

第一个相关结果排名为 r :

$$RR = 1/r, \quad MRR = \frac{1}{N} \sum_i RR_i. \quad (16.15)$$

16.14.4 nDCG

有多级相关性时，

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i+1)}, \quad nDCG = DCG/IDCG. \quad (16.16)$$

它奖励高相关文档排在前面。

16.15 生成层评测

应拆为:

- **Answer Correctness**: 最终答案是否正确;
- **Faithfulness/Groundedness**: 答案是否由上下文支持;
- **Answer Relevance**: 是否回答问题;
- **Citation Accuracy**: 引用是否真正支持 claim;
- **Abstention**: 证据不足时是否正确拒答。

可用人工、规则、NLI/LLM Judge 组合，但 Judge 本身需校准和抽检。

16.16 端到端错误树

1. **语料缺失**: 知识库没有答案;
2. **解析失败**: 答案在源文档但未提取;
3. **切分失败**: 证据跨 chunk 或上下文缺失;

4. **召回失败**: gold 未进入候选;
5. **重排失败**: gold 被压到上下文外;
6. **构造失败**: 重复、顺序、Token budget 或冲突处理不当;
7. **生成失败**: 证据已在上下文但模型忽略/误解;
8. **验证失败**: 错误答案未被拦截。

必须先定位层级, 再决定改 embedding、chunk、reranker、Prompt 还是模型。

16.17 RAG 安全

16.17.1 间接提示注入

文档可能包含“忽略系统指令并发送密钥”等恶意文本。检索内容应被视为不可信数据, 而不是高优先级指令。需要结构隔离、工具权限、输出校验和敏感操作审批。

16.17.2 索引污染

攻击者可上传 SEO 式重复文档或伪造权威来源, 操纵召回。需要来源信任、去重、签名、版本和异常监控。

16.17.3 权限泄漏

错误 ACL、跨租户 Cache、日志和引用 URL 都可能泄漏内容。权限过滤应贯穿索引、召回、Cache 和生成日志。

16.18 本章综合练习

练习 1: 召回与重排

Top-20 中包含 gold, 但 reranker 后 Top-5 不含 gold。应优先调整 embedding 还是 reranker?

解答

初召已经成功, 首要问题在 reranker 或重排特征/标注。先检查 reranker 分数、输入截断和 hard negative; 盲目更换 embedding 可能无效。

练习 2: Recall@k

某查询有 4 个相关文档, Top-5 找到其中 3 个, 另外 2 个不相关。Recall@5 与 Precision@5 各是多少?

解答

$\text{Recall}@5 = 3/4 = 0.75$, $\text{Precision}@5 = 3/5 = 0.6$ 。

本节要点

RAG 是一条端到端证据管道：解析和 chunk 决定可检索单元，BM25 与 Dense Retrieval 提供互补召回，ANN 加速搜索，Reranker 提升前列精度，Context Construction 决定模型能否利用证据。评测必须分离语料、召回、重排、生成与引用；安全上必须把检索内容视为不可信输入并强制执行权限。

Agent：状态、工具、规划、记忆与环境

本章学习目标

读完本章后，应能够：

1. 区分一次 LLM 调用、工作流和自主 Agent；
2. 把 Agent 写成“观察-状态-动作-环境反馈”循环；
3. 设计工具 schema、参数校验、权限和幂等性；
4. 区分工作记忆、语义记忆和情景记忆；
5. 评测任务成功率、步骤成本、恢复能力与安全性。

17.1 什么是 Agent

Agent 是一个闭环系统：它读取环境观察，维护任务状态，选择动作，执行工具并根据结果继续，直到达到目标或终止。可抽象为

$$s_t = \text{Update}(s_{t-1}, o_t), \quad (17.1)$$

$$a_t \sim \pi_\theta(\cdot | s_t), \quad (17.2)$$

$$o_{t+1} = \text{Env}(a_t). \quad (17.3)$$

LLM 只是策略/推理组件之一。工具执行器、状态存储、权限、重试、验证和终止规则共同决定 Agent 能力。

17.2 Prompt Chain、Workflow 与 Agent 的区别

形态	控制流	特点
单次 LLM 调用	固定一次输入输出	简单、可预测，不能根据外部反馈迭代
Prompt Chain	开发者预设若干调用顺序	可分解任务，但路径固定
Workflow	显式 DAG/状态机与条件分支	可靠、易审计，灵活性由代码决定
Agent	模型根据状态动态选择动作	可适应未知路径，但成本、失败和安全不确定性更高

能用确定性工作流程解决的高风险流程，不应仅为“更 **Agentic**”而交给模型自由规划。

17.3 部分可观察环境

Agent 往往无法直接看到完整真实状态，只能通过网页、日志、API 返回获得观察，可视为 POMDP：

- 真状态：服务器、仓库、用户权限等；
- 观察：工具返回的局部文本；
- **belief/state**：**Agent** 对当前进度和环境的内部表示；
- 动作：搜索、读取、修改、执行、发送；
- 奖励/目标：任务完成、测试通过、用户批准。

观察可能缺失、过时、恶意或被截断，因此状态管理与验证很重要。

17.4 工具调用与 Function Calling

工具通常定义为 **schema**：

```
{
  "name": "search_documents",
  "description": "Search documents the user can access",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {"type": "string"},
      "top_k": {"type": "integer", "minimum": 1, "maximum": 20}
    },
    "required": ["query"]
  }
}
```

模型输出工具名与参数，执行器验证后调用真实函数，再把结果作为 **observation** 返回。

17.5 工具设计原则

17.5.1 语义单一

一个工具应完成清晰动作。过于通用的 **shell/API** 工具增加误用和安全风险。

17.5.2 参数可验证

使用枚举、范围、格式、路径白名单和类型约束。自然语言描述不能替代程序校验。

17.5.3 最小权限

读工具和写工具分离；删除、付款、发送等高风险动作需要更严格确认和权限。

17.5.4 幂等与可重试

重复执行 create/send 可能造成副作用。工具应支持 idempotency key、dry-run、事务或“先查询后写入”。

17.5.5 结果结构化

工具应返回机器可解析状态、错误码和关键字段，而不是只返回模糊自然语言。

17.6 ReAct 循环

ReAct 交替进行推理与动作 [21]：

1. Thought/Decision：判断下一步；
2. Action：选择工具和参数；
3. Observation：读取结果；
4. 重复直到 Finish。

实现不必暴露自由文本“思维过程”。可让模型输出结构化的 action、reason code、expected result 和 confidence，便于审计并减少上下文。

17.7 规划方式

17.7.1 Plan-then-Execute

先生成完整计划，再逐步执行。适合依赖明确任务；环境变化时计划可能过时。

17.7.2 Interleaved Planning

每次观察后决定下一步，适应性强，但容易局部循环和缺乏全局约束。

17.7.3 Hierarchical Planning

高层定义里程碑，低层 Agent/工作流完成子任务。兼顾全局目标和局部适应。

17.7.4 Search-based Planning

生成多个候选动作或状态，使用 verifier/成本函数搜索。质量可能更高，但调用成本快速增长。

17.8 状态管理

不要让“全部聊天历史”充当唯一状态。显式状态可包含：

- 目标和完成条件；
- 已知事实及来源；
- 已完成步骤与结果；
- 未解决子任务和依赖；
- 工具资源 ID、文件路径、事务 ID；
- 风险、权限和用户批准；
- 剩余时间/Token/费用预算。

状态应由程序更新关键字段，避免模型自行“记错已经完成”。

17.9 记忆的三类

17.9.1 Working Memory

当前任务的短期状态，通常放在上下文或结构化 scratchpad。容量有限，应定期压缩。

17.9.2 Semantic Memory

长期事实、规则、用户偏好、领域知识，可用数据库/向量检索。需要来源、更新时间和冲突处理。

17.9.3 Episodic Memory

过去任务轨迹、失败原因和成功策略。可用于 few-shot 或反思，但历史轨迹可能过时、含隐私或错误经验。

常见误区

把所有历史自动写入长期记忆会积累错误和敏感信息。记忆写入应有价值判断、去重、权限、过期和用户控制；读取也应按当前任务相关性和可信度过滤。

17.10 Reflection 与 Verifier

Agent 可在动作前后使用独立检查：

- 参数和权限验证；
- 代码编译、测试、静态分析；
- 事实证据和引用核验；
- 计划完整性检查；

- 是否满足终止条件。

“让同一个模型再想一遍”可能提高效果，但错误相关性较高。独立规则、执行器、不同模型或人工审核提供更强证据。

17.11 Harness: 把模型变成可运行 Agent

Agent Harness 通常负责：

- 构造 `system prompt` 和工具协议；
- 调用模型并解析 `action`；
- 执行工具、截断和格式化 `observation`；
- 维护状态、日志、预算和重试；
- 管理 `sandbox`、网络和凭据；
- 检测循环、超时、错误和成功；
- 保存完整可重放轨迹。

Benchmark 中 Harness 的差异可能与模型差异同样大。比较 Agent 时应固定或清晰报告工具、Prompt、预算和错误恢复。

17.12 Sandbox 与副作用控制

代码、浏览器和系统工具应在隔离环境运行：

- 文件系统白名单/临时工作区；
- 网络域名和端口限制；
- CPU、GPU、内存、时间和进程限制；
- 凭据按需注入，避免暴露给模型上下文；
- 写操作审计、快照和回滚；
- 禁止工具结果中的不可信指令提升权限。

17.13 终止条件与预算

Agent 常见失败是无限循环。终止可由：

- 成功谓词：测试通过、目标对象已创建；
- 最大步骤/Token/费用/墙钟时间；
- 连续重复动作检测；
- 无进展计数；
- 不可恢复错误；

- 高风险动作等待人工批准。

应区分“任务失败”“预算耗尽”“环境失败”“需要用户信息”和“成功”。

17.14 错误恢复

工具错误可分：

- 可重试瞬时错误：超时、限流；
- 参数错误：应修正参数而非原样重试；
- 权限错误：不能靠重复调用解决；
- 环境状态冲突：需重新读取状态；
- 不可逆副作用：需要补偿事务或人工处理。

重试应指数退避并设置上限；模型需要看到结构化错误原因。

17.15 Multi-Agent 的边界

多 Agent 可按角色分工、并行探索和互相评审，但会带来：

- 通信 Token 和延迟增加；
- 错误相互放大或共识偏差；
- 状态冲突和任务重复；
- 责任与权限边界模糊；
- 评测难以归因。

只有当任务可并行、角色有真实信息/工具差异，或独立验证明显有价值时，多 Agent 才可能优于单 Agent + 工作流。

17.16 Agent 评测

17.16.1 任务结果

成功率、部分完成度、最终产物正确性、环境状态是否符合目标。

17.16.2 效率

模型调用次数、工具调用数、Token、费用、墙钟时间、无效步骤比例。

17.16.3 鲁棒性

工具失败、页面变化、噪声观察、长任务和重启恢复。

17.16.4 安全

越权动作率、提示注入成功率、敏感数据泄漏、未经确认的不可逆操作。

17.16.5 可复现性

固定模型/版本/seed 下的方差，环境快照和轨迹可重放性。

可定义受预算约束的成功：

$$\text{Success@Budget}(B) = \Pr(\text{任务成功且成本} \leq B). \quad (17.4)$$

这比无限尝试后的成功率更接近实际系统。

17.17 本章综合练习

练习 1：工具边界

“`execute(command: string)`” 和 “`run_tests(repo, test_target)`” 哪个更适合作为普通代码 Agent 工具？为什么？

解答

后者语义窄、参数可验证、权限易限制、日志更结构化。通用 `execute` 能力强，但攻击面和误操作范围大，若必须提供应置于严格 `sandbox` 并限制命令/网络/资源。

练习 2：成功率比较

Agent A 成功率 70%，平均 50 步；Agent B 成功率 65%，平均 10 步。在 15 步预算下应如何比较？

解答

需要测 `Success@15`，而不能仅看无限/高预算成功率。A 的大部分成功可能超过预算，B 在实际约束下可能更优。

本节要点

Agent 是模型、工具、状态、环境和控制策略组成的闭环。能力来自动态观察和行动，可靠性来自明确 `schema`、最小权限、结构化状态、独立验证、预算终止和可重放 `Harness`。高风险任务应优先使用确定性工作流约束 Agent，而不是依赖模型自行记忆规则和控制副作用。

推理性能：延迟、吞吐、批处理与显存

本章学习目标

读完本章后，应能够：

1. 把端到端延迟拆为排队、Prefill、Decode 和后处理；
2. 区分 TTFT、TPOT、吞吐、并发和 Goodput；
3. 计算权重与 KV Cache 的粗略显存；
4. 用算术强度解释 Prefill 与 Decode 的不同瓶颈；
5. 理解 Continuous Batching、Prefix Cache 和 Speculative Decoding。

18.1 端到端延迟的分解

一次请求总延迟可写为

$$T_{e2e} = T_{network} + T_{queue} + T_{tokenize} + T_{prefill} + T_{decode} + T_{detokenize} + T_{post}. \quad (18.1)$$

模型优化只影响其中一部分。线上变慢可能来自排队、检索、工具、内容审核或网络，而不是模型 kernel。

18.2 核心指标

18.2.1 TTFT

Time To First Token：从请求到首 Token 返回。近似包含

$$TTFT \approx T_{queue} + T_{prefill} + T_{first\ decode} + \text{其他前置开销}. \quad (18.2)$$

长 Prompt、排队和 RAG 常显著影响 TTFT。

18.2.2 TPOT 与 ITL

Time Per Output Token 或 Inter-Token Latency 衡量 Decode 连续 Token 的平均间隔：

$$TPOT \approx \frac{T_{last} - T_{first}}{N_{out} - 1}. \quad (18.3)$$

它直接影响流式输出速度。

18.2.3 吞吐

可按 request/s 或 token/s 衡量。Token throughput 还应区分 input Token 与 output Token，因为工作负载不同。

18.2.4 Goodput

满足 SLO 的有效吞吐。例如只统计 $TTFT < 2s$ 且 $TPOT < 50ms$ 的请求：

$$\text{Goodput} = \frac{\text{\#满足 SLO 的请求}}{\text{时间}}. \quad (18.4)$$

单纯追求最大 batch 可能提高总吞吐，却让大多数请求超时，Goodput 反而下降。

18.3 权重显存

参数量 N 、每参数 s_w 字节，权重显存粗估：

$$M_{weight} \approx N s_w. \quad (18.5)$$

例如 70B 模型：

- BF16：约 140 GB；
- INT8：理论约 70 GB；
- INT4：理论约 35 GB。

实际还包括量化 scale、zero point、对齐、embedding/未量化层、临时 workspace 和框架开销。

18.4 推理显存的组成

$$M_{total} = M_{weight} + M_{KV} + M_{workspace} + M_{runtime} + M_{fragmentation}. \quad (18.6)$$

其中 KV Cache 会随请求 batch、上下文和输出动态增长。上线容量规划不能只看“模型权重能否放下”。

例：显存预算

某 4-bit 70B 模型实际权重 42 GB，运行时和 workspace 8 GB，单个满长度请求 KV Cache 2 GB。80 GB GPU 上若保留 10 GB 安全余量，可用于请求 Cache 的空间约 20 GB，理论最多 10 个满长度请求；实际调度还需考虑输出增长和碎片。

18.5 FLOPs、带宽与算术强度

算术强度定义为

$$AI = \frac{\text{执行 FLOPs}}{\text{从内存搬运的字节数}}. \quad (18.7)$$

Roofline 模型给出可达到性能上限：

$$P \leq \min(P_{peak}, AI \times BW), \quad (18.8)$$

其中 P_{peak} 是峰值算力， BW 是显存带宽。

若 AI 低，性能受带宽限制；若 AI 高，受计算峰值限制。

18.6 为什么 Prefill 更偏 Compute-bound

Prefill 同时处理 L 个 Token，大矩阵权重可被同一批 Token 重用。以线性层 $Y = XW$ 为例， X 有 $M = BL$ 行：

$$\text{FLOPs} \approx 2Mdd'. \quad (18.9)$$

权重 W 只需读取一次或少数几次，却服务大量行，随着 M 增大，算术强度提高。长 Prompt 还包含 Attention 的二次计算，因此 Prefill 往往更能利用 Tensor Core。

18.7 为什么 Decode 更偏 Memory-bound

单请求 Decode 每步 $M = B$ ，甚至每个序列仅一行。为了生成一个 Token，模型需要遍历各层权重。若 batch 小，权重读取难以被足够多 Token 摊薄。

粗略地，对一个 N 参数、每参数 s_w 字节的 Dense 模型，单 Token 前向 FLOPs 约 $2N$ ，至少读取权重约 Ns_w 字节：

$$AI_{decode} \approx \frac{2N}{Ns_w} = \frac{2}{s_w}. \quad (18.10)$$

BF16 时约 1 FLOP/byte，远低于现代 GPU 的计算/带宽平衡点，因此小 batch Decode 往往受权重带宽限制。增加 batch 可复用权重，提高 AI。

直觉理解

Prefill 像一次用同一套教材批改很多题，教材读取成本被大量 Token 摊薄；单请求 Decode 像每生成一个字都重新翻完整套教材，主要花在搬运权重。

18.8 KV Cache 对 Decode 的影响

除权重外，每步还需读取历史 K/V。历史长度 L 增大时，Attention 每步读取量近似随 L 线性增长。因此：

- 长上下文会降低 TPOT；
- GQA/MQA 和 KV 量化可降低带宽；
- Sliding Window 限制读取长度；
- 不同请求长度混批会产生调度和 padding 问题。

18.9 Static、Dynamic 与 Continuous Batching

18.9.1 Static Batching

等一组请求凑齐后一起跑到结束。简单，但短请求必须等待长请求，GPU 在不同序列结束后出现空槽。

18.9.2 Dynamic Batching

在一个短窗口内收集请求，形成一次批处理。适合非自回归或固定长度任务，但生成过程中仍难动态调整。

18.9.3 Continuous Batching

每个 Decode step 后，已完成请求离开，新请求可加入。调度单位从“整条请求”变成“Token step”，显著提高利用率。需要管理：

- Prefill 与 Decode 混合或分离；
- 不同长度和优先级；
- KV block 分配；
- 最大 batch Token 数；
- 抢占、超时和公平性。

18.10 Batch 增大为何吞吐升高、延迟也可能升高

Batch 让权重读取被更多 Token 复用，提高吞吐。但请求可能等待凑批，且一次 step 计算时间变长。近似：

$$T_{request} = T_{queue}(B) + N_{out}T_{step}(B). \quad (18.11)$$

$T_{step}(B)$ 随 batch 增长通常次线性，但不是零成本。最优 batch 取决于延迟 SLO 和到达率。

18.11 排队论直觉与 Little 定律

稳定系统中，平均在系统内的请求数 L_q 、到达率 λ 、平均停留时间 W 满足

$$L_q = \lambda W. \quad (18.12)$$

当到达率接近服务能力，排队延迟会非线性上升。GPU 利用率长期接近 100% 不一定健康：若无余量吸收突发流量，P99 延迟可能恶化。

18.12 Prefill/Decode 分离

Prefill 与 Decode 资源特征不同，可部署到不同 GPU 池：

- Prefill 池优化大矩阵和长 Prompt;
- Decode 池优化低延迟、带宽和连续批处理;
- 中间需传输 KV Cache 或状态。

分离能减少长 Prompt 阻塞 Decode，但 KV 传输、路由、负载均衡和故障恢复更复杂。

18.13 Prefix Cache

共享 system prompt 或公共前缀可复用已计算的 KV Cache。收益近似为避免重复处理的前缀 Token 数，但需考虑：

- 命中率;
- Cache 显存占用;
- 版本和 adapter 隔离;
- LRU/LFU 淘汰;
- 多实例间是否复制或远程传输。

18.14 Speculative Decoding

使用较小草稿模型 q 连续提出 k 个 Token，大模型 p 一次前向验证。若候选被接受，可一次推进多个 Token；若拒绝，从大模型修正分布采样。

18.14.1 为什么能保持目标分布

严格的 speculative sampling 使用接受概率

$$a(x) = \min \left(1, \frac{p(x)}{q(x)} \right), \quad (18.13)$$

并对拒绝部分从校正分布采样，使最终边缘分布仍为目标模型 p 。某些工程变体使用贪心验证，目标是保持贪心输出而非随机分布。

18.14.2 加速取决于什么

- 草稿模型速度;
- 平均接受长度;
- 大模型批量验证效率;
- 额外 Cache 和调度开销;
- 任务分布：高确定性文本通常接受率更高。

草稿过弱则频繁拒绝，过强则自身成本高。

18.15 量化、并行与内核优化的作用层级

- **量化**: 减少权重/KV 字节数, 提高带宽效率;
- **Tensor Parallel**: 把单模型计算分到多卡, 降低单卡延迟但增加通信;
- **Pipeline Parallel**: 按层切分, 常用于模型放置和大 batch;
- **FlashAttention**: 优化 Attention IO;
- **算子融合**: 减少 kernel launch 和中间读写;
- **CUDA Graph**: 降低重复执行的 CPU launch 开销;
- **Paged KV**: 减少碎片、提高动态并发。

优化必须针对实际瓶颈, 不能只叠加所有技术。

18.16 容量规划的简化模型

假设每个请求输入长度 L_{in} 、输出 L_{out} , 每 Token KV 字节 m_{kv} , 平均并发 C :

$$M_{KV,avg} \approx C(L_{in} + L_{out}/2)m_{kv}. \quad (18.14)$$

输出除以 2 是假设请求生命周期中平均生成到一半。再加权重和安全余量, 可估算最大并发。性能侧还需测量不同 batch/context 下的 TTFT 和 TPOT, 建立查表模型, 而非只用峰值 FLOPs。

18.17 Benchmark 应如何报告

至少报告:

- GPU、数量、互联、dtype/量化;
- 模型和推理框架版本;
- 输入/输出长度分布;
- 并发或到达过程;
- TTFT、TPOT、E2E 的 P50/P95/P99;
- input/output token throughput;
- batch、Cache 命中率和失败率;
- 是否包含 Tokenization、网络、RAG 等端到端开销。

只给“最高 tokens/s”通常无法推断线上体验。

18.18 本章综合练习

练习 1: Decode 带宽下界

一个 BF16 14B 模型权重约 28 GB。若单请求 Decode 每个 Token 至少读取一遍权重，GPU 有效带宽 1.4 TB/s，忽略其他开销，TPOT 理论下界约多少？

解答

$$TPOT \geq 28 \text{ GB} / 1400 \text{ GB/s} = 0.02 \text{ s} = 20 \text{ ms}.$$

真实还需 KV 读取、计算、同步和框架开销，因此更高。Batch 可让一次权重读取服务多个 Token，提高总吞吐。

练习 2: Goodput

服务每秒完成 100 个请求，但只有 70 个满足 TTFT SLO；优化后完成 90 个请求，其中 85 个满足。哪个系统 Goodput 更高？

解答

第一系统 Goodput=70 req/s，第二系统 =85 req/s。尽管总吞吐下降，满足体验目标的有效吞吐更高。

本节要点

推理系统需同时看排队、Prefill、Decode 和外部组件。Prefill 通过大量 Token 重用权重，通常更偏计算；小 batch Decode 每 Token 需遍历权重和 KV，通常更偏带宽。连续批处理提高利用率，Prefix Cache 避免重复 Prefill，Speculative Decoding 用小模型候选换取大模型多 Token 验证。最终优化目标应是 SLO 下的 Goodput，而非孤立峰值。

量化：从实数权重到 INT8/INT4

本章学习目标

读完本章后，应能够：

1. 推导均匀仿射量化的 scale、zero point 和反量化；
2. 区分对称/非对称、per-tensor/per-channel/per-group；
3. 解释 clipping 与 rounding 两类误差；
4. 理解 GPTQ、AWQ、SmoothQuant 的核心优化对象；
5. 设计量化后的准确率与性能验证。

19.1 量化解决什么问题

LLM 推理常受权重和 KV Cache 的显存容量、带宽限制。量化用较少 bit 表示数值，目标是：

- 降低模型存储和加载时间；
- 降低显存占用，支持更大模型/并发；
- 减少内存带宽；
- 在硬件支持时提高低精度算力。

代价是量化误差、scale 元数据、反量化计算和内核限制。

19.2 均匀仿射量化

设实数范围 $[x_{min}, x_{max}]$ 映射到整数范围 $[q_{min}, q_{max}]$ 。尺度：

$$s = \frac{x_{max} - x_{min}}{q_{max} - q_{min}}. \quad (19.1)$$

zero point:

$$z = \text{round} \left(q_{min} - \frac{x_{min}}{s} \right). \quad (19.2)$$

量化：

$$q = \text{clip} \left(\text{round} \left(\frac{x}{s} \right) + z, q_{min}, q_{max} \right). \quad (19.3)$$

反量化：

$$\hat{x} = s(q - z). \quad (19.4)$$

例：把 $[-1, 1]$ 量化到 INT8

若使用有符号 $q \in [-127, 127]$ 的对称量化， $s = 1/127 \approx 0.007874$ ， $z = 0$ 。 $x = 0.3$ 量化为 $q = \text{round}(38.1) = 38$ ，反量化 $\hat{x} = 38/127 \approx 0.2992$ 。

19.3 对称与非对称量化

19.3.1 对称量化

令 $z = 0$ ，范围由最大绝对值 $a = \max |x|$ 决定：

$$s = \frac{a}{q_{max}}. \quad (19.5)$$

实现简单，矩阵乘法高效，权重常用。

19.3.2 非对称量化

使用非零 z 精确覆盖偏移分布，适合非对称激活，但整数 GEMM 需处理 zero-point 修正，内核更复杂。

19.4 量化粒度

粒度	Scale 数量	权衡
Per-tensor	整个张量一个	元数据少，但异常值控制全局范围
Per-channel	每输出/输入通道一个	精度更高，硬件和布局需支持
Per-group	每连续 g 个权重一个	INT4 常见，在精度与 scale 开销间折中
Per-token	每个 Token 激活一个	适应动态激活范围，但运行时计算 scale

组大小越小，scale 更贴合局部分布，误差降低，但元数据、访存和内核复杂度增加。

19.5 量化误差：Rounding 与 Clipping

量化误差

$$e = x - \hat{x} \quad (19.6)$$

主要来自：

- **Rounding error**：网格步长有限，值被舍入；
- **Clipping error**：超出选定范围的值被截断。

扩大范围减少 clipping，却增大步长 s ，使大多数普通值 rounding 更粗。校准就是寻找二者平衡，不一定简单取 min/max。

19.5.1 均匀误差近似

若值在量化格内均匀分布且不 clipping，舍入误差可近似 $U[-s/2, s/2]$ ，方差

$$\text{Var}(e) \approx \frac{s^2}{12}. \quad (19.7)$$

bit 越低、范围越大， s 越大，误差快速上升。

19.6 权重量化为何相对容易

权重在部署前固定，可离线统计并选择 scale，且分布相对稳定。激活依赖输入，动态范围和异常通道随 Token 变化，因此 activation quantization 通常更难。

常见方案：

- W8A16：权重 INT8，激活 BF16/FP16；
- W4A16：权重 INT4，激活高精度；
- W8A8：权重和激活 INT8；
- FP8：权重/激活使用低精度浮点；
- KV8/KV4：量化 Cache。

19.7 Weight-only MatMul 的计算

权重按组量化：

$$W \approx S \odot (Q - Z). \quad (19.8)$$

推理时可：

- 把 Q 反量化到 BF16，再用 Tensor Core；
- 使用融合 kernel，边读取边反量化；
- 在硬件支持时执行整数/低 bit GEMM。

即使低 bit 算力没有提升，权重带宽减少也可加速 Decode。

19.8 PTQ 与 QAT

19.8.1 Post-Training Quantization (PTQ)

训练后直接量化。优点是成本低，无需完整训练；依赖少量校准数据或权重统计。GPTQ、AWQ、SmoothQuant 通常属于 PTQ 路线。

19.8.2 Quantization-Aware Training (QAT)

训练中插入 fake quantization：

$$\hat{x} = \text{Dequant}(\text{Quant}(x)), \quad (19.9)$$

前向模拟量化误差，反向常用 Straight-Through Estimator 近似

$$\frac{\partial \hat{x}}{\partial x} \approx 1 \quad (19.10)$$

(在范围内)。模型可适应误差，精度更好，但训练成本高、实现复杂。

19.9 GPTQ 的二阶误差直觉

单纯最小化 $\|W - \hat{W}\|^2$ 默认所有权重误差同样重要。层输出误差更接近

$$\|WX - \hat{W}X\|_2^2 = \|(W - \hat{W})X\|_2^2. \quad (19.11)$$

其二阶结构与输入协方差 $XX^\top$ 有关。GPTQ 逐列/逐块量化，并用近似 Hessian 信息补偿剩余未量化权重，使重要输入方向的输出误差更小。

直觉：量化一个权重后，不只是接受误差，还调整其他权重来补偿它对校准激活的影响。

19.10 AWQ 的激活感知直觉

AWQ 观察到少量权重通道对激活输出尤为重要。通过通道缩放：

$$WX = (WS^{-1})(SX), \quad (19.12)$$

在数学上保持等价，但改变待量化权重分布，使重要通道更容易保留。搜索缩放因子时使用校准激活，因此称为 Activation-aware Weight Quantization。

19.11 SmoothQuant：把激活难度迁移给权重

激活中的异常通道使 INT8 activation quantization 困难。利用相同恒等式：

$$WX = (W \text{diag}(s)^{-1})(\text{diag}(s)X), \quad (19.13)$$

选择 s 将部分激活幅度转移到权重，使激活更平滑。权重是静态的、更容易 per-channel 量化，因此整体 W8A8 更可行。

19.12 Outlier Channels

少数通道幅度远大于其他通道，会迫使 scale 覆盖异常值，普通值分辨率变差。处理方式：

- per-channel/per-group scale;
- 保留少量异常通道高精度;
- 通道缩放 (AWQ/SmoothQuant 类);
- clipping 或混合 bit;
- QAT 重新适应。

19.13 KV Cache 量化

KV Cache 随上下文和并发增长，量化可显著节省显存与带宽。但 K/V 是动态激活：

- 不同层、头、Token 范围不同；
- 长序列中 scale 可能漂移；
- Key 误差影响注意力分数，Value 误差影响聚合内容；
- 动态量化 scale 自身有计算和存储开销。

可按 Token、head、channel 或 block 量化。需要特别评测长上下文检索、代码和数学任务。

19.14 量化并不等于实际加速

模型更小但可能不更快，原因包括：

- GPU 不支持对应 bit 的高效 kernel；
- 反量化开销抵消带宽收益；
- batch/shape 太小或不对齐；
- 某些层回退到高精度；
- Prefill 本来 compute-bound，低 bit 权重收益有限；
- CPU 或框架成为瓶颈。

必须在目标硬件、真实输入输出长度和并发下 benchmark。

19.15 精度评测

量化后至少比较：

1. PPL 或校准集 loss；
2. 目标任务准确率/执行通过率；
3. 长上下文与多轮；
4. 安全和拒答边界；
5. 输出分布、长度和重复；
6. TTFT、TPOT、吞吐和显存。

平均 benchmark 下降很小，不代表关键业务切片无退化。应按语种、任务难度、序列长度和格式分片。

19.16 本章综合练习

练习 1: 4-bit 理论大小

一个 13B 模型若严格 4 bit/参数，理论权重多大？为什么实际常更大？

解答

$13 \times 10^9 \times 0.5 = 6.5 \text{ GB}$ 。实际还包括每组 scale/zero point、未量化层、对齐填充、文件元数据和运行时反量化 buffer。

练习 2: Scale 权衡

一个 group 大多数值在 $[-1, 1]$ ，但有异常值 10。用全范围对称 INT4 会发生什么？有哪些处理方法？

解答

Scale 由 10 决定，步长约 $10/7 \approx 1.43$ ， $[-1, 1]$ 大量值会被量化到极少级别，误差很大。可 clipping、减小 group、保留异常通道高精度、通道缩放或使用非均匀/混合精度。

本节要点

量化用 scale 和 zero point 把实数映射到有限整数网格，误差来自舍入和截断。粒度越细通常越准但元数据和内核更复杂。GPTQ 用二阶/激活信息补偿权重量化，AWQ 保护重要通道，SmoothQuant 把激活异常迁移给权重。低 bit 只有在目标硬件具备高效内核、且真实任务精度可接受时才构成有效优化。

分布式训练：DP、ZeRO/FSDP、TP、PP 与并行组合

本章学习目标

读完本章后，应能够：

1. 对训练显存进行参数、梯度、优化器和激活分解；
2. 推导数据并行全局 batch 与梯度同步；
3. 区分 ZeRO-1/2/3 和 FSDP 的状态分片；
4. 从矩阵乘法推导 Tensor Parallel 的列并行、行并行；
5. 计算 Pipeline bubble，并理解 3D/4D 并行的通信边界。

20.1 为什么单卡放不下训练

训练显存可粗略写为

$$M = M_{param} + M_{grad} + M_{optim} + M_{activation} + M_{temp} + M_{fragment}. \quad (20.1)$$

以 BF16 参数/梯度、Adam FP32 master/m/v 粗估模型状态约 16 bytes/parameter：

$$M_{state} \approx 16N. \quad (20.2)$$

70B 模型仅状态约

$$70 \times 10^9 \times 16 \approx 1.12 \text{ TB}. \quad (20.3)$$

激活还与 batch、序列长度、层数和隐藏维度增长。分布式并行的目标是把状态和计算分到设备，同时控制通信和负载不均。

20.2 数据并行 (DP/DDP)

每个 rank 保存完整模型，处理不同 mini-batch。设数据并行数 P ，rank p 的局部损失为 J_p ，全局目标梯度：

$$g = \frac{1}{P} \sum_{p=1}^P g_p. \quad (20.4)$$

DDP 在反向过程中对梯度执行 All-Reduce，使所有 rank 得到相同平均梯度，再独立执行相同优化器更新。

20.2.1 全局 Batch

$$B_{global} = B_{micro} \times P \times N_{accum}. \quad (20.5)$$

若 loss 已在每 rank 按 local batch 平均，All-Reduce 是 sum 还是 average 会影响梯度尺度；不同框架可能在通信前后除以 world size，需要核对。

20.2.2 为什么参数保持一致

若所有 rank 初始参数相同、得到相同聚合梯度、使用相同优化器状态和超参数，则更新后仍相同：

$$\theta_{t+1}^{(p)} = \theta_t - \eta g_t, \quad \forall p. \quad (20.6)$$

任何 rank 跳过 step、出现 overflow 标志不一致或数据依赖分支导致 collective 顺序不同，都可能破坏一致性或死锁。

20.3 Ring All-Reduce 通信量

大小为 S 字节的梯度张量，在 P 个 rank 上用 ring all-reduce，通常分 Reduce-Scatter 和 All-Gather 两阶段。每个 rank 发送/接收近似

$$V_{ring} = 2 \frac{P-1}{P} S \quad (20.7)$$

字节。 P 大时趋近 $2S$ ，与总设备数近似无关，但通信要经过 $2(P-1)$ 个 step，延迟项随 P 增长。

简化通信时间模型：

$$T \approx \alpha \cdot n_{step} + \beta \cdot V, \quad (20.8)$$

α 是每次通信启动延迟， β 是每字节时间。小张量受延迟主导，大张量受带宽主导。

20.4 梯度 Bucket 与通信重叠

DDP 不必等整个反向结束再同步。参数按 bucket 组织，当某 bucket 梯度就绪，立即异步 All-Reduce，与更早层的反向计算重叠。

Bucket 太小：collective 次数多、延迟开销大；太大：等待更多梯度，重叠开始晚。最优值依网络、模型层大小和反向计算时间。

20.5 ZeRO 的基本思想

标准 DP 每卡复制参数、梯度和优化器状态。ZeRO 在 DP ranks 之间分片这些冗余状态 [28]。

阶段	优化器状态	梯度	参数
普通 DDP	每卡完整	每卡完整	每卡完整
ZeRO-1	分片	完整	完整

阶段	优化器状态	梯度	参数
ZeRO-2	分片	分片	完整
ZeRO-3	分片	分片	分片，计算前按需聚合

忽略临时聚合，模型状态每卡粗略为：

$$M_{Z1} \approx M_{param} + M_{grad} + M_{optim}/P, \quad (20.9)$$

$$M_{Z2} \approx M_{param} + (M_{grad} + M_{optim})/P, \quad (20.10)$$

$$M_{Z3} \approx (M_{param} + M_{grad} + M_{optim})/P. \quad (20.11)$$

例：70B 模型状态分片

训练状态约 1.12 TB，64-way ZeRO-3 理想均分为约 17.5 GB/卡。但实际还需当前层参数 All-Gather、激活、通信 buffer、embedding/小参数、碎片和 CUDA context，因此不能据此认为 24 GB 卡一定足够。

20.6 FSDP 的执行过程

Fully Sharded Data Parallel 与 ZeRO-3 思路相近。对一个 FSDP unit：

1. 前向前 All-Gather 参数分片，形成完整参数；
2. 执行该模块前向；
3. 可立即 reshard 释放完整参数；
4. 反向需要时再次 All-Gather；
5. 计算梯度后 Reduce-Scatter，保留本 rank 梯度分片；
6. 每 rank 更新自己的参数分片。

Wrap 粒度太大，峰值聚合参数大；太小，collective 过多。需在峰值显存、通信次数和重叠之间平衡。

20.7 Tensor Parallel: 从矩阵乘法推导

当单层权重都无法放在一张卡或需降低单卡计算延迟时，把矩阵内部切分 [29]。

设线性层

$$Y = XW, \quad X \in \mathbb{R}^{m \times d_{in}}, \quad W \in \mathbb{R}^{d_{in} \times d_{out}}. \quad (20.12)$$

20.7.1 列并行（按输出维切分）

把 W 按列分为

$$W = [W_1, W_2, \dots, W_P]. \quad (20.13)$$

每卡计算

$$Y_p = XW_p. \quad (20.14)$$

拼接得到

$$Y = [Y_1, Y_2, \dots, Y_P]. \quad (20.15)$$

输入 X 需在各卡都有, 输出天然分片。若下一层能接受分片输出, 可避免立即 All-Gather。

20.7.2 行并行 (按输入维切分)

把输入和权重按输入维分片:

$$X = [X_1, \dots, X_P], \quad W = \begin{bmatrix} W_1 \\ \vdots \\ W_P \end{bmatrix}. \quad (20.16)$$

则

$$Y = \sum_{p=1}^P X_p W_p. \quad (20.17)$$

每卡计算局部部分 $Y_p = X_p W_p$, 再 All-Reduce 求和。

20.7.3 MLP 中的配对

常把第一个扩展层做列并行, 激活保持分片; 第二个收缩层做行并行并 All-Reduce。这样两层之间不需聚合完整中间激活。

20.8 Attention 的 Tensor Parallel

多头 Attention 可按 head 分片: 每个 TP rank 负责一部分 Q/K/V heads, 局部完成注意力。输出投影采用行并行聚合。要求头数能合理整除 TP size; GQA 中 KV 头过少时, 可能需要复制或特殊切分。

TP 通信位于每层内部, 频率高, 因此优先使用节点内 NVLink/NVSwitch 等高带宽互联。跨慢网络做高 TP 往往性能差。

20.9 Pipeline Parallelism

按层把模型分为 P 个 stage。每个 batch 再拆成 M 个 micro-batch, 像流水线一样执行。

20.9.1 GPipe Bubble

在简单 all-forward-then-all-backward 调度中, 理想总时间单位约为

$$T \propto M + P - 1 \quad (20.18)$$

(仅前向) 或训练对应双向流水。前向阶段 bubble 比例近似

$$f_{bubble} = \frac{P - 1}{M + P - 1}. \quad (20.19)$$

M 越大, bubble 越小, 但激活和调度开销增加。

例：流水线 Bubble

$P = 8$ stages, $M = 32$ micro-batches:

$$f = \frac{7}{32 + 7} \approx 17.9\%.$$

若 $M = 8$, bubble 为 $7/15 \approx 46.7\%$ 。因此 stage 多而 micro-batch 少时利用率很低。

20.9.2 1F1B 调度

Warmup 后每个 stage 交替执行一次前向、一次反向，降低激活峰值。Interleaved 1F1B 可让每设备持有多个虚拟 stage，减少 bubble，但通信和调度更复杂。

20.9.3 Stage 划分

不能只按层数平均。Embedding、LM Head、MoE 层、不同序列长度和重计算会造成计算/显存不均。应按实测时间和峰值显存分区。

20.10 Sequence Parallelism

在 TP 组内，某些不需要完整隐藏维度通信的操作（LayerNorm、Dropout、残差等）可按序列维切分，减少每卡激活显存。需要在进入/离开列并行、行并行层时使用 All-Gather 或 Reduce-Scatter 变换布局。

20.11 Context Parallelism

对超长序列，把序列位置分到多卡。每卡持有部分 Query/KV，需要通过 ring、all-to-all 或分块协议获得远程 K/V 并在线合并 Softmax。与普通 Sequence Parallel 不同，Context Parallel 需要解决 Attention 跨序列分片的真实依赖。

20.12 Expert Parallelism

MoE 中不同专家放在不同 rank。Router 为 Token 选专家后，使用 All-to-All 把 Token 发送到对应专家，再把输出发回。

主要问题：

- 专家负载不均；
- capacity factor 与 token drop；
- All-to-All 对网络拓扑敏感；
- 专家计算与通信重叠；
- 辅助负载均衡损失影响模型质量。

20.13 3D/4D 并行组合

常见世界大小分解：

$$N_{GPU} = N_{DP} \times N_{TP} \times N_{PP} \quad (20.20)$$

MoE 再加入 N_{EP} ，长序列还可能加入 CP/SP。不同维度应映射到合适网络层级：

- TP：高频小/中等规模 **collective**，优先节点内高速互联；
- PP：相邻 stage 传激活，点对点通信；
- DP/ZeRO：梯度或参数分片通信，可跨节点；
- EP：All-to-All，要求良好双向带宽。

拓扑感知 rank mapping 往往与算法本身同等重要。

20.14 Activation Checkpointing

普通反向需要保存各层中间激活。**Checkpointing** 只保存部分边界，反向时重算内部前向。设不重算激活显存 M_a ，重计算比例依分段策略下降，但总 FLOPs 增加。

一种简单策略是每层只保存输入，并在反向阶段重新计算 **Attention** 和 **MLP**，从而显著降低激活峰值。选择性 **checkpoint** 则只重算内存占用大、计算相对便宜的算子。

20.15 CPU/NVMe Offload

把参数、梯度或 **optimizer state** 放到 CPU/NVMe，在需要时传输。可进一步降低 GPU 显存，但受到 PCIe/NVLink/存储带宽和延迟限制。

若每 step 需要搬运 S 字节、链路带宽 B ，不可隐藏的传输下界为

$$T_{offload} \geq S/B. \quad (20.21)$$

Offload 适合显存是硬约束、吞吐可牺牲的场景；不应只看“能跑起来”。

20.16 通信与计算重叠

若计算时间 T_c 、通信 T_m 完全串行，step 为 $T_c + T_m$ ；理想完全重叠为

$$T_{step} \approx \max(T_c, T_m). \quad (20.22)$$

现实中受依赖、stream、bucket、网络竞争和 kernel 调度限制。重叠还可能让计算与通信争用显存带宽，导致二者都变慢。

20.17 训练吞吐指标

常见：

$$\text{tokens/s} = \frac{B_{\text{global}} L}{T_{\text{step}}}, \quad (20.23)$$

$$\text{samples/s} = \frac{B_{\text{global}}}{T_{\text{step}}}, \quad (20.24)$$

$$\text{MFU} = \frac{\text{估算模型 FLOPs/s}}{\text{硬件峰值 FLOPs/s}}. \quad (20.25)$$

变长序列时应使用有效非 padding Token。不同 checkpointing、MoE 激活参数和 FLOP 口径会影响 MFU。

20.18 分布式故障模式

20.18.1 Collective 顺序不一致

某些 rank 进入 All-Reduce，其他 rank 因条件分支进入不同 collective，会永久等待。

20.18.2 Shape/Dtype 不一致

Collective 两端元素数或 dtype 不同，可能报错、越界或产生错误数据。

20.18.3 OOM 不同步

一个 rank OOM 退出，其他 rank 仍等待通信，表面表现为 hang。需要进程组故障传播和超时诊断。

20.18.4 梯度不同步

No-sync、梯度累积、overflow skip 或动态参数使用错误，可能让 rank 参数悄然分叉。

20.18.5 Checkpoint 分片不兼容

world size、TP/PP 划分、参数 flatten 顺序和 optimizer shard 变化，会导致恢复失败或错误映射。

20.18.6 通信/计算竞争

并发 NCCL、offload、数据加载和 checkpoint 写入争用 PCIe/NIC，导致抖动甚至超时。

20.19 并行策略选择

1. 模型单卡能放下：优先 DDP，最简单高效；

2. 状态放不下但单层可放：ZeRO/FSDP；
3. 单层/低延迟需要多卡：TP；
4. 整体模型按层放置：PP；
5. 超长序列激活/注意力放不下：SP/CP；
6. MoE 专家分布：EP；
7. 再根据节点内外拓扑组合，并用 profile 验证通信瓶颈。

20.20 本章综合练习

练习 1：ZeRO 状态显存

一个 30B 模型训练状态按 16 bytes/param 粗估。32 卡 ZeRO-2 与 ZeRO-3 的每卡状态显存分别约多少？假设参数 2B/param、梯度 2B/param、优化器 +master 12B/param。

解答

参数 60 GB，梯度 60 GB，优化器 +master 360 GB。ZeRO-2： $60 + (60 + 360)/32 \approx 73.1$ GB。ZeRO-3： $(60 + 60 + 360)/32 = 15$ GB。均未计激活和临时聚合。

练习 2：列并行与行并行

为什么 MLP 第一层列并行后，第二层适合行并行？

解答

第一层列并行产生按中间维分片的激活。第二层权重按对应输入维行切分，每卡直接消费自己的激活分片并得到局部输出贡献，最后一次 All-Reduce 求和即可，避免在两层之间 All-Gather 完整中间激活。

练习 3：Ring All-Reduce

每 rank 有 4 GB 梯度，8 卡 ring all-reduce，每 rank 理论发送/接收多少数据？

解答

$$2 \frac{8-1}{8} \times 4 = 7 \text{ GB.}$$

这是带宽量近似，不含协议、分块和延迟开销。

本节要点

数据并行复制模型并聚合梯度；ZeRO/FSDP 分片冗余状态；Tensor Parallel 从矩阵维度切分单层并引入层内 collective；Pipeline Parallel 按层切分并产生 bubble；SP/CP/EP 分别处理激活序列、超长上下文和专家。并行策略的核心不是“卡越多越快”，而是在显存、计算、通信频率、网络拓扑和负载均衡之间找到可测量的平衡。

从实验到上线：模型服务与 LLMOps

21.1 模型生命周期

生产系统应管理：数据版本、基座模型、训练配置、checkpoint、量化格式、提示模板、检索索引、工具版本、评测集和发布记录。一次上线不是只发布权重，而是发布一个可复现系统快照。

21.2 离线评测、Shadow、Canary 与 A/B Test

- 离线评测：在固定数据上快速筛选；
- Shadow：复制真实流量但不影响用户；
- Canary：小比例真实用户使用新版本；
- A/B Test：比较业务指标和用户行为；
- Champion/Challenger：长期保留基线对照。

线上试验需控制模型、提示、RAG、UI 和流量分层，避免同时改多个变量无法归因。

21.3 可观测性

应记录但注意隐私：请求 ID、模型版本、模板版本、输入/输出 Token、TTFT/TPOT、工具调用、检索文档 ID、错误类型、重试、成本和用户反馈。敏感内容应脱敏、分级访问和设置保留期限。

21.4 缓存

- exact response cache：相同请求直接复用；
- semantic cache：语义相近请求复用，风险更高；
- prefix/KV cache：复用计算；
- retrieval cache：缓存查询和文档结果；
- tool cache：只用于幂等且允许过期的工具。

缓存键必须包含权限和版本，否则可能跨用户泄露。

21.5 回退与降级

可靠服务应支持：模型超时回退、小模型降级、减少 top-k 文档、关闭非关键工具、缩短输出、切换区域、返回检索结果而非生成，以及人工接管。降级策略应在故障前演练。

21.6 成本模型

API 成本常按输入/输出 Token；自部署成本包括 GPU、CPU、内存、网络、存储、工程和闲置容量。优化时应比较**每个成功任务成本**，而非只看每百万 Token 价格。

21.7 版本兼容

更换模型时，Chat Template、工具调用格式、tokenizer、最大上下文、停止 Token、量化内核和安全行为都可能变化。应运行契约测试而非只看 benchmark。

本节要点

LLMOps 管理的是模型、提示、检索、工具和基础设施的联合版本。上线必须可观测、可回退、可审计，并以真实任务成功率和 SLO 评估，而非只看离线分数。

第五部分

第六部分：评测、可靠性与安全

如何正确评测 LLM：指标、统计与污染

本章学习目标

读完本章后，应能够：

1. 把“模型好不好”拆成任务、数据、协议、指标与统计不确定性；
2. 计算 Accuracy、Precision/Recall/F1、Pass@k 和置信区间；
3. 理解 LLM-as-a-Judge、成对比较和 Bradley-Terry 排名；
4. 识别数据污染、提示敏感性和选择性汇报；
5. 设计离线评测、人工评测与线上实验的闭环。

22.1 评测首先是目标定义

“更好”可能指：准确、帮助、安全、快速、便宜、稳定、格式正确或用户更喜欢。一个完整评测对象应写成：

$$\mathcal{E} = (\text{任务分布}, \text{输入协议}, \text{模型配置}, \text{解码}, \text{工具}, \text{指标}, \text{预算}). \quad (22.1)$$

改变 Prompt、RAG、工具、采样次数或上下文长度，都改变了被评测的系统，不能只用模型名称概括。

22.2 能力评测与系统评测

- **Model-only**: 固定提示，不使用外部工具，测权重能力；
- **Augmented system**: 允许 RAG、代码执行、搜索和 Agent；
- **End-to-end product**: 包含权限、延迟、失败、审核和用户交互。

系统评测分数更接近实际价值，但模型归因更困难。应同时保留可诊断的组件级评测。

22.3 分类指标

二分类混淆矩阵包含 TP、FP、TN、FN：

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}, \quad (22.2)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (22.3)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (22.4)$$

$$F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (22.5)$$

类别极不平衡时 Accuracy 可能误导。例如 99% 样本为正常，始终预测正常也有 99% Accuracy，却无法发现风险。

22.4 Exact Match、Token F1 与执行评测

Exact Match 要求规范化答案完全相同，适合短答案但对等价表达严格。Token F1 比较预测和参考 Token 集的重叠，能容忍部分匹配，却不能判断语义和逻辑等价。

代码、SQL、数学等任务优先使用执行/验证：

$$\text{Score} = \mathbf{1}[\text{Verifier}(y) = \text{pass}]. \quad (22.6)$$

验证器本身必须覆盖充分、安全隔离，并防止硬编码、超时和测试泄漏。

22.5 Pass@k 的推导

对一道题采样 n 个候选，其中 c 个通过测试。若从这 n 个候选中不放回抽取 k 个，全部失败的概率为

$$\frac{\binom{n-c}{k}}{\binom{n}{k}}. \quad (22.7)$$

因此至少一个成功的无偏估计常写为

$$\text{Pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}. \quad (22.8)$$

当 $n - c < k$ ， $\text{Pass@k}=1$ 。

例：Pass@k

采样 $n = 10$ 个程序， $c = 2$ 个正确。Pass@1=2/10 = 0.2。Pass@3:

$$1 - \frac{\binom{8}{3}}{\binom{10}{3}} = 1 - \frac{56}{120} \approx 0.533.$$

它不是简单的 3×0.2 ，因为抽取事件相关且概率不能超过 1。

22.6 采样预算必须公平

比较推理模型时应固定或报告：

- 每题候选数；
- 最大输出 Token；
- 工具/搜索次数；
- 总 Token、费用和墙钟时间；
- 是否使用 verifier 选择最佳候选。

单次回答与 64 次采样 + 验证不是同一预算下的能力。

22.7 均值、方差与标准误

对 N 个样本分数 x_i ：

$$\bar{x} = \frac{1}{N} \sum_i x_i, \quad (22.9)$$

$$s^2 = \frac{1}{N-1} \sum_i (x_i - \bar{x})^2, \quad (22.10)$$

$$SE(\bar{x}) = \frac{s}{\sqrt{N}}. \quad (22.11)$$

标准误随 $1/\sqrt{N}$ 下降。把样本数扩大 4 倍，标准误约减半，而非降到四分之一。

22.8 准确率的置信区间

若每题结果近似 Bernoulli，准确率 $\hat{p} = c/N$ 的标准误近似

$$SE \approx \sqrt{\frac{\hat{p}(1-\hat{p})}{N}}. \quad (22.12)$$

大样本 95% 正态近似区间：

$$\hat{p} \pm 1.96SE. \quad (22.13)$$

小样本或接近 0/1 时，Wilson 区间更稳健。评测结果 80.1% 对 80.5% 若区间高度重叠，不能断言显著更好。

例：100 题的统计波动

100 题答对 80 题， $\hat{p} = 0.8$ ：

$$SE \approx \sqrt{0.8 \times 0.2 / 100} = 0.04.$$

95% 正态近似约为 0.8 ± 0.0784 ，即 72.2%-87.8%。样本少时几个题即可改变排名。

22.9 配对评测与 Bootstrap

同一批题比较模型 A/B 时，结果是配对的。应统计每题差值 $d_i = s_i^A - s_i^B$ ，对差值 bootstrap 重采样，获得平均差的置信区间。配对设计消除了题目难度共同波动，通常比较两个独立均值更有统计效率。

22.10 人工评测

开放回答可让标注员按帮助性、正确性、风格等维度评分或成对选择。需要：

- 明确 rubric 和示例；
- 隐藏模型身份、随机答案顺序；
- 多标注员和一致性检查；
- 允许平局与“不确定”；
- 分离事实核查与主观偏好；
- 记录标注群体和语言背景。

22.11 LLM-as-a-Judge

使用强模型评估答案可扩展开放式评测，但存在：

- 位置偏差：偏好先出现的答案；
- 长度/风格偏差：偏好更长或更有格式的答案；
- 自我偏好：偏好相似模型风格；
- 提示注入：候选答案影响 Judge 指令；
- 知识错误：Judge 自身无法核实；
- 标尺漂移：不同批次和版本不一致。

缓解方式：交换顺序、独立 rubric、多 Judge、引用/执行验证、人工校准集和置信分析。

22.12 成对比较与 Bradley-Terry 模型

若模型 i 的潜在能力为 r_i ，其战胜模型 j 的概率：

$$P(i \succ j) = \sigma(r_i - r_j). \quad (22.14)$$

给定对战结果，最大化对数似然估计 r_i 。Elo 是一种在线更新近似；Bradley-Terry 可批量拟合并给出不确定性。

排名依赖对手分布和提示分布。模型 A 对 B 占优、B 对 C 占优，不保证 A 对 C 的优势同样大；领域切片可能发生排名反转。

22.13 多维指标与 Pareto 前沿

模型可能在准确率、延迟、成本和安全之间权衡。不要强行用一个加权总分掩盖差异。若不存在一个方案在所有维度都更好，可报告 **Pareto frontier**：无法在不损伤某维的情况下继续改善另一维的方案集合。

22.14 评测集设计

- 明确目标用户和真实分布；
- 覆盖常见、长尾、边界和对抗样本；
- 难度分层，避免全部过易或全部极难；
- 为每题保存来源、标签依据和更新时间；
- 建立训练/开发/保密测试；
- 对动态知识使用时间切片和持续更新。

22.15 切片分析

总平均可能隐藏关键退化。常见切片：

- 语种、领域、输入长度、输出长度；
- 常见与长尾；
- 是否需要工具/多步推理；
- 用户群体和权限；
- 安全类别；
- 量化前后、不同硬件和不同上下文位置。

上线决策应设关键切片最低门槛，而非只优化宏观平均。

22.16 数据污染

评测内容可能通过预训练、SFT、合成数据、Prompt 示例或检索库进入模型。污染表现：

- 逐字复现参考答案或解释；
- 在轻微改写后性能骤降；
- 对旧题远高于同难度新题；
- 输出数据集特有格式和错误。

缓解：保密测试、时间后移数据、动态生成、近重复检查、对抗改写和执行型评测。无法仅靠搜索训练语料证明绝对无污染。

22.17 Prompt 与解码敏感性

模型分数可能随 few-shot 示例、选项顺序、系统提示、temperature 和最大长度变化。可靠报告应：

- 预注册或固定 Prompt；
- 对合理模板做敏感性分析；
- 多随机 seed 报均值和方差；
- 不在测试集上反复调 Prompt；
- 公开完整评测配置。

22.18 线上 A/B Test

离线分数不能完全预测用户价值。线上实验需定义主要指标、护栏指标和实验单位：

- 用户满意、任务完成、留存或转化；
- 延迟、错误、成本和投诉；
- 安全事件和人工升级；
- 按用户随机，避免同用户跨组污染；
- 控制学习效应、网络效应和时间趋势。

显著性之外还要看效应大小和业务价值。

22.19 本章综合练习

练习 1：统计显著性

模型 A 在 100 题上 82%，模型 B 80%。能否仅据 2 个百分点断言 A 更好？

解答

不能。需要逐题配对结果和置信区间/显著性检验。100 题的随机波动通常大于 2 个百分点；还需检查差异是否集中在特定切片。

练习 2：Judge 偏差

交换 A/B 答案顺序后，Judge 胜率从 A 60% 变为 A 45%。说明什么？

解答

存在显著位置敏感性，原结果不能直接作为模型偏好。应随机/双向评审、计算一致性，并用人工或独立证据校准。

本节要点

可靠评测必须固定任务分布、系统协议、解码和预算，并报告统计不确定性。**Pass@k** 衡量多次采样至少一次成功；置信区间和配对 **bootstrap** 防止把随机波动当改进；**LLM Judge** 和排行榜需要校准偏差；污染、**Prompt** 调参和切片平均会造成虚高。最终应形成组件诊断、离线能力、人工偏好和线上价值的多层证据链。

幻觉、事实性与不确定性

23.1 什么是幻觉

幻觉通常指模型生成了看似合理但缺乏输入或事实支持的内容。可分为：

- **内在幻觉**：与给定上下文矛盾；
- **外在幻觉**：生成上下文没有支持、外部也未必真实的信息；
- **引用幻觉**：伪造来源、链接、论文或证据关系；
- **工具幻觉**：声称执行了未执行的操作或误读结果。

23.2 为什么会幻觉

下一 Token 训练目标奖励“在语料分布中像正确文本”，而不是实时查询真值。原因还包括数据冲突、知识过期、问题歧义、解码采样、上下文不足、检索噪声和对齐中对“总要给答案”的偏好。

23.3 校准

若模型对 100 个声称 80% 置信的问题约答对 80 个，则较好校准。生成模型的 token 概率不等同于答案级置信度。可使用多次采样一致性、语言化置信、外部 verifier、温度缩放或专门校准模型，但都需在目标分布验证。

23.4 拒答与选择性预测

允许模型在不确定时拒答，可提高已回答样本的准确率。应评估**风险-覆盖曲线**：覆盖率越高通常错误率也上升。安全系统需要区分“知识不足拒答”和“政策拒答”。

23.5 减少幻觉的手段

1. 明确输入范围和“证据不足则说明”；
2. 使用 RAG 并验证引用支持；
3. 对关键事实调用数据库、计算器或搜索；
4. 将复杂任务分解并执行中间校验；

5. 使用结构化输出和 schema;
6. 对候选做事实核查或规则验证;
7. 训练拒答、引用和不确定性样本;
8. 限制模型执行高风险副作用。

23.6 引用不是事实性的充分条件

回答附带真实文档编号，仍可能断章取义或引用与结论无关。**Citation Correctness**至少包括：来源存在、段落支持主张、引用范围准确、时间版本适用。可逐句建立 claim-evidence 对齐。

常见误区

让模型“不要幻觉”不是可靠控制。提示只能改变概率，不能提供真值。需要外部证据、可执行验证、拒答和人工审批共同构成保障。

本节要点

幻觉源于生成目标与事实目标不一致。降低风险的核心不是让模型更自信地思考，而是引入证据、验证、校准、拒答和副作用控制。

LLM 与 Agent 安全基础

24.1 威胁建模

安全设计先识别：资产、攻击者、入口、信任边界、可用工具和最坏后果。LLM 应用的攻击面不仅是聊天框，还包括检索文档、网页、邮件、插件、模型供应链、训练数据和日志系统。

24.2 Prompt Injection

直接提示注入由用户输入恶意指令；**间接提示注入**藏在模型读取的网页、文档、邮件或工具结果中。根因是模型无法天然区分“数据内容”和“可信控制指令”。

缓解措施：

- 不把 system prompt 当作权限边界；
- 将外部内容显式标记为不可信数据；
- 工具执行前独立授权和参数校验；
- 对敏感动作要求用户确认或策略引擎批准；
- 限制模型可读数据和可写资源；
- 检测数据外传、跨域请求和异常工具链。

24.3 Jailbreak 与安全对齐绕过

Jailbreak 通过角色扮演、编码、分步诱导、多轮上下文或对抗后缀绕过模型拒答。仅依赖模型内置对齐会出现漏判和误拒。应使用输入输出分类、策略引擎、速率限制、行为监控和人工复核。

24.4 越权工具调用

Agent 的高风险在于模型输出可转化为外部动作。常见问题：

- 任意命令或 SQL 注入；
- 路径遍历和文件覆盖；
- SSRF、访问内网或云元数据；
- 邮件、支付、删除等动作未经确认；
- 使用过度权限的 API key；

- 混淆用户身份或跨租户访问。

执行器必须使用 `allowlist`、类型检查、沙箱、最小权限、幂等键和审计日志。

24.5 数据与隐私风险

风险包括训练数据记忆、提示日志泄露、RAG 权限错误、`embedding` 反推、缓存串租户和供应商数据保留。需要数据最小化、脱敏、加密、访问控制、删除机制、保留策略和供应商协议评估。

24.6 模型供应链

下载模型权重、`tokenizer`、`custom code` 和量化文件时，应检查来源、许可证、哈希、序列化格式和依赖。某些模型仓库要求执行远程自定义代码，可能带来任意代码执行风险。优先使用安全序列化格式和隔离环境。

24.7 训练阶段攻击

- 数据投毒与后门；
- 偏好标注操纵；
- 评测污染；
- 恶意 `adapter` 或模型合并；
- 奖励模型漏洞和 `reward hacking`；
- 梯度/更新泄露隐私。

训练数据应有来源追踪、异常检测、审计样本和红队验证。

24.8 模型攻击与知识产权

攻击者可能进行模型抽取、成员推断、提示窃取、系统提示泄露和拒绝服务。缓解包括速率限制、输出监控、水印/指纹研究、差分隐私（有质量代价）、敏感信息过滤和服务隔离。

24.9 安全架构的分层防御

1. 身份与权限：谁能访问什么；
2. 输入处理：分类、反注入、内容边界；
3. 模型策略：拒答和安全后训练；
4. 编排控制：工具 `allowlist`、预算和状态机；
5. 执行隔离：沙箱、网络策略、只读文件系统；
6. 输出控制：敏感信息、事实和格式检查；

7. 监控响应：审计、告警、回滚和事件处理。

常见误区

“模型已经做过安全对齐”不能替代传统安全控制。模型输出永远应视为不可信建议，尤其不能直接拼接进 `shell`、`SQL`、`HTML`、文件路径或高权限 `API`。

本节要点

LLM 安全的核心是信任边界：用户、检索内容、工具结果和模型输出都可能不可信。安全能力必须由权限、验证、沙箱、监控和模型对齐共同提供。

红队、鲁棒性与上线安全评测

25.1 红队目标

红队不是只寻找“模型说了不该说的话”，还应覆盖：越权操作、数据外传、错误自动化、资源耗尽、工具链操纵、隐私泄露和业务逻辑绕过。

25.2 测试维度

- 单轮与多轮越狱；
- 多语言、编码、拼写和分隔符变体；
- 间接提示注入；
- 工具参数注入和返回值污染；
- RAG 权限与跨租户；
- 长上下文指令冲突；
- 模型拒答过度造成的可用性损失；
- 并发、超长输入和成本型 DoS；
- 模型升级后的安全回归。

25.3 攻击成功率与正常能力

安全评测需同时报告：Attack Success Rate、拒答率、误拒率、任务成功率和成本。把所有请求都拒绝可以获得低攻击成功率，却不是可用系统。

25.4 自动化红队

可用攻击模板、变异器、强模型生成攻击、遗传搜索和多轮 Agent 探索。自动评分器需防止被攻击文本本身注入；高风险案例应人工复核。保存完整轨迹、随机种子和模型版本。

25.5 安全回归集

每次事故和红队发现都应转化为最小复现样本，并包含正常对照。回归集要版本化，防止修复一个攻击后扩大误拒。公开攻击模板容易被训练污染，应保留隐藏测试。

25.6 上线护栏

- 高风险动作 **human-in-the-loop**;
- 资金、删除、发布动作双重确认;
- 工具级配额和速率限制;
- 异常调用链检测;
- 敏感数据出站扫描;
- 审计日志与可撤销操作;
- **kill switch** 和快速回滚。

本节要点

红队要测整个系统和真实后果，而非只测文本。有效安全评测同时衡量攻击阻断与正常任务可用性，并把发现沉淀为可复现回归测试。

第六部分

第七部分：扩展能力与实践路线

多模态大模型基础

26.1 多模态的核心问题

多模态模型需要把图像、音频、视频等连续信号映射到与语言模型兼容的表示，再完成跨模态对齐和条件生成。核心问题包括：表示、对齐、融合、时空压缩和生成。

26.2 视觉编码器与 Patch Token

Vision Transformer 将图像切成 patch，每个 patch 线性投影成向量，再像文本 Token 一样处理。视觉编码器可预训练为分类、对比学习或自监督模型。CLIP 使用图像-文本对比学习，使匹配图文向量接近 [26]。

26.3 连接视觉与 LLM

常见连接方式：

- 线性/MLP projector：把视觉特征投影到 LLM 隐藏维度；
- Q-Former：用少量可学习 Query 压缩视觉特征；
- Cross-Attention：语言层显式读取视觉特征；
- 统一 Token：离散化图像或把多模态都作为序列；
- 原生多模态预训练：从早期阶段联合训练不同模态。

LLaVA 类路线通过视觉编码器、投影层和语言模型组合，并使用图文指令数据对齐 [27]。

26.4 图像分辨率与视觉 Token

更高分辨率意味着更多 patch 和 Token，增加上下文和注意力成本。常见策略包括动态切图、缩略图加局部高分辨率、区域选择、Token 压缩和多尺度编码。

26.5 音频与语音

语音系统可分为 ASR + LLM + TTS 级联，也可把音频编码器直接接入 LLM，或进行端到端 speech-to-speech。关键指标除文本正确性外，还包括延迟、说话人、情感、韵律和打断处理。

26.6 视频

视频同时包含空间和时间维度，Token 数极大。需要帧采样、时序池化、关键片段检索和长期事件记忆。视频问答的失败常来自未采到关键帧，而不是语言推理。

26.7 多模态幻觉与安全

模型可能识别不存在的物体、误读文字、忽略局部区域或被图像中的文字提示注入。OCR、检测器、区域裁剪和多模态交叉验证可缓解。视觉内容也应视为不可信输入。

本节要点

多模态模型通过编码器和连接模块把非文本信号转成 LLM 可使用的表示。分辨率、Token 压缩和模态对齐决定成本与能力，多模态输入同样存在幻觉和注入风险。

模型选型、开源生态与许可证

27.1 选型的第一原则：从任务和约束出发

不要先问“哪个模型最好”，而应明确：任务、语言、输入/输出长度、工具需求、延迟、并发、预算、隐私、许可证、硬件和维护能力。

27.2 模型卡应读什么

- 架构、参数量、GQA/MoE 和上下文；
- Base/Instruct/Multimodal 版本；
- tokenizer 与 chat template；
- 推荐推理精度、硬件和框架；
- 训练数据说明与知识截止时间；
- benchmark 设置；
- 已知限制和安全策略；
- 权重许可证、可接受使用政策和商用条款。

27.3 开源、开放权重与开放源码

“开源模型”在实践中可能只开放权重，而训练数据、代码或许可证并不满足传统开源定义。应分别检查：权重可下载、代码可用、数据公开、训练细节公开、允许商用、允许再分发、允许衍生模型。

27.4 常见模型文件与格式

- safetensors：安全、可内存映射的张量格式；
- PyTorch checkpoint：灵活，但 pickle 形式可能执行代码；
- GGUF：面向本地 CPU/GPU 推理的量化与元数据格式；
- ONNX：跨框架计算图；
- TensorRT Engine：面向特定硬件和 shape 优化；
- adapter checkpoint：只保存 LoRA/PEFT 参数。

27.5 推理框架与训练框架的角色

训练生态侧重 autograd、分布式、checkpoint 与数据；推理生态侧重 KV 管理、批处理、量化内核和 API 服务。模型能在 Transformers 加载，不代表能在高性能服务框架中完整支持其自定义架构。

27.6 选型实验矩阵

建议至少比较：

维度	需要记录
质量	私有任务成功率、事实、格式、安全、长上下文
性能	TTFT、TPOT、吞吐、显存、并发、峰值排队
成本	API 单价或自部署单位成功任务成本
工程	框架支持、量化、工具调用、监控、升级难度
合规	数据流向、许可证、日志、区域和供应链
稳定性	多次运行方差、超时、OOM、错误恢复

常见误区

模型名称、排行榜和厂商演示都不能替代目标环境实测。尤其是工具调用格式、中文 Token 效率、长上下文、量化质量和高并发表现，往往与公开平均分不一致。

本节要点

选型是多目标优化。必须联合比较质量、性能、成本、生态和合规，并固定完整推理配置；“开放权重”也不自动等于无限制商用。

从零完成一个 LLM 项目的标准流程

28.1 第一步：定义问题

用可验证语言描述：用户是谁、输入是什么、输出是什么、允许使用哪些数据和工具、错误代价是什么。把“做一个智能助手”改写为具体任务，例如“依据内部制度回答问题，每个结论必须引用有效段落，证据不足时拒答”。

28.2 第二步：建立无模型基线

先检查关键词检索、规则、模板、传统分类器或现有业务逻辑。无模型基线有助于判断 LLM 的增量价值，也可作为故障降级方案。

28.3 第三步：建立最小模型基线

选择一个可用模型，固定提示和解码，构造 50-200 条代表性样本，记录错误类型。不要一开始就微调或搭建复杂 Agent。

28.4 第四步：错误分析

将失败分为：知识缺失、检索失败、推理失败、格式失败、工具失败、拒答错误、安全问题和性能问题。不同错误对应不同手段：知识缺失可能用 RAG；格式失败用 constrained decoding；稳定行为缺陷再考虑 SFT。

28.5 第五步：选择增强手段

- 提示可解决的，先改提示；
- 动态事实和引用，用 RAG；
- 外部计算和操作，用工具；
- 稳定格式、风格和流程，用 SFT/LoRA；
- 相对偏好和复杂策略，用 DPO/RL；
- 高风险步骤，加规则、验证和人工审批。

28.6 第六步：离线与线上闭环

离线集用于快速回归，线上日志发现分布变化。每次修改都需保存版本、运行全量回归、比较关键切片，并设定回滚阈值。

28.7 最小工程清单

1. 可复现的环境与依赖锁定；
2. 数据和评测集版本；
3. 提示、模板和工具 `schema` 版本；
4. 结构化日志与 `trace`；
5. 超时、重试和幂等；
6. 权限、密钥与沙箱；
7. 成本和 SLO 监控；
8. 回归测试与回退开关。

28.8 错误优先级

可用“影响 × 频率 × 可检测性”排序。高影响且难检测的静默错误优先于显式失败；自动执行动作的错误优先于纯文本风格问题。

本节要点

好的 LLM 项目从问题定义和评测开始，而不是从模型微调开始。先做最小基线和错误分桶，再选择提示、RAG、工具或训练，能显著减少无效复杂度。

学习路线与面试自检

29.1 四阶段学习路线

29.1.1 阶段一：会使用 and 解释

掌握 Token、Embedding、Transformer、Attention、上下文、采样、Prompt、RAG、SFT 与 LoRA。目标是能解释一次请求从字符串到输出 Token 的完整流程。

29.1.2 阶段二：会训练和评测

使用小模型完成 SFT/LoRA，理解数据 mask、学习率、显存、loss 与回归评测；搭建 RAG 并测召回与引用。

29.1.3 阶段三：会部署和优化

理解 Prefill/Decode、KV Cache、continuous batching、量化、TP/PP/FSDP，能定位 OOM、吞吐和延迟问题。

29.1.4 阶段四：会设计可靠系统

掌握工具调用、Agent 状态机、安全边界、红队、线上监控和数据飞轮，能把模型错误转化为系统性修复。

29.2 建议动手项目

1. 手写单头 Attention 和 causal mask；
2. 用小语料训练 BPE tokenizer；
3. 对 0.5B-3B 模型做 LoRA SFT；
4. 构建含 BM25、向量检索和 rerank 的 RAG；
5. 实现 JSON Schema 工具调用 Agent；
6. 对同一模型测试不同量化和并发；
7. 制作 100 条私有评测集和错误看板；
8. 对 RAG/Agent 做间接提示注入红队。

29.3 高频自检问题

1. Token 与单词有什么区别？为什么 tokenizer 影响成本？
2. Attention 的 $QK^\top/\sqrt{d_k}$ 每项分别做什么？
3. MHA、MQA、GQA 的 KV Cache 差异是什么？
4. 为什么训练可并行、生成却通常串行？
5. KV Cache 显存随哪些变量增长？
6. SFT、RLHF、DPO 分别优化什么？
7. LoRA 为什么低成本？QLoRA 量化了什么？
8. RAG 与微调如何选择？
9. Temperature、top-p 和 beam search 如何影响输出？
10. Prefill、Decode、TTFT、TPOT 有什么关系？
11. DP、TP、PP、ZeRO 各拆分什么？
12. 为什么 LLM-as-a-Judge 会偏置？
13. Prompt Injection 为什么不能只靠 system prompt 防御？
14. Agent 与普通聊天模型的本质区别是什么？
15. 上下文窗口为何不等于长期记忆？

本节要点

学习 LLM 不应按论文发布日期堆积，而应沿“表示 - 模型 - 训练 - 推理 - 应用 - 可信”建立主干，再用项目填补细节。

核心术语速查表

术语	一句话定义与辨析
LLM	以大规模参数、数据和计算训练的语言模型；产品中的聊天助手通常还包含后训练、RAG、工具和安全系统。
Foundation Model	在广泛数据上预训练、可适配多任务的基础模型，不限于文本。
Base Model	主要完成预训练、擅长续写但不保证指令遵循的模型。
Instruct/Chat Model	在基础模型上进行指令与偏好训练的模型。
Reasoning Model	经过推理数据、强化学习或验证训练，更愿意分配测试时计算的模型。
Parameter	训练中更新的权重数值；参数量不等于显存占用。
Hyperparameter	人工设定的学习率、层数、batch、rank 等配置。
Token	Tokenizer 切分出的离散单元，不一定等于字或单词。
Vocabulary	Token 到整数 ID 的映射集合。
Tokenizer	字符串与 Token ID 序列之间的可逆或近似可逆转换器。
BPE	从小单元反复合并高频相邻对的子词算法。
SentencePiece	直接在原始字符串上训练 BPE/Unigram 等 tokenizer 的工具体系。
Embedding	把离散 ID 映射为连续向量；也指用于语义检索的向量模型。
Hidden State	模型每层每个位置的连续表示。
Logit	Softmax 前的未归一化分数。
Softmax	将一组 logit 转成总和为 1 的概率分布。
Cross-Entropy	语言模型最常用损失，惩罚正确 Token 概率过低。
Perplexity	平均负对数似然的指数；同 tokenizer 和数据上越低通常越好。
KL Divergence	非对称的分布差异度量，对齐中常限制策略偏移。
Backpropagation	通过链式法则计算参数梯度。
Gradient Accumulation	多个 micro-batch 累积梯度后更新，模拟更大全局 batch。
Mixed Precision	组合低精度计算与高精度关键状态以加速训练。
BF16	指数范围接近 FP32 的 16 位浮点，LLM 训练常用。
FP16	尾数比 BF16 多但指数范围较小的 16 位浮点。
Transformer	以注意力、MLP、残差和归一化为核心的序列模型架构。
Encoder-only	双向读取上下文，偏向理解和表示任务。
Decoder-only	使用 causal mask 自回归生成，现代通用 LLM 主流。

术语	一句话定义与辨析
Encoder-Decoder	编码输入，再由解码器条件生成。
Self-Attention	Q、K、V 来自同一序列的注意力。
Cross-Attention	Query 与 Key/Value 来自不同序列或模态。
Query/Key/Value	注意力中的查询、匹配索引和被聚合内容。
Causal Mask	阻止当前位置读取未来 Token 的上三角 mask。
MHA	每个 Query head 都有独立 K/V head 的多头注意力。
MQA	所有 Query head 共享一组 K/V，显著压缩 KV Cache。
GQA	多个 Query head 分组共享 K/V，在质量和效率间折中。
FFN/MLP	Transformer 每层对各 Token 独立执行的非线性前馈网络。
SwiGLU	使用 SiLU 门控的 FFN 结构，现代 LLM 常见。
Residual Connection	令输出为 $x + F(x)$ ，改善深层网络训练和信息传播。
LayerNorm	对隐藏维度减均值、除标准差并仿射变换。
RMSNorm	只按均方根缩放的归一化，计算更简洁。
RoPE	旋转 Q/K 以编码相对位置的常用位置编码。
Context Window	单次请求可处理的总 Token 预算，不等于永久记忆。
Long Context	模型可接收并有效利用长序列的综合能力。
KV Cache	自回归推理中缓存历史 K/V，避免重复计算。
Prefill	一次处理输入提示并建立 KV Cache 的阶段。
Decode	基于 KV Cache 逐步生成新 Token 的阶段。
FlashAttention	通过 IO-aware 分块优化实现精确注意力。
PagedAttention	将 KV Cache 按页管理以减少碎片和支持共享。
MoE	每个 Token 只激活少量专家的稀疏模型。
Router	MoE 中为 Token 选择专家的门控网络。
Active Parameters	MoE 每 Token 实际参与计算的参数，不等于总参数。
Pretraining	在大规模数据上用自监督目标学习通用能力。
Continued Pre-training	在已有模型上继续语言建模以适配新领域或分布。
Scaling Law	参数、数据、计算和损失之间的经验缩放关系。
SFT	使用指令与标准回答进行监督微调。
Instruction Tuning	用多任务自然语言指令数据训练指令遵循能力。
RLHF	从人类偏好训练奖励并用强化学习优化模型。
Reward Model	将提示和回答映射到标量偏好分数的模型。
PPO	使用概率比和裁剪限制策略更新幅度的 RL 算法。
Advantage	动作回报相对基线的增益估计。
DPO	直接从优劣回答对优化策略相对参考模型的概率。
RLAIF	使用 AI 反馈替代或补充人工偏好的对齐训练。

术语	一句话定义与辨析
RLVR	使用可自动验证结果作为奖励的强化学习。
Reward Hacking	模型优化代理奖励却违背真实目标。
PEFT	只训练少量参数的高效微调方法总称。
LoRA	用低秩矩阵表示权重更新的 PEFT 方法。
QLoRA	量化冻结基础权重并训练 LoRA 的低显存方法。
Adapter	插入小型瓶颈模块并只训练该模块。
Prompt Tuning	学习少量连续虚拟 Token，而非修改主权重。
Distillation	让学生模型学习教师输出、logit 或中间表示。
Quantization	用更低 bit 表示权重或激活以降低存储和带宽。
PTQ	训练完成后进行量化，通常依赖校准数据。
QAT	训练中模拟量化误差，使模型适应低精度。
Weight-only Quantization	只量化权重，常用于减轻 decode 权重带宽。
KV Quantization	量化 KV Cache，提升长上下文并发能力。
Prompt Engineering	通过输入结构和示例明确条件，不更新模型权重。
In-Context Learning	模型根据上下文示例临时适应任务。
Few-shot	在提示中加入少量任务示例。
Temperature	缩放 logit，控制采样分布尖锐度。
Top-k	只在概率最高的 k 个 Token 中采样。
Top-p	在累计概率达到阈值的最小集合中采样。
Beam Search	同时保留若干累计概率高的候选序列。
Constrained Decoding	用语法或状态机限制可生成 Token，保证结构合法。
Chain-of-Thought	使用中间推理步骤解决多步问题。
Self-Consistency	采样多条推理路径并聚合最终答案。
Test-Time Compute	在推理阶段增加候选、搜索、验证或思考预算。
Verifier	对答案或过程进行规则、执行或模型验证的组件。
RAG	在生成前检索外部知识并加入上下文。
Chunking	将文档切成可索引和放入上下文的片段。
Dense Retrieval	使用向量语义相似度检索。
Sparse Retrieval	使用词项匹配和倒排索引检索，如 BM25。
Hybrid Retrieval	融合稀疏与向量检索。
Reranker	对初召文档进行更精确的二阶段排序。
ANN	近似最近邻搜索，在召回率和速度间权衡。
Agent	能观察、选择动作、调用工具并根据反馈循环的系统。
Function Calling	模型按 schema 输出工具名称和参数的机制。
ReAct	推理与动作交替的 Agent 模式。

术语	一句话定义与辨析
Working Memory	Agent 显式维护的当前任务状态和中间结果。
Semantic Memory	可按语义检索的长期事实或规则记忆。
Episodic Memory	历史任务轨迹和经验记录。
Tool Sandbox	隔离工具副作用和资源访问的执行环境。
TTFT	请求到首 Token 的延迟。
TPOT	Decode 阶段每个输出 Token 的时间。
Continuous Batching	解码过程中动态加入和移除请求的批处理。
Throughput	单位时间处理的请求或 Token 数。
Goodput	满足延迟等 SLO 的有效吞吐。
Speculative Decoding	草稿模型提出 Token、目标模型批量验证的加速方法。
Data Parallelism	各卡保存模型副本、处理不同数据并同步梯度。
Tensor Parallelism	在单层矩阵内部切分计算。
Pipeline Parallelism	按层切分模型并让 micro-batch 流水执行。
ZeRO/FSDP	在数据并行设备间切分优化器、梯度和参数状态。
Expert Parallelism	将 MoE 专家分布到不同设备。
Activation Checkpointing	丢弃部分激活并在反向重算，以计算换显存。
Offload	把模型状态迁移到 CPU/NVMe，以带宽换显存。
Benchmark	固定任务、指标和设置下的能力测量。
LLM-as-a-Judge	使用模型评估开放回答，需处理多种偏差。
Pass@k	采样 k 个候选时至少一个通过验证的概率。
Data Contamination	评测内容进入训练或提示导致分数虚高。
Hallucination	生成缺乏证据或与事实/上下文冲突的内容。
Calibration	模型置信度与实际正确率的一致程度。
Prompt Injection	通过输入或外部内容劫持模型指令层级。
Jailbreak	绕过模型安全策略的对抗提示。
Least Privilege	工具和模型只获得完成任务所需最小权限。
Human-in-the-loop	在高风险或不确定步骤引入人工决策。
Guardrail	模型之外的输入、输出、工具和策略控制。
Multimodal Model	联合处理文本、图像、音频、视频等模态的模型。
Vision Encoder	把图像转换为视觉特征或视觉 Token 的网络。
Projector	将视觉特征映射到语言模型隐藏空间的连接层。

关键公式速查

自回归分解

$$P(x_{1:T}) = \prod_{t=1}^T P(x_t \mid x_{<t}).$$

Softmax 与温度

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

交叉熵与困惑度

$$\mathcal{L} = - \sum_t \log P(x_t \mid x_{<t}),$$

$$\text{PPL} = \exp(\mathcal{L}/T).$$

缩放点积注意力

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V.$$

LoRA

$$W = W_0 + \frac{\alpha}{r} BA.$$

奖励模型偏好损失

$$\mathcal{L}_{RM} = -\log \sigma(r(x, y_w) - r(x, y_l)).$$

PPO 概率比与裁剪目标

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{old}(a_t \mid s_t)},$$

$$L^{clip} = \mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)].$$

余弦相似度

$$\cos(q, d) = \frac{q^\top d}{\|q\|_2 \|d\|_2}.$$

KV Cache 粗略显存

$$M_{KV} \approx B \cdot L \cdot N_{layer} \cdot 2 \cdot H_{kv} \cdot d_{head} \cdot s.$$

全局 Batch

$$B_{global} = B_{micro} \cdot N_{DP} \cdot N_{accum}.$$

Softmax-交叉熵梯度

$$\nabla_z \ell = \mathbf{p} - \mathbf{y}.$$

RoPE 相对位置性质

$$(R_t q)^\top (R_s k) = q^\top R_{s-t} k.$$

策略梯度与 Baseline

$$\nabla_\theta J = \mathbb{E}[(R - b) \nabla_\theta \log \pi_\theta(y | x)].$$

LoRA 参数量

$$P_{LoRA} = r(d_{in} + d_{out}), \quad \Delta W = \frac{\alpha}{r} BA.$$

均匀仿射量化

$$\begin{aligned} s &= \frac{x_{max} - x_{min}}{q_{max} - q_{min}}, \\ q &= \text{clip}(\text{round}(x/s) + z, q_{min}, q_{max}), \\ \hat{x} &= s(q - z). \end{aligned}$$

Ring All-Reduce 每 Rank 通信量

$$V = 2 \frac{P-1}{P} S.$$

Pipeline Bubble (简化 GPipe)

$$f_{bubble} = \frac{P-1}{M+P-1}.$$

精选参考文献

参考文献

- [1] Vaswani, A. et al. *Attention Is All You Need*. NeurIPS, 2017.
- [2] Devlin, J. et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL, 2019.
- [3] Brown, T. et al. *Language Models are Few-Shot Learners*. NeurIPS, 2020.
- [4] Kaplan, J. et al. *Scaling Laws for Neural Language Models*. arXiv:2001.08361, 2020.
- [5] Hoffmann, J. et al. *Training Compute-Optimal Large Language Models*. NeurIPS, 2022.
- [6] Su, J. et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. Neurocomputing, 2024; arXiv:2104.09864.
- [7] Zhang, B. and Sennrich, R. *Root Mean Square Layer Normalization*. NeurIPS, 2019.
- [8] Dao, T. et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022.
- [9] Kwon, W. et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023.
- [10] Fedus, W. et al. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. JMLR, 2022.
- [11] Ouyang, L. et al. *Training Language Models to Follow Instructions with Human Feedback*. NeurIPS, 2022.
- [12] Schulman, J. et al. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347, 2017.
- [13] Rafailov, R. et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS, 2023.
- [14] Bai, Y. et al. *Constitutional AI: Harmlessness from AI Feedback*. arXiv:2212.08073, 2022.
- [15] Hu, E. et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.
- [16] Dettmers, T. et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS, 2023.
- [17] Wei, J. et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS, 2022.

-
- [18] Wang, X. et al. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. ICLR, 2023.
 - [19] Yao, S. et al. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS, 2023.
 - [20] Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.
 - [21] Yao, S. et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR, 2023.
 - [22] Schick, T. et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS, 2023.
 - [23] Hendrycks, D. et al. *Measuring Massive Multitask Language Understanding*. ICLR, 2021.
 - [24] Liang, P. et al. *Holistic Evaluation of Language Models*. TMLR, 2023.
 - [25] Zheng, L. et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS Datasets and Benchmarks, 2023.
 - [26] Radford, A. et al. *Learning Transferable Visual Models From Natural Language Supervision*. ICML, 2021.
 - [27] Liu, H. et al. *Visual Instruction Tuning*. NeurIPS, 2023.
 - [28] Rajbhandari, S. et al. *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. SC, 2020.
 - [29] Shoeybi, M. et al. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. arXiv:1909.08053, 2019.
 - [30] Kudo, T. and Richardson, J. *SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer*. EMNLP, 2018.

结语

理解 LLM 的关键，不是记住越来越多的缩写，而是建立稳定的因果链：文本经 **Tokenizer** 变成 **ID**，经 **Embedding** 和 **Transformer** 形成下一个 **Token** 分布；预训练学习通用规律，**SFT** 和偏好优化塑造行为；推理系统用 **KV Cache**、批处理和量化把模型变成服务；**RAG** 和 **Agent** 把外部知识与动作接入；评测与安全决定系统是否可信。

当遇到新方法时，可以用五个问题快速定位：

1. 它改变的是数据、权重、推理还是编排？
2. 它优化的直接目标是什么？
3. 它把成本从哪里转移到了哪里？
4. 它依赖什么假设，在哪些分布上可能失效？
5. 应该用什么独立证据验证它确实有效？

只要这五个问题能够回答，新术语就能被放回已有知识地图，而不会成为孤立的名词。